

CUDA Kernel 面试背题笔记

本书整理自 LeetCUDA 项目，系统汇编了 CUDA 面试中最高频的 GPU Kernel 实现，涵盖从 Warp/Block 原语、Elementwise、Softmax、GEMM、FlashAttention 等面试核心知识体系。

每个 Kernel 配备详细的中文面试要点注释 (WHY + HOW)，并以递进式优化路线 (naive → tiling → vectorize → tensor core → warp specialized) 展示 GPU 性能调优思维。

30 个 Kernel · 9 个 Phase · 全中文注释

项目主页: <https://github.com/xlite-dev/LeetCUDA>

更新日期: 2026 年 7 月 14 日

```

1 // =====
2 // notes-v2.cu — CUDA Kernel 面试刷题笔记
3 // =====
4 //
5 // 整理自 LeetCUDA 项目 (https://github.com/xlite-dev/LeetCUDA) , 涵盖:
6 //   - 面试高频 CUDA kernel 的完整实现 (~30 个 kernel)
7 //   - 每类 kernel 附带详细的面试要点注释 (WHY + HOW)
8 //   - 优化技术的递进式讲解 (naive → tiling → vectorize → tensor core → ws)
9 //   - BLAS 语义: N=col-major(Normal), T=row-major(Transposed)
10 //
11 // 10 个 Phase 覆盖:
12 //   Phase 0 — 面试框架速查 (GPU 架构 / Memory Hierarchy / Roofline / 优化清单)
13 //   Phase 1 — 基础原语: Warp Reduce / Block Reduce / Dot Product (含 broadcast 增强版)
14 //   Phase 2 — Elementwise: ReLU / Elementwise Add / Histogram (基础 + float4 向量化 + atomic)
15 //   Phase 3 — Softmax: naive → safe → online + RMS/Layer Norm
16 //   Phase 4 — RoPE: 旋转位置编码 (Llama 风格 theta=10000)
17 //   Phase 5 — Mat Transpose: 基础版 + BCF merge_write 最佳版 (Bank Conflict专题)
18 //   Phase 6 — GEMV: SGEMV K32/K128/K16 (warp-per-row)
19 //   Phase 7 — GEMM *: SGEMM → HGEMM → MMA m16n8k16(TN布局) → WGMMMA m64n128k16
20 //   Phase 8 — FlashAttention split_q (FA-2, 含 online softmax + P@V 寄存器复用)
21 //
22 // =====
23 // Phase 0: 面试框架速查 (纯注释, 面试开场必备的基础知识)
24 // =====
25
26 // ---- GPU 架构速查 ----
27 //
28 // SM (Streaming Multiprocessor) 内部结构:
29 //   - Warp Scheduler x4: 每 SM 4 个 warp scheduler, 每个每周期可发射 1 条指令
30 //   - Register File: 每 SM 65536 × 32-bit (4 bytes) = 256KB
31 //   - Shared Memory / L1: 可配置, 最大 shared memory ~228KB (Hopper)
32 //   - Tensor Cores: Hopper 每 SM 4 个; Blackwell 数量随型号/定义不同, 建议以官方 ISV guide 为准
33 //   - Warp = 32 threads: 最小调度单元, SIMT 执行模型
34 //
35 // Memory Hierarchy 带宽数量级 (H100 参考):
36 //   HBM3: ~3.35 TB/s (理论), 实际 ~2.5-3.0 TB/s
37 //   L2 Cache: ~12 TB/s (50MB, 跨 SM 共享)
38 //   L1/SMEM: ~19 TB/s (每 SM ~228KB)
39 //   Register: ~0 延迟, ~100+ TB/s 等效带宽
40 //
41 // 关键瓶颈判断:
42 //   Memory-bound: AI (Arithmetic Intensity) < 机器 FLOPS/带宽 比值
43 //   Compute-bound: AI 足够大, 受限于计算吞吐
44 //   Latency-bound: 线程不够多, 无法隐藏内存延迟
45 //
46 // Occupancy 公式:
47 //   occupancy = active_warps / max_warps_per_SM
48 //   受三类资源分别取下限: 每线程寄存器数 → threads/SM; 每 block shared memory
49 //   → blocks/SM; block 大小 → blocks/SM
50 //
51 // ---- 常见优化手段速查清单 ----
52 //
53 // 1. Coalesced Memory Access (合并访问)
54 //   - 同一 warp 的线程访问连续的 128B 对齐地址 → 1 次内存事务
55 //   - 否则产生多次事务 (最坏 32 次)
56 //
57 // 2. Tiling (分块)
58 //   - 将数据从 HBM 分块加载到 shared memory 复用, 减少 HBM 访问
59 //   - GEMM: Block Tile (BM×BN) + K Tile (BK)
60 //
61 // 3. Vectorized Memory Access (向量化)
62 //   - 使用 float4/half2 等向量类型, 减少 load/store 指令数

```

```

63 // - float4 = 128-bit, 单条指令加载 16 bytes
64 //
65 // 4. Thread Tile (寄存器分块)
66 // - 每个线程计算多个输出元素 (TM×TN), 提高计算密度
67 // - 减少线程总数, 降低同步开销
68 //
69 // 5. Bank Conflict Avoidance
70 // - Shared memory 有 32 banks × 4 bytes
71 // - 同 warp 多线程访问同一 bank 的不同地址 → bank conflict → 串行化
72 // - 解决方案: PAD (在每行末尾加 1 个元素打破对齐)
73 //
74 // 6. Pipeline / Double Buffering (流水线)
75 // - cp.async 异步拷贝下一批数据时同时做当前批的计算
76 // - Stage 数 = 2/3/4, 权衡 shared memory 占用和延迟隐藏
77 //
78 // 7. Tensor Core (MMA / WGMMMA)
79 // - MMA m16n8k16 (Ampere): warp 级指令, 单 warp 完成 16×8×16 的矩阵乘
80 // - WGMMMA m64n128k16 (Hopper): warpgroup 级指令 (128 threads), 异步执行
81 //
82 // 8. Warp Specialization (Hopper+)
83 // - Producer warpgroup 做 TMA 数据搬运, Consumer warpgroup 做计算
84 // - 通过 cuda::barrier 同步, 完全解耦数据搬运和计算
85 //
86 // 9. TMA (Tensor Memory Accelerator, Hopper+)
87 // - 硬件 DMA 引擎, 支持 1D~5D 寻址, 低寄存器开销
88 // - 配合 cp.async.bulk 实现异步数据搬运
89
90 // ---- Roofline 分析公式 ----
91 //
92 // AI (Arithmetic Intensity) = FLOPs / Bytes_transferred
93 //
94 // GEMM (M=N=K=4096):
95 // FLOPs = 2 × M × N × K = 2 × 40963 ≈ 137 GFLOPs
96 // Bytes = (M×K + K×N + M×N) × sizeof(float) ≈ 200 MB
97 // AI ≈ 137G / 200M ≈ 685 FLOPs/Byte → compute-bound (远超 H100 ridge point:
98 // FP16 TC ≈ 295:1, FP32 ≈ 20:1)
99 //
100 // GEMV (M=4096, K=4096):
101 // Bytes = (M×K + K + M) × sizeof(float) ≈ 67 MB
102 // AI ≈ 33M / 67M ≈ 0.5 FLOPs/Byte → severely memory-bound
103 //
104 // Softmax (N=4096): AI ≈ (5×N) / (2×N×4) = 5/8 ≈ 0.625 FLOPs/Byte → memory-bound
105
106 // =====
107 // Phase 1: 头文件 + 宏定义 + 基础原语 (Warp Reduce / Block Reduce)
108 // =====
109 // 面试要点:
110 // - warp_reduce: 用 __shfl_xor_sync 做蝶形归约, O(logN) 步, 无需 shared
111 //   memory
112 // - block_reduce: 两级归约 (warp → shared memory → warp0 broadcast),
113 //   注意最后必须 broadcast 回所有线程 (__shfl_sync), 否则只有 warp0 知道结果
114 // - 为什么不用 __shfl_down_sync? xor 模式所有线程做相同工作量, 更均衡
115
116 #include <algorithm>
117 #include <cstring>
118 #include <cuda_fp16.h>
119 #include <cuda_runtime.h>
120 #include <float.h>
121 #include <stdio.h>
122 #include <stdlib.h>
123 #include <math.h>
124 #include <cublas_v2.h>
125 #include <cuda.h>

```

```

126 #include <cuda/barrier>
127
128 #define WARP_SIZE 32
129 #define INT4(value) (reinterpret_cast<int4*>(&(value)))[0])
130 #define FLOAT4(value) (reinterpret_cast<float4*>(&(value)))[0])
131 #define HALF2(value) (reinterpret_cast<half2*>(&(value)))[0])
132
133 // =====
134 // Phase 1a: Warp Reduce (warp 内归约, 纯寄存器操作, 无需 shared memory)
135 // =====
136
137 // Warp Reduce Sum — generic (used by both FP32 and FP16 contexts)
138 // 使用 __shfl_xor_sync 做蝶形归约 (butterfly reduction)
139 // 复杂度  $O(\log N)$ ,  $N=32$  时仅需 5 步
140 // 模板参数: T=数据类型, kWarpSize=segment width (默认 32)
141 // kWarpSize 会作为 __shfl_xor_sync 的第 4 个实参 width, 限制 shuffle 在同一 segment 内
142 // 当 kWarpSize < 32 (如 FA 中 kWarpSize=4) 时, 只有同 segment 的 lane 参与通信
143 // 蝶形归约示意 (以 warpSize=8 为例, 实际 warpSize=32 有 5 次迭代):
144 //
145 // 初始: 每个 lane 持有自己的值 v0..v7
146 // lane:  0    1    2    3    4    5    6    7
147 // val:  v0   v1   v2   v3   v4   v5   v6   v7
148 //
149 // mask=4 (第1次迭代, lane i 与 lane i^4 交换并累加):
150 //
151 //      ┌──────────┐
152 // 对:   (0,4) (1,5) (2,6) (3,7)
153 //
154 // lane:  0    1    2    3    4    5    6    7
155 // val:  v0+v4 v1+v5 v2+v6 v3+v7 v4+v0 v5+v1 v6+v2 v7+v3
156 //
157 // mask=2 (第2次迭代, lane i 与 lane i^2 交换并累加):
158 //
159 //      ┌───┐ ┌───┐ ┌───┐ ┌───┐
160 // 对:   (0,2) (1,3) (4,6) (5,7)
161 //      ─── 前4个一组 ───      ─── 后4个一组 ───
162 //
163 // lane:  0    1    2    3    4    5    6    7 (每lane持有前一轮2个值的和再加本轮配对)
164 // val:   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$ 
165 //      = v0+v2+v4+v6 ... (逐步归约)
166 //
167 // mask=1 (第3次迭代, lane i 与 lane i^1 交换并累加):
168 //
169 //      ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐
170 // 对:   (0,1) (2,3) (4,5) (6,7)
171 //
172 // lane:  0    1    2    3    4    5    6    7
173 // val:   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$  ← 所有 lane 拥有全归约结果!
174 //
175 // mask=0: 循环终止, 归约完成。
176 //
177 // 关键性质:
178 // - XOR 配对是对称的: lane i 的配对对象是 lane  $i^{mask}$ , 而  $(i^{mask})^{mask} = i$ 
179 // - 每轮每个 lane 只和恰好 1 个其他 lane 通信 (一对一, 无冲突)
180 // - 每轮信息传递距离减半:  $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$  (距离减半, 信息翻倍)
181 // -  $O(\log_2 N)$  步完成, 无需 shared memory, 纯寄存器操作
182 // - __shfl_xor_sync 第四个参数 kWarpSize 限制 segment width:
183 //   当 kWarpSize=4 时, 只有同 segment(4个一组)内的 lane 参与 shuffle
184
185 template <const int kWarpSize = WARP_SIZE, typename T = float>
186 __device__ __forceinline__ T warp_reduce_sum(T val) {
187 #pragma unroll
188     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
189         val += __shfl_xor_sync(0xffffffff, val, mask, kWarpSize);
190     }
191     return val;
192 }

```

```

189
190 // Warp Reduce Max — generic
191 template <const int kWarpSize = WARP_SIZE, typename T = float>
192 __device__ __forceinline__ T warp_reduce_max(T val) {
193 #pragma unroll
194     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
195         val = max(val, __shfl_xor_sync(0xffffffff, val, mask, kWarpSize));
196     }
197     return val;
198 }
199
200 // =====
201 // Phase 1b: Block Reduce (block 内归约, 两级: warp → shared memory → warp reduce)
202 // =====
203
204 // Block Reduce Sum — FP32 (增强版, 带 broadcast)
205 // 两级归约流程:
206 // 1. 每个 warp 内做 warp_reduce_sum → 得到每 warp 的一个值
207 // 2. warp leader (lane=0) 写入 shared memory
208 // 3. syncthreads 后, lane 0~kNumWarps-1 读取 shared memory
209 // 4. 所有 warp 内再做一次 warp_reduce<kNumWarps> → 得到最终结果
210 // 5. __shfl_sync broadcast 到所有线程 (关键! 否则每个warp只有lane<kNumWarps 知道结果)
211 template <const int kNumThreads = 256>
212 __device__ float block_reduce_sum(float val) {
213     constexpr int kNumWarps = (kNumThreads + WARP_SIZE - 1) / WARP_SIZE;
214     int warp = threadIdx.x / WARP_SIZE;
215     int lane = threadIdx.x % WARP_SIZE;
216     __shared__ float shared[kNumWarps];
217
218     float value = warp_reduce_sum<WARP_SIZE>(val);
219     if (lane == 0)
220         shared[warp] = value;
221     __syncthreads();
222     value = (lane < kNumWarps) ? shared[lane] : 0.0f;
223     value = warp_reduce_sum<kNumWarps>(value);
224     // 关键: broadcast 结果到所有线程, 后续用 result
225     // 做除法等操作时所有线程都能拿到
226     value = __shfl_sync(0xffffffff, value, 0, 32);
227     return value;
228 }
229
230 // Block Reduce Max — FP32 (增强版, 带 broadcast)
231 template <const int kNumThreads = 256>
232 __device__ float block_reduce_max(float val) {
233     constexpr int kNumWarps = (kNumThreads + WARP_SIZE - 1) / WARP_SIZE;
234     int warp = threadIdx.x / WARP_SIZE;
235     int lane = threadIdx.x % WARP_SIZE;
236     __shared__ float shared[kNumWarps];
237
238     float value = warp_reduce_max<WARP_SIZE>(val);
239     if (lane == 0)
240         shared[warp] = value;
241     __syncthreads();
242     value = (lane < kNumWarps) ? shared[lane] : -FLT_MAX;
243     value = warp_reduce_max<kNumWarps>(value);
244     value = __shfl_sync(0xffffffff, value, 0, 32);
245     return value;
246 }
247
248 // ---- Block Reduce Sum All: y = sum(a[0..N-1]) ----
249 // 多 block 各自做 warp→smem→warp0 reduce, 然后 atomicAdd 到全局 y
250 // 跨 block 求和的常见模式, 适合 N 较大时使用;
251 // Grid: ((N + 255) / 256, 1, 1)

```

```

252 // Block: (256, 1, 1)
253 // source: LeetCUDA/kernels/reduce/block_all_reduce.cu
254 template <const int kNumThreads = 256>
255 __global__ void block_reduce_all(float *a, float *y, int N) {
256     int tid = threadIdx.x;
257     int idx = blockIdx.x * kNumThreads + tid;
258     constexpr int kNumWarps = (kNumThreads + WARP_SIZE - 1) / WARP_SIZE;
259     __shared__ float shared[kNumWarps];
260
261     float val = (idx < N) ? a[idx] : 0.0f;
262     int warp = tid / WARP_SIZE;
263     int lane = tid % WARP_SIZE;
264
265     val = warp_reduce_sum<WARP_SIZE>(val);
266     if (lane == 0)
267         shared[warp] = val;
268     __syncthreads();
269
270     val = (lane < kNumWarps) ? shared[lane] : 0.0f;
271     if (warp == 0)
272         val = warp_reduce_sum<kNumWarps>(val);
273     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果, 避免重复累加
274         atomicAdd(y, val);
275 }
276
277 // ---- Dot Product: y = sum(a[i] * b[i]) ----
278 // 核心模式: elementwise 乘法 → block reduce → atomicAdd 全局累加
279 // Grid: ((N + 255) / 256, 1, 1)
280 // Block: (256, 1, 1)
281 // source: LeetCUDA/kernels/dot-product/dot_product.cu
282 template <const int kNumThreads = 256>
283 __global__ void dot(float *a, float *b, float *y, int N) {
284     int tid = threadIdx.x;
285     int idx = blockIdx.x * kNumThreads + tid;
286     constexpr int kNumWarps = (kNumThreads + WARP_SIZE - 1) / WARP_SIZE;
287     __shared__ float shared[kNumWarps];
288
289     float prod = (idx < N) ? a[idx] * b[idx] : 0.0f;
290     int warp = tid / WARP_SIZE;
291     int lane = tid % WARP_SIZE;
292
293     prod = warp_reduce_sum<WARP_SIZE>(prod);
294     if (lane == 0)
295         shared[warp] = prod;
296     __syncthreads();
297
298     prod = (lane < kNumWarps) ? shared[lane] : 0.0f;
299     if (warp == 0) // 只需要 warp 0 的线程继续 reduce 即可
300         prod = warp_reduce_sum<kNumWarps>(prod);
301     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果, 避免重复累加
302         atomicAdd(y, prod);
303 }
304
305 // Dot Product + float4
306 // Grid: ((N + 255) / 256, 1, 1), 每个block处理256元素, float4向量化后每个线程处理4元素
307 // Block: (64, 1, 1), 256/4=64
308 // 注意: 该版本默认输入地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
309 // source: LeetCUDA/kernels/dot-product/dot_product.cu
310 template <const int kNumThreads = 256 / 4>
311 __global__ void dot_vec4(float *a, float *b, float *y, int N) {
312     int tid = threadIdx.x;
313     int idx = (blockIdx.x * kNumThreads + tid) * 4;
314     constexpr int kNumWarps = (kNumThreads + WARP_SIZE - 1) / WARP_SIZE;

```

```

315     __shared__ float shared[kNumWarps];
316
317     float4 reg_a = FLOAT4(a[idx]);
318     float4 reg_b = FLOAT4(b[idx]);
319     float prod = (idx < N) ? (reg_a.x * reg_b.x + reg_a.y * reg_b.y +
320                             reg_a.z * reg_b.z + reg_a.w * reg_b.w)
321                       : 0.0f;
322     int warp = tid / WARP_SIZE;
323     int lane = tid % WARP_SIZE;
324
325     prod = warp_reduce_sum<WARP_SIZE>(prod);
326     if (lane == 0)
327         shared[warp] = prod;
328     __syncthreads();
329
330     prod = (lane < kNumWarps) ? shared[lane] : 0.0f;
331     if (warp == 0)
332         prod = warp_reduce_sum<kNumWarps>(prod);
333     if (tid == 0)
334         atomicAdd(y, prod);
335 }
336
337
338 // =====
339 // Phase 2: Elementwise Ops (逐元素操作, 演示 coalesced access + vectorize)
340 // =====
341 // 面试要点:
342 //   - 逐元素操作是最简单的 kernel, 核心考点是 memory coalescing
343 //   - float4 向量化可将内存事务数减为 1/4, 大幅提升 bandwidth utilization
344 //   - grid/block 维度设计: grid(N/threads), block(threads), 一维即可
345
346 // ---- ReLU: y = max(0, x) ----
347 // Grid: ((N + 255) / 256, 1, 1)
348 // Block: (256, 1, 1)
349 // source: LeetCUDA/kernels/relu/relu.cu
350 __global__ void relu(float *x, float *y, int N) {
351     int idx = blockIdx.x * blockDim.x + threadIdx.x;
352     if (idx < N)
353         y[idx] = fmaxf(0.0f, x[idx]);
354 }
355
356 // ReLU + float4 向量化: 每个线程处理 4 个元素, 减少 75% 的 load/store 指令
357 // block(64)*4(float4)=256 元素/block, 与基础版吞吐相同
358 // Grid: ((N + 255) / 256, 1, 1)
359 // Block: (64, 1, 1)
360 // 注意: 该版本默认地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
361 // source: LeetCUDA/kernels/relu/relu.cu
362 __global__ void relu_vec4(float *x, float *y, int N) {
363     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
364     if (idx < N) {
365         float4 reg_x = FLOAT4(x[idx]); // 单条 128-bit load
366         float4 reg_y;
367         reg_y.x = fmaxf(0.0f, reg_x.x);
368         reg_y.y = fmaxf(0.0f, reg_x.y);
369         reg_y.z = fmaxf(0.0f, reg_x.z);
370         reg_y.w = fmaxf(0.0f, reg_x.w);
371         FLOAT4(y[idx]) = reg_y; // 单条 128-bit store
372     }
373 }
374
375 // ---- Elementwise Add: c = a + b ----
376 // Grid: ((N + 255) / 256, 1, 1)
377 // Block: (256, 1, 1)

```

```

378 // source: LeetCUDA/kernels/elementwise/elementwise.cu
379 __global__ void elementwise_add(float *a, float *b, float *c, int N) {
380     int idx = blockIdx.x * blockDim.x + threadIdx.x;
381     if (idx < N)
382         c[idx] = a[idx] + b[idx];
383 }
384
385 // Elementwise Add + float4 向量化
386 // Grid: ((N + 255) / 256, 1, 1)
387 // Block: (64, 1, 1), block(64)×4(float4)=256 元素/block
388 // 注意: 主路径要求 4 元素对齐; 尾部不足 4 个元素时回退到标量处理
389 // source: LeetCUDA/kernels/elementwise/elementwise.cu
390 __global__ void elementwise_add_vec4(float *a, float *b, float *c, int N) {
391     int idx = 4 * (blockIdx.x * blockDim.x + threadIdx.x);
392     if ((idx + 3) < N) {
393         float4 reg_a = FLOAT4(a[idx]);
394         float4 reg_b = FLOAT4(b[idx]);
395         float4 reg_c;
396         reg_c.x = reg_a.x + reg_b.x;
397         reg_c.y = reg_a.y + reg_b.y;
398         reg_c.z = reg_a.z + reg_b.z;
399         reg_c.w = reg_a.w + reg_b.w;
400         FLOAT4(c[idx]) = reg_c;
401     } else if (idx < N) {
402         for (int i = 0; (idx + i) < N; i++) {
403             c[idx + i] = a[idx + i] + b[idx + i];
404         }
405     }
406 }
407
408 // ---- Histogram: y[a[i]]++ ----
409 // 演示 atomicAdd 的用法: 多个线程可能同时更新同一个 bin
410 // Grid: ((N + 255) / 256, 1, 1)
411 // Block: (256, 1, 1)
412 // source: LeetCUDA/kernels/histogram/histogram.cu
413 __global__ void histogram(int *a, int *y, int N) {
414     int idx = blockIdx.x * blockDim.x + threadIdx.x;
415     if (idx < N)
416         atomicAdd(&(y[a[idx]]), 1);
417 }
418
419 // ---- Merge Attn States: 合并 Split-KV Attention 的部分结果 ----
420 // 实现 FlashInfer 论文 (arXiv 2501.01005) Section 2.2 的 attention 合并逻辑。
421 //
422 // 面试要点:
423 // - 应用场景: Split-KV attention 将序列沿 K 维分段计算 attention,
424 //   每段产出部分结果 (O_i, LSE_i), 本 kernel 将两段按指数权重加权合并。
425 // - 数学公式 (5 步推导):
426 //   Step 1 — max 归一化 (数值稳定, 与 safe softmax 同理):
427 //       L_max = max(LSE_1, LSE_2)
428 //   Step 2 — 还原指数权重 (un-normalized):
429 //       w_1 = exp(LSE_1 - L_max), w_2 = exp(LSE_2 - L_max)
430 //   Step 3 — 归一化为混合比例:
431 //       alpha = w_1 / (w_1 + w_2), beta = w_2 / (w_1 + w_2)
432 //       满足 alpha + beta = 1
433 //   Step 4 — 逐元素加权合并:
434 //       O = alpha * O_1 + beta * O_2
435 // - LSE 布局 [num_heads, num_tokens] (与 FlashAttention 惯例一致,
436 //   lse[head_idx][token_idx])
437 // - Output 布局 [num_tokens, num_heads, head_size] (token 维在最外,
438 //   展平为 [T0H0, T0H1, ..., T1H0, ...])
439 // - inf LSE → -inf: 空 attention 段 (causal mask 等导致全部 score
440 //   为 -inf) 的 LSE 可能为 +inf, 替换后 exp(-inf - L_max) = 0,

```

```

441 // 该段权重退化为 0
442 // - 128-bit 向量化: uint4 = 4 × float, 每线程处理 4 个输出元素,
443 //   pack load → FMA → pack store 一条流水线
444 // - AI ≈ 3 FLOP / 20 bytes ≈ 0.15 FLOPS/Byte → severely memory-bound
445 //
446 // 线程到数据的 3D 映射 (以 num_tokens=2, num_heads=2, head_size=8 为例):
447 //   kPackSize = 16 / sizeof(float) = 4 (每线程 4 个 float = 128-bit)
448 //   threads_per_head = head_size / 4 = 2
449 //   total_threads = num_tokens * num_heads * threads_per_head = 8
450 //
451 //   global_idx:      0      1      2      3      4      5      6      7
452 //   token_head_idx:  0      0      1      1      2      2      3      3
453 //   (第几个 (token,head) 对, 行优先)
454 //   pack_idx:        0      1      0      1      0      1      0      1
455 //   (该 (token,head) 对内第几个 pack)
456 //   token_idx:        0      0      0      0      1      1      1      1
457 //   (token_head_idx / num_heads, token 变化慢)
458 //   head_idx:         0      0      1      1      0      0      1      1
459 //   (token_head_idx % num_heads, head 变化快)
460 //   pack_offset:      0      4      0      4      0      4      0      4
461 //   (pack_idx * 4, 该 pack 在 head_size 中的起始元素偏移)
462 //   head_offset:      0      0      8      8     16     16     24     24
463 //   (token_idx * num_heads * head_size + head_idx * head_size,
464 //    该 (token,head) 对在 output 展平数组中的起始偏移)
465 //
466 // Grid: ((total_threads + 127) / 128, 1, 1)
467 // Block: (128, 1, 1)
468 // source: LeetCUDA/kernels/openai-triton/merge-attn-states/cuda_merge_attn_states.cu
469 _global__ void merge_attn_states(
470     float *output,           // [num_tokens, num_heads, head_size]
471     const float *prefix_output, // [num_tokens, num_heads, head_size]
472     const float *prefix_lse,   // [num_heads, num_tokens]
473     const float *suffix_output, // [num_tokens, num_heads, head_size]
474     const float *suffix_lse,   // [num_heads, num_tokens]
475     int num_tokens, int num_heads, int head_size) {
476
477     constexpr int kNumThreads = 128;
478     constexpr int kPackSize = 16 / sizeof(float); // 4 floats = 128-bit
479     using pack_t = uint4;
480
481     // 每个 (token, head) 对需要 threads_per_head 个线程覆盖 head_size 个元素
482     const int threads_per_head = head_size / kPackSize;
483     // 实际有效总线程数, 超过这个数的线程会被忽略, block是按照NUM_THREADS=128
484     // 来分配的, 那么最后一个block的线程数可能会超过实际需要的线程数
485     const int total_threads = num_tokens * num_heads * threads_per_head;
486
487     const int global_idx = blockIdx.x * kNumThreads + threadIdx.x;
488     if (global_idx >= total_threads)
489         return;
490
491     // global_idx → (token_head_idx, pack_idx) → (token_idx, head_idx, pack_offset)
492     // token_head_idx: 第几个 (token, head) 对, 行优先展平
493     const int token_head_idx = global_idx / threads_per_head;
494     // pack_idx: 该 (token, head) 对内第几个 pack, 0 ~ threads_per_head-1
495     const int pack_idx = global_idx % threads_per_head;
496
497     // token_head_idx 分解为 (token_idx, head_idx)
498     const int token_idx = token_head_idx / num_heads; // token 变化慢 (外维)
499     const int head_idx = token_head_idx % num_heads;   // head 变化快 (内维)
500
501     // pack_offset: 该 pack 覆盖的元素在 head_size 中的起始偏移 (0, 4, 8, ...)
502     const int pack_offset = pack_idx * kPackSize;
503     // head_offset: 该 (token, head) 对在 output 展平一维数组中的起始偏移

```

```

504 const int head_offset = token_idx * num_heads * head_size + head_idx * head_size;
505
506 // 定位到当前 (token, head) 的输出段起始
507 const float *prefix_head = prefix_output + head_offset;
508 const float *suffix_head = suffix_output + head_offset;
509 float *output_head = output + head_offset;
510
511 // LSE 布局 [num_heads, num_tokens]: lse[head_idx][token_idx]
512 float p_lse = prefix_lse[head_idx * num_tokens + token_idx];
513 float s_lse = suffix_lse[head_idx * num_tokens + token_idx];
514
515 // inf → -inf: 空 attention 段 LSE 可能为 +inf, 替换后权重退化为 0
516 p_lse = isinf(p_lse) ? -INFINITY : p_lse;
517 s_lse = isinf(s_lse) ? -INFINITY : s_lse;
518
519 // Step 1: max 归一化 (与 safe softmax 同理, 防 exp 溢出)
520 const float max_lse = fmaxf(p_lse, s_lse);
521 p_lse -= max_lse;
522 s_lse -= max_lse;
523
524 // Step 2-3: 指数还原 → 混合比例 alpha, beta
525 const float p_se = expf(p_lse);
526 const float s_se = expf(s_lse);
527 const float out_se = p_se + s_se;
528 const float p_scale = p_se / out_se; // alpha = w_1 / (w_1 + w_2)
529 const float s_scale = s_se / out_se; // beta = w_2 / (w_1 + w_2)
530
531 // Step 4: 逐元素加权合并 0 = alpha * 0_1 + beta * 0_2
532 // 128-bit 量化: uint4 load → per-element FMA → uint4 store
533 if (pack_offset < head_size) {
534     pack_t p_pack =
535         reinterpret_cast<const pack_t *>(prefix_head)[pack_idx];
536     pack_t s_pack =
537         reinterpret_cast<const pack_t *>(suffix_head)[pack_idx];
538     pack_t o_pack;
539
540 #pragma unroll
541     for (int i = 0; i < kPackSize; ++i) {
542         const float p_v = reinterpret_cast<const float *>(&p_pack)[i];
543         const float s_v = reinterpret_cast<const float *>(&s_pack)[i];
544         const float o_v = p_v * p_scale + (s_v * s_scale); // FMA
545         reinterpret_cast<float *>(&o_pack)[i] = o_v;
546     }
547
548     reinterpret_cast<pack_t *>(output_head)[pack_idx] = o_pack;
549 }
550 }
551
552 // =====
553 // Phase 3: Reduce 类 Ops — Softmax / RMS Norm / Layer Norm
554 // =====
555 // 面试要点:
556 //   - Softmax 三种实现递进: naive (溢出) → safe (2-pass, max 减法) →
557 //     online (1-pass, 增量更新)
558 //   - Online Softmax 是 FlashAttention 的数学基础
559 //   - RMS Norm vs Layer Norm: RMS 只需 1 次 reduce, Layer Norm 需要 2 次 (mean
560 //     + variance)
561 //   - Per-token 设计: 一个 block 处理一个 token, 无需跨 block 同步
562
563 // =====
564 // Phase 3a: Softmax — 三级递进 (面试核心考点)
565 // =====
566 // 面试常问: 「Softmax 有哪些实现方式? 各有什么优缺点? 」

```

```

567 // 回答线索: naive(溢出) → safe → online
568
569 // ---- Level 1: 基础 Softmax (per-token, 无 max 减法, 数值不稳定) ----
570 // grid(S*h/h, h), block(h), 一个 block 处理一个 token
571 // 问题: x 值很大时 exp(x) 溢出为 inf
572 template <const int kNumThreads = 256>
573 // Grid: (S, 1, 1), S=batch*seq_len, DISPATCH_SOFTMAX_F32_PER_TOKEN_KERNEL
574 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024, 一个 block 处理一个 token
575 // source: LeetCUDA/kernels/softmax/softmax.cu
576 __global__ void softmax_per_token(float *x, float *y, int N) {
577     const int tid = threadIdx.x;
578     const int idx = blockIdx.x * blockDim.x + tid;
579
580     float exp_val = (idx < N) ? expf(x[idx]) : 0.0f;
581     float exp_sum = block_reduce_sum<kNumThreads>(exp_val);
582     if (idx < N)
583         y[idx] = exp_val / exp_sum;
584 }
585
586 // ---- Level 2: Safe Softmax (2-pass: 先 max 再 exp, 数值稳定) ----
587 // 面试重点: 为什么 Softmax 需要 Safe?
588 //   - expf 溢出阈值约 88.7: exp(88)≈1.65e38 (接近 float32 上限 3.4e38), exp(89)≈4.5e38 已溢出为 inf
589 //   - 减去 max 后: exp(x - max) ≤ exp(0) = 1.0, 永不超过 1
590 //   - 数学等价性: softmax(x) = softmax(x - c) 对任意常数 c 成立
591 //   - 代价: 2 次 block reduce (先 max, 再 sum), 但仍 O(N/B) 高效
592 // Grid: (S, 1, 1)
593 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
594 // source: LeetCUDA/kernels/softmax/softmax.cu
595 template <const int kNumThreads = 256>
596 __global__ void safe_softmax_per_token(float *x, float *y, int N) {
597     const int tid = threadIdx.x;
598     const int idx = blockIdx.x * blockDim.x + tid;
599
600     // Pass 1: block reduce max — 找最大值
601     float val = (idx < N) ? x[idx] : -FLT_MAX;
602     float max_val = block_reduce_max<kNumThreads>(val);
603
604     // Pass 2: exp(x - max) → block reduce sum
605     float exp_val = (idx < N) ? expf(x[idx] - max_val) : 0.0f;
606     float exp_sum = block_reduce_sum<kNumThreads>(exp_val);
607
608     if (idx < N)
609         y[idx] = exp_val / exp_sum;
610 }
611
612 // ---- Level 3: Online Safe Softmax (FlashAttention 的数学基础) ----
613 // 面试重点:
614 // 两种使用场景 (公式不同, 但结果等价):
615 // 1) 单元素增量更新 (online update, 处理新元素 x_i):
616 //     m_new = max(m_old, x_i)
617 //     d_new = d_old * exp(m_old - m_new) + exp(x_i - m_new)
618 // 2) 二元合并 (binary merge, 合并两个部分累加器, warp/block reduce 使用):
619 //     m = max(m1, m2) safe softmax用的max值
620 //     d = d1*exp(m1-m) + d2*exp(m2-m) softmax用的分母值
621 //     当 m1≥m2 时退化为 d1 + d2*exp(m2-m1) (d_bigger + d_smaller*exp(m_smaller - m_bigger))
622 // 算法来源: "Online normalizer calculation for softmax" (arXiv:1805.02867)
623 // Warp Reduce for Online Softmax — binary merge of two partial (m,d) accumulators.
624 //
625 // 不同于单元素增量更新公式 (m_new = max(m_old, x_i); d_new = d_old*exp(m_old-m_new) + exp(x_i-m_new)),
626 // warp reduce 中每次合并的是两个**已各自归一化到各自 max 的部分累加器**:
627 // (m1,d1): max 和 Σexp(x_j - m1), 覆盖集合 S1
628 // (m2,d2): max 和 Σexp(x_k - m2), 覆盖集合 S2
629 //

```

```

630 // 合并公式 (对称, 不依赖"新/旧"概念):
631 //   m = max(m1, m2)
632 //   d = d1*exp(m1-m) + d2*exp(m2-m)   ← m1≥m2 时退化为 d1 + d2*exp(m2-m1)
633 //
634 // 该操作满足结合律 → XOR butterfly 任意归约顺序均保证全局正确结果。
635 struct __align__(8) MD {
636     float m; // running max
637     float d; // running denominator (sum of exp(x - max))
638 };
639
640 template <const int kWarpSize = WARP_SIZE>
641 __device__ __forceinline__ MD warp_reduce_md(MD md1) {
642     #pragma unroll
643     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
644         MD md2;
645         md2.m = __shfl_xor_sync(0xffffffff, md1.m, mask, kWarpSize);
646         md2.d = __shfl_xor_sync(0xffffffff, md1.d, mask, kWarpSize);
647
648         MD b_m = (md1.m > md2.m) ? md1 : md2; // max
649         MD s_m = (md1.m > md2.m) ? md2 : md1;
650
651         // b_m.d 无需 rescale (其基准 max 已是全局 max),
652         // s_m.d 需 rescale: exp(s_m.m - b_m.m)
653         md1.d = b_m.d + s_m.d * __expf(s_m.m - b_m.m);
654         md1.m = b_m.m;
655     }
656     return md1;
657 }
658
659 // 注意: 这里默认一个 block 处理一个 token; 边界线程的 d=0 只参与归约, 不会写回 y
660 // Grid: (S, 1, 1)
661 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
662 // source: LeetCUDA/kernels/softmax/softmax.cu
663 template <const int kNumThreads = 256>
664 __global__ void online_safe_softmax_per_token(const float *x, float *y, int N) {
665     int tid = threadIdx.x;
666     int idx = blockIdx.x * kNumThreads + threadIdx.x;
667     const int kNumWarps = (kNumThreads + WARP_SIZE - 1) / WARP_SIZE;
668     __shared__ MD shared[kNumWarps];
669
670     float val = (idx < N) ? x[idx] : -FLT_MAX;
671     int warp = tid / WARP_SIZE;
672     int lane = tid % WARP_SIZE;
673
674     // 初始化: 每个线程持有一个 (max, denom) 对
675     MD md;
676     md.m = idx < N ? val : -FLT_MAX;
677     md.d = idx < N ? 1.0f : 0.0f;
678
679     // 第一级规约: warp_reduce_md 在归约中自动更新 m 和 d
680     md = warp_reduce_md<WARP_SIZE>(md);
681
682     if (lane == 0)
683         shared[warp] = md;
684     __syncthreads();
685
686     // 第二级归约: 每个 warp 结果再做一次 warp_reduce_md (复用 block_reduce 模式)
687     md = lane < kNumWarps ? shared[lane] : MD{-FLT_MAX, 0.0f};
688     md = warp_reduce_md<WARP_SIZE>(md); // 用WARP_SIZE确保每个lane都能拿到最终结果
689
690     // 用全局 max 和 denom 做最终 softmax
691     float d_inv = __fdivdef(1.0f, md.d);
692     // 边界线程即使看到 d=0 的填充值, 也不会走到写回路径

```

```

693     if (idx < N) {
694         y[idx] = __expf(val - md.m) * d_inv;
695     }
696 }
697
698 // =====
699 // Phase 3b: RMS Normalization (1-pass reduce)
700 // =====
701 // 面试要点:
702 //   - RMS Norm:  $y = (x / \text{rms}(x)) * g$ ,  $1/\text{rms}(x) = \text{rsqrt}(\text{mean}(x^2))$ 
703 //   - 只需 1 次 block reduce (sum of squares), 比 Layer Norm 少 1 次同步
704 //   - Llama 系列使用 RMS Norm
705 //   - grid(N, K/K), block(K): 一行一个 block
706 // Grid: (N, 1, 1), N=batch*seq_len, 每行一个 block
707 // Block: (128, 1, 1), kNumThreads=K=128 (K>128 时调整模板参数)
708 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
709 template <const int kNumThreads = 128>
710 __global__ void rms_norm(float *x, float *y, float g, int N, int K) {
711     int tid = threadIdx.x;
712     int idx = blockIdx.x * blockDim.x + threadIdx.x;
713     const float epsilon = 1e-5f;
714
715     __shared__ float s_variance;
716     float value = (idx < N * K) ? x[idx] : 0.0f;
717     float variance = value * value;
718     variance = block_reduce_sum<kNumThreads>(variance);
719     if (tid == 0)
720         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/rms(x)
721     __syncthreads();
722     if (idx < N * K)
723         y[idx] = (value * s_variance) * g;
724 }
725
726 // RMS Norm + float4
727 // Grid: (N, 1, 1)
728 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
729 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
730 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
731 template <const int kNumThreads = 128 / 4>
732 __global__ void rms_norm_vec4(float *x, float *y, float g, int N, int K) {
733     int tid = threadIdx.x;
734     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
735     const float epsilon = 1e-5f;
736
737     __shared__ float s_variance;
738     float4 reg_x = FLOAT4(x[idx]);
739     float variance = (idx < N * K) ? (reg_x.x * reg_x.x + reg_x.y * reg_x.y +
740                                     reg_x.z * reg_x.z + reg_x.w * reg_x.w)
741                         : 0.0f;
742     variance = block_reduce_sum<kNumThreads>(variance);
743     if (tid == 0)
744         s_variance = rsqrtf(variance / (float)K + epsilon);
745     __syncthreads();
746     float4 reg_y;
747     reg_y.x = reg_x.x * s_variance * g;
748     reg_y.y = reg_x.y * s_variance * g;
749     reg_y.z = reg_x.z * s_variance * g;
750     reg_y.w = reg_x.w * s_variance * g;
751     if (idx < N * K)
752         FLOAT4(y[idx]) = reg_y;
753 }
754
755 // =====

```

```

756 // Phase 3c: Layer Normalization (2-pass reduce)
757 // =====
758 // 面试要点:
759 //   - Layer Norm:  $y = ((x - \text{mean}) / \text{std}) * g + b$ ,  $\text{std} = \sqrt{\text{variance}}$ ,  $\text{variance} = \text{mean}((x - \text{mean})^2)$ 
760 //   - 需要 2 次 block reduce: 先 mean (sum/K), 再 variance (sum((x-mean)2)/K)
761 //   - 两次 __syncthreads 必须到位, 否则 s_mean 未对所有线程可见就计算 variance
762 // Grid: (N, 1, 1), 一行一个 block
763 // Block: (128, 1, 1), kNumThreads=K=128
764 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
765 template <const int kNumThreads = 128>
766 __global__ void layer_norm(float *x, float *y, float g, float b, int N, int K) {
767     int tid = threadIdx.x;
768     int idx = blockIdx.x * blockDim.x + threadIdx.x;
769     const float epsilon = 1e-5f;
770
771     __shared__ float s_mean;
772     __shared__ float s_variance;
773     float value = (idx < N * K) ? x[idx] : 0.0f;
774
775     // Pass 1: compute mean
776     float sum = block_reduce_sum<kNumThreads>(value);
777     if (tid == 0)
778         s_mean = sum / (float)K;
779     __syncthreads(); // 必须等待 s_mean 对所有线程可见
780
781     // Pass 2: compute variance = (x - mean)2
782     float variance = (value - s_mean) * (value - s_mean);
783     variance = block_reduce_sum<kNumThreads>(variance);
784     if (tid == 0)
785         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/std
786     __syncthreads(); // 必须等待 s_variance 对所有线程可见
787
788     if (idx < N * K)
789         y[idx] = ((value - s_mean) * s_variance) * g + b;
790 }
791
792 // Layer Norm + float4
793 // Grid: (N, 1, 1)
794 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
795 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
796 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
797 template <const int kNumThreads = 128 / 4>
798 __global__ void layer_norm_vec4(float *x, float *y, float g, float b, int N, int K) {
799     int tid = threadIdx.x;
800     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
801     const float epsilon = 1e-5f;
802
803     __shared__ float s_mean;
804     __shared__ float s_variance;
805     float4 reg_x = FLOAT4(x[idx]);
806     float value = (idx < N * K) ? (reg_x.x + reg_x.y + reg_x.z + reg_x.w) : 0.0f;
807
808     float sum = block_reduce_sum<kNumThreads>(value);
809     if (tid == 0)
810         s_mean = sum / (float)K;
811     __syncthreads();
812
813     float4 reg_x_hat;
814     reg_x_hat.x = reg_x.x - s_mean;
815     reg_x_hat.y = reg_x.y - s_mean;
816     reg_x_hat.z = reg_x.z - s_mean;
817     reg_x_hat.w = reg_x.w - s_mean;
818     float variance = reg_x_hat.x * reg_x_hat.x + reg_x_hat.y * reg_x_hat.y +

```

```

819         reg_x_hat.z * reg_x_hat.z + reg_x_hat.w * reg_x_hat.w;
820     variance = block_reduce_sum<kNumThreads>(variance);
821     if (tid == 0)
822         s_variance = rsqrtf(variance / (float)K + epsilon);
823     __syncthreads();
824
825     float4 reg_y;
826     reg_y.x = reg_x_hat.x * s_variance * g + b;
827     reg_y.y = reg_x_hat.y * s_variance * g + b;
828     reg_y.z = reg_x_hat.z * s_variance * g + b;
829     reg_y.w = reg_x_hat.w * s_variance * g + b;
830     if (idx < N * K)
831         FLOAT4(y[idx]) = reg_y;
832 }
833
834 // =====
835 // Phase 4: RoPE — 旋转位置编码 (Rotary Position Embedding)
836 // =====
837 // 面试要点:
838 //   - RoPE 数学公式: 对每对相邻维度做 2D 旋转
839 //       [x1']   [cos(θ)  -sin(θ)] [x1]
840 //       [x2'] = [sin(θ)   cos(θ)] [x2]
841 //   -  $\theta_i = 1 / (\text{theta}^{(2i/d)})$ , theta=10000.0f (Llama 风格)
842 //   - token_pos = idx / N: token 在序列中的位置
843 //   - token_idx = idx % N: token 内的维度索引
844 //   - 输入 [seq_len, hidden_size], 输出同形状
845
846 // Grid: ((seq_len * N + 255) / 256, 1, 1)
847 // Block: (256, 1, 1)
848 // source: LeetCUDA/kernels/rope/rope.cu
849 __global__ void rope(float *x, float *out, int seq_len, int N) {
850     int idx = blockIdx.x * blockDim.x + threadIdx.x;
851     float x1 = x[idx * 2];
852     float x2 = x[idx * 2 + 1];
853     int token_pos = idx / N; // 序列位置
854     int token_idx = idx % N; // 维度索引
855
856     // 频率计算:  $\theta_i = 1 / (10000^{(2i/d)})$ 
857     float exp_v = 1.0f / powf(10000.0f, 2 * token_idx / (N * 2.0f));
858     float sin_v = sinf(token_pos * exp_v);
859     float cos_v = cosf(token_pos * exp_v);
860
861     // 2D 旋转
862     float out1 = x1 * cos_v - x2 * sin_v;
863     float out2 = x1 * sin_v + x2 * cos_v;
864     out[idx * 2] = out1;
865     out[idx * 2 + 1] = out2;
866 }
867
868 // =====
869 // Phase 5: Mat Transpose — 矩阵转置 (Bank Conflict 专题)
870 // =====
871 // 面试要点 (Bank Conflict 专题):
872 //   - Shared memory 有 32 个 bank, 每个 bank 4 bytes (32-bit)
873 //   - 同一 warp 的多个线程访问同一 bank 的不同地址 → bank conflict
874 //   - n-way bank conflict: n 个线程冲突 → 访问串行化为 n 次
875 //   - 解决方案: PAD (在每行末尾加 1 个元素, 打破地址对齐)
876 //
877 // 转置的四步演进:
878 //   naive: 非合并写入 (列优先写) → 每个 warp 产生 32 次内存事务
879 //   shared: 写入 smem (行优先) → 从 smem 读取 (列优先) → 合并写入 gmem
880 //   BCF: smem 布局 [WARP_SIZE_S*4][WARP_SIZE_S+PAD] = [64][17], PAD=1 加在第二维消除 bank conflict
881 //   merge_write: 进一步将 4 次 separate store 合并为 1 次 float4 store

```

```

882 // 注: 本文件仅实现 Level 1(naive) 与 Level 4(BCF+merge_write), Level 2/3 省略
883
884 // 每个线程处理 1 个元素, block(16,16)
885 // 读: x 按行优先访问, warp 的 32 线程访问 32 个连续地址 → 合并读取 ✓
886 // 写: y 按列优先写入, 32 线程跨 row 行分散 → 非合并写入 x (32 次内存事务)
887 // Grid: ((col + 15) / 16, (row + 15) / 16, 1), 每线程 1 元素
888 // Block: (16, 16, 1)
889 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
890 __global__ void mat_transpose(float *x, float *y, const int row, const int col) {
891     const int c = blockIdx.x * blockDim.x + threadIdx.x; // col in x
892     const int r = blockIdx.y * blockDim.y + threadIdx.y; // row in x
893     if (r < row && c < col) {
894         // x[r][c] → y[c][r]; x stride=col, y (transposed) stride=row
895         y[c * row + r] = x[r * col + c];
896     }
897 }
898
899 // Grid: ((col + 15) / 16, (row + 63) / 64, 1), 每线程 4 元素(float4)
900 // Block: (16, 16, 1)
901 // 注意: 该版本默认按 float4 打包写回; 最适合 row 能按 4 对齐的场景
902 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
903 __global__ void mat_transpose_padded(
904     float *x, float *y, const int row, const int col) {
905     const int tx = threadIdx.x;
906     const int ty = threadIdx.y;
907
908     constexpr int TILE = 16;
909     constexpr int PAD = 1;
910     // Bank conflict fix: 每行多 1 个元素, 打破 32-bank 对齐
911     __shared__ float tile[TILE * 4][TILE + PAD]; // 64x16
912
913     // x 空间坐标; 每线程覆盖 4 行 (actual row = x_r * 4)
914     const int x_c = blockIdx.x * TILE + tx;
915     const int x_r = blockIdx.y * TILE + ty;
916
917     if (x_r * 4 < row && x_c < col) {
918         // Step 1: 4 rows × 1 col → smem (row-major);
919 #pragma unroll
920         for (int i = 0; i < 4; ++i) {
921             tile[ty * 4 + i][tx] = x[(x_r * 4 + i) * col + x_c];
922         }
923         __syncthreads();
924
925         // Step 2: Transposed read from smem
926         float4 reg_trans;
927         reg_trans.x = tile[tx * 4 + 0][ty];
928         reg_trans.y = tile[tx * 4 + 1][ty];
929         reg_trans.z = tile[tx * 4 + 2][ty];
930         reg_trans.w = tile[tx * 4 + 3][ty];
931
932         // y 空间坐标: y_r = x 的列, y_c = x 的行
933         const int y_r = blockIdx.x * TILE + ty; // = x col
934         const int y_c = (blockIdx.y * TILE + tx) * 4; // = x row
935
936         // Coalesced write: y[y_r][y_c .. y_c+3]
937         FLOAT4(y[y_r * row + y_c]) = reg_trans;
938     }
939 }
940
941 // =====
942 // Phase 6: GEMV — 矩阵向量乘 (M or N = 1, 纯 memory-bound 算子, warp-per-row 策略)
943 // =====
944 // 面试要点:

```

```

945 // - GEMV 是典型的 memory-bound 算子:  $AI \approx O(1)$ , 瓶颈在内存带宽
946 // - 核心策略: warp-per-row (每个 warp 负责矩阵的一行)
947 // - 不同 K 值对应不同分块策略:
948 //   K=32 倍数: 一个 warp 的 32 线程恰好覆盖 K 维
949 //   K=128 倍数: 每个线程用 float4 处理 4 元素, warp 覆盖 128
950 //   K=16 < 32: 一个 warp 用不满, 用 ROW_PER_WARP=2 让每个 warp 处理 2 行
951 //
952 // a: MxK, x: Kx1, y: Mx1, 计算:  $y = a * x$ ; N = 1
953
954 // ---- SGEMV K32: 基础 warp-per-row ----
955 // 设计: block(32, 4), blockDim.x=WARP_SIZE=32 (K 需为 32 倍数时一轮覆盖, 否则内层循环 kNumWarps 次)
956 // grid(M/4), 每个 warp 负责一行
957 // K 为 32 的倍数时, warp 的 32 个线程恰好覆盖 K 维
958 // Grid: ((M + 3) / 4, 1, 1), 每 block 处理 4 行
959 // Block: (32, 4, 1), 每行 1 warp
960 // 注意: 该版本最适合 K 按 32 对齐; 当 K 更小时通常切到 K16 这类专用分支
961 // source: LeetCUDA/kernels/sgemv/sgemv.cu
962 __global__ void sgemv_k32(float *a, float *x, float *y, int M, int K) {
963     int tx = threadIdx.x; // 0~31
964     int ty = threadIdx.y; // 0~3
965     int lane = tx % WARP_SIZE; // 0~31
966     int m = blockIdx.x * blockDim.y + ty; // 全局行号
967     if (m < M) {
968         float sum = 0.0f;
969         // 沿 K 维的迭代数 = ceil(K/32), 每个 warp 要累加完整的K, 那么
970         // 每个thread就要负责累加NUM_ITERS个元素, NUM_ITERS = ceil(K/32)
971         const int NUM_ITERS = (K + WARP_SIZE - 1) / WARP_SIZE;
972 #pragma unroll
973         for (int w = 0; w < NUM_ITERS; ++w) {
974             // 假设K是32的整倍数, m * K 本行的起始地址, x: Kx1
975             int k = w * WARP_SIZE + lane;
976             sum += a[m * K + k] * x[k];
977         }
978         sum = warp_reduce_sum<WARP_SIZE>(sum);
979         // 每个 warp 处理一行, lane 0 写回结果
980         if (lane == 0)
981             y[m] = sum;
982     }
983 }
984
985 // ---- SGEMV K128: float4 向量化 ----
986 // 每个线程处理 4 个元素(float4), 一个 warp 覆盖 128 个元素
987 // Grid: ((M + 3) / 4, 1, 1)
988 // Block: (32, 4, 1)
989 // 注意: 该版本最适合 K 按 128 对齐, 且 x/a 的地址满足 float4 对齐
990 // source: LeetCUDA/kernels/sgemv/sgemv.cu
991 __global__ void sgemv_k128(float *a, float *x, float *y, int M, int K) {
992     int tx = threadIdx.x; // 0~31
993     int ty = threadIdx.y; // 0~3
994     int lane = tx % WARP_SIZE; // 0~31
995     int m = blockDim.y * blockIdx.x + ty;
996
997     if (m < M) {
998         float sum = 0.0f;
999         // 沿 K 维的迭代数 = ceil(K/128), 每个 warp 每轮用 float4 覆盖 128 个 K 元素
1000         const int NUM_ITERS = (((K + WARP_SIZE - 1) / WARP_SIZE) + 4 - 1) / 4;
1001 #pragma unroll
1002         for (int w = 0; w < NUM_ITERS; ++w) {
1003             int k = (w * WARP_SIZE + lane) * 4;
1004             float4 reg_x = FLOAT4(x[k]);
1005             float4 reg_a = FLOAT4(a[m * K + k]);
1006             sum += (reg_a.x * reg_x.x + reg_a.y * reg_x.y + reg_a.z * reg_x.z +
1007                    reg_a.w * reg_x.w);

```

```

1008     }
1009     sum = warp_reduce_sum<WARP_SIZE>(sum);
1010     if (lane == 0)
1011         y[m] = sum;
1012 }
1013 }
1014
1015 // ---- SGEMV K16: K < WarpSize, ROW_PER_WARP=2 ----
1016 // 面试亮点: K=16 < 32, 一个 warp 可以处理多行
1017 // ROW_PER_WARP=2, K_WARP_SIZE=16, 前 16 个 lane 处理 row0, 后 16 个 lane 处理 row1
1018 // Grid: ((M + 7) / 8, 1, 1), NUM_ROWS=8
1019 // Block: (32, 4, 1)
1020 // 注意: 这一版是面向 K=16 的专用写法; ROW_PER_WARP=2 时一个 warp 同时处理 2 行
1021 // source: LeetCode/kernels/sgemv/sgemv.cu
1022 template <const int ROW_PER_WARP = 2>
1023 __global__ void sgemv_k16(float *A, float *x, float *y, int M, int K) {
1024     constexpr int K_WARP_SIZE = (WARP_SIZE + ROW_PER_WARP - 1) / ROW_PER_WARP; // 16
1025     int tx = threadIdx.x;
1026     int ty = threadIdx.y;
1027     int lane = tx % WARP_SIZE;
1028     int k = lane % K_WARP_SIZE; // 0~15
1029     int m = (blockDim.y * blockIdx.x + ty) * ROW_PER_WARP + lane / K_WARP_SIZE;
1030     if (m < M) {
1031         float sum = A[m * K + k] * x[k];
1032         // 按照K_WARP_SIZE=16, 分2组各自做 warp reduce sum, k==0的lane写回结果
1033         sum = warp_reduce_sum<K_WARP_SIZE>(sum);
1034         // 注意: 判断条件是 k == 0, 不是 lane == 0!
1035         if (k == 0)
1036             y[m] = sum;
1037     }
1038 }
1039
1040 // =====
1041 // Phase 7: GEMM — 矩阵矩阵乘 (GPU 最重要的算子, 面试核心考点)
1042 // =====
1043 // 面试要点 (GEMM 优化五层金字塔):
1044 //   Level 1 — Tiling (分块 + shared memory): 将数据从 HBM 搬到 SMEM 复用
1045 //   Level 2 — Thread Tile (寄存器分块): 每个线程计算 TM×TN
1046 //   个元素, 提高计算密度 Level 3 — Vectorize (向量化访存): float4/half2, 减少
1047 //   load/store 指令数 Level 4 — Tensor Core (MMA
1048 //   m16n8k16): 硬件矩阵乘单元, warp 级指令 Level 5 — Warp Specialization +
1049 //   TMA (WGMMMA m64n128k16): Hopper 异步执行
1050 //
1051 // 计算密度递进:
1052 //   Level 1: AI ≈ B_K / (2×sizeof) ≈ 32/8 = 4 → 仍是 memory-bound
1053 //   Level 2: AI ≈ TM×TN×B_K / (2×sizeof) ≈ 8×8×8/8 = 64 → compute-bound
1054 //   Level 4: Tensor Core 提供硬件加速的 256 FMA/cycle/warp → 大幅提升吞吐
1055 //
1056 // =====
1057 // Phase 7a: SGEMM (非 Tensor Core 路径)
1058 // =====
1059
1060 // ---- Level 1: SGEMM — Block Tile 32×32 + K Tile 32 ----
1061 // 最基础的 tiling 实现, 演示 shared memory 的核心用法
1062 // C = A × B, C[M, N] = A[M, K] × B[K, N]
1063 // BM=BN=32, BK=32, block(32, 32), 一个线程计算 c 的一个元素
1064 // Grid: ((N + 31) / 32, (M + 31) / 32, 1)
1065 // Block: (32, 32, 1), 1024 线程
1066 // source: LeetCode/kernels/sgemm/sgemm.cu
1067 __global__ void sgemm(float *a, float *b, float *c, int M, int N, int K) {
1068     constexpr int BM = 32; // vec 版: 32×4 = 128
1069     constexpr int BN = 32; // vec 版: 32×4 = 128
1070     constexpr int BK = 32;

```

```

1071 __shared__ float s_a[BM][BK], s_b[BK][BN]; // 32x32x4=4KB smem, float = 4 bytes
1072
1073 int bx = blockIdx.x;
1074 int by = blockIdx.y;
1075 int tx = threadIdx.x;
1076 int tid = threadIdx.y * blockDim.x + tx;
1077
1078 // 线程到 smem 的映射: 32x32 线程, 每个线程加载 a 和 b 各 1 个元素
1079 int load_smem_a_m = tid / 32; // row 0~31 由 32 线程加载;
1080 int load_smem_a_k = tid % 32; // col 0~31 由 32 线程加载;
1081 int load_smem_b_k = tid / 32; // row 0~31 由 32 线程加载;
1082 int load_smem_b_n = tid % 32; // col 0~31 由 32 线程加载;
1083 int load_gmem_a_m = by * BM + load_smem_a_m; // gmem row;
1084 int load_gmem_b_n = bx * BN + load_smem_b_n; // gmem col;
1085
1086 float sum = 0.f; // 遍历完整的K, slice K;
1087 for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1088     int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
1089     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1090     s_a[load_smem_a_m][load_smem_a_k] = a[load_gmem_a_addr];
1091     int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
1092     int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1093     s_b[load_smem_b_k][load_smem_b_n] = b[load_gmem_b_addr];
1094     __syncthreads(); // 确保整个 smem tile 加载完毕
1095
1096 #pragma unroll
1097     for (int k = 0; k < BK; ++k) {
1098         int comp_smem_a_m = load_smem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续)
1099         int comp_smem_b_n = load_smem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1100         sum += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
1101     }
1102     __syncthreads(); // 确保 smem 不会在下一轮加载时被覆盖
1103 }
1104 int store_gmem_c_m = load_gmem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续) [128x128]
1105 int store_gmem_c_n = load_gmem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1106 int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
1107 c[store_gmem_c_addr] = sum; // C [M, N] = A[M, K] x B[K, N]
1108 }
1109
1110 // ---- Level 1+: SGEMM Vec4 — Block Tile 128x128 + K Tile 32 + Thread Tile 4x4 ----
1111 // 在 Level 1 基础上引入两层优化:
1112 // 1) float4 向量化加载: A/B 各用 1 条 128-bit load 取代 4 条 32-bit load
1113 // 2) Thread Tile 4x4: 每线程计算 16 个 C 元素, 提升计算/访存比 (AI 从
1114 // BK/2≈16 提升到 TM*TN*BK/2≈256), 减少线程总数带来的同步开销
1115 // C = A x B, C[M, N] = A[M, K] x B[K, N], A/B 均 row-major
1116 // BM=BN=128, BK=32, block(32, 32)=1024 线程, 每线程负责 4x4=16 个 C 元素
1117 // 1024 x 16 = 16384 = 128 x 128 ✓
1118 //
1119 // 线程到 4x4 tile 的映射 (与加载映射解耦, 独立计算更清晰):
1120 // m_tile = tid / 32 (0~31), 每 tile 4 行 → 行 [m_tile*4, m_tile*4+3], 覆盖 0~127
1121 // n_tile = tid % 32 (0~31), 每 tile 4 列 → 列 [n_tile*4, n_tile*4+3], 覆盖 0~127
1122 //
1123 // 加载映射 (每线程 4 个元素, float4):
1124 // A[128][32]: a_m = tid/8 (8 线程/行), a_k = (tid%8)*4 (4 列/线程) → 8x4=32 列 ✓
1125 // B[32][128]: b_k = tid/32 (32 线程/行), b_n = (tid%32)*4 (4 列/线程) → 32x4=128 列 ✓
1126 // row-major 下 A[m][k..k+3] 与 B[k][n..n+3] 均连续 → float4 load 合法
1127 //
1128 // △ Bank Conflict 提示 (面试加分点):
1129 // s_b[32][128] 上 warp 内 32 线程按 stride=4 访问 (tid%32 决定列 0,4,8,...,124)
1130 // → 每 4 个线程落同一 bank 不同地址 → 4-way bank conflict。生产代码可用
1131 // s_b[BK][BN+1] PAD 打散, 这里保持最简布局便于讲解。
1132 //
1133 // Grid: ((N + 127) / 128, (M + 127) / 128, 1)

```

```

1134 // Block: (32, 32, 1), 1024 线程
1135 // 假设: M/N 为 128 的倍数, K 为 32 的倍数 (与 Level 1 naive 版一致的边界约定)
1136 // source: LeetCUDA/kernels/sgemm/sgemm.cu (vec4 variant)
1137 __global__ void sgemm_vec4(float *a, float *b, float *c, int M, int N, int K) {
1138     constexpr int BM = 128;
1139     constexpr int BN = 128;
1140     constexpr int BK = 32;
1141     __shared__ float s_a[BM][BK]; // 128*32*4 = 16KB, float = 4 bytes
1142     __shared__ float s_b[BK][BN]; // 32*128*4 = 16KB
1143
1144     int bx = blockIdx.x;
1145     int by = blockIdx.y;
1146     int tx = threadIdx.x;
1147     int tid = threadIdx.y * blockDim.x + tx; // 0~1023
1148
1149     // 加载 A: 每线程加载 s_a[a_m][a_k..a_k+3] 共 4 个元素
1150     int load_smem_a_m = tid / 8; // 0~127, 8 线程/行
1151     int load_smem_a_k = (tid % 8) * 4; // 0,4,...,28
1152     // 加载 B: 每线程加载 s_b[b_k][b_n..b_n+3] 共 4 个元素
1153     int load_smem_b_k = tid / 32; // 0~31, 32 线程/行
1154     int load_smem_b_n = (tid % 32) * 4; // 0,4,...,124
1155
1156     int load_gmem_a_m = by * BM + load_smem_a_m;
1157     int load_gmem_b_n = bx * BN + load_smem_b_n;
1158
1159     // 4x4 Thread Tile 基址 (独立于加载映射), 这里compute索引的计算逻辑要和
1160     // load索引的计算逻辑分开, load/compute是可以独立索引的, 理解这点很重要。
1161     // 目标C Tile为[BM,BN]=[128x128], 有32x32线程, 则每个线程处理4x4 tile
1162     // 那么, 就可以不重不漏地覆盖[32x4,32x4]=[128x128]的大小
1163     int comp_smem_a_m_base = (tid / 32) * 4; // 0,4,8,...,124
1164     int comp_smem_b_n_base = (tid % 32) * 4; // 0,4,8,...,124
1165
1166     float sum[4][4] = {0.f};
1167     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1168         int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
1169         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1170         FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr]); // s_a [BM,BK]
1171         int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
1172         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1173         FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]); // s_b [BK,BN]
1174         __syncthreads();
1175
1176     #pragma unroll
1177     for (int k = 0; k < BK; ++k) {
1178         // 每次迭代加载 4 个 A 元素 + 4 个 B 元素, 再做 4x4=16 次 FMA
1179         float a_vals[4] = {s_a[comp_smem_a_m_base + 0][k],
1180                             s_a[comp_smem_a_m_base + 1][k],
1181                             s_a[comp_smem_a_m_base + 2][k],
1182                             s_a[comp_smem_a_m_base + 3][k]};
1183         float b_vals[4] = {s_b[k][comp_smem_b_n_base + 0],
1184                             s_b[k][comp_smem_b_n_base + 1],
1185                             s_b[k][comp_smem_b_n_base + 2],
1186                             s_b[k][comp_smem_b_n_base + 3]};
1187
1188     #pragma unroll
1189     for (int i = 0; i < 4; ++i) {
1190     #pragma unroll
1191         for (int j = 0; j < 4; ++j) {
1192             sum[i][j] += a_vals[i] * b_vals[j];
1193         }
1194     }
1195     __syncthreads();
1196 }

```

```

1197 // 存储 4x4: 每行 4 个元素连续 → 可用 float4 store (要求 N 为 4 的倍数以保证对齐)
1198 int store_gmem_c_m = by * BM + comp_smem_a_m_base;
1199 int store_gmem_c_n = bx * BN + comp_smem_b_n_base;
1200 #pragma unroll
1201 for (int i = 0; i < 4; ++i) {
1202     int store_gmem_c_addr = (store_gmem_c_m + i) * N + store_gmem_c_n;
1203     float4 reg_c;
1204     reg_c.x = sum[i][0];
1205     reg_c.y = sum[i][1];
1206     reg_c.z = sum[i][2];
1207     reg_c.w = sum[i][3];
1208     FLOAT4(c[store_gmem_c_addr]) = reg_c;
1209 }
1210 }
1211 }
1212
1213 // =====
1214 // Phase 7b: HGEMM — Tensor Core 路径 (MMA m16n8k16 + WGMMMA m64n128k16)
1215 // =====
1216 // 面试要点:
1217 // - MMA (Matrix Multiply-Accumulate): Ampere+ 的 Tensor Core 指令
1218 // - m16n8k16 含义: M=16, N=8, K=16 的矩阵乘, 结果 [16x8] = [16x16]x[16x8]
1219 // - ldmatrix: 从 shared memory 加载 16x16 的矩阵片段到寄存器 (4 条 32-bit
1220 // 寄存器)
1221 // - Multistage Pipeline: s2/s3/s4 个 stage, 用 cp.async 异步加载下一批数据
1222 // - Block Swizzle: 在 grid 维度做 swizzle, 改善 L2 cache locality
1223 //
1224 // ★ TN 布局详解 (面试高频考点):
1225 // TN 命名约定: 来自 BLAS 的 op(A) × op(B) 语义
1226 // BLAS 源自 Fortran, 默认列优先 (column-major) 存储
1227 // N = Normal (列优先, BLAS 原生格式)
1228 // T = Transposed (行优先, 相对 BLAS 来说是"转置过的")
1229 // 第一个字母 → A 的 op, 第二个字母 → B 的 op
1230 // 所以 TN 表示: A 是行优先 (相对 BLAS=Transposed), B 是列优先 (相对
1231 // BLAS=Normal) 即: C = op(A) × op(B) = A^T × B? 不对! 在 row-major
1232 // 视角下: TN = A row-major [MxK], B^T row-major [NxK] (等价于 B col-major [KxN]) 在 cuBLAS
1233 // 调用中: cublasGemmEx(..., CUBLAS_OP_T, CUBLAS_OP_N, ...)
1234 // T on A: BLAS 把 row-major 的 A 视为 A^T, 传 T 表示"转置回去"
1235 // N on B: B 已经是 BLAS 原生的 col-major, 无需转置
1236 //
1237 // 记忆口诀: TN = A行(T) B列(N), 第一字母 A 第二字母 B
1238 // T = row-major (行优先, 对 BLAS 来说是 transposed)
1239 // N = col-major (列优先, BLAS native = normal)
1240 //
1241 // LeetCUDA _nn 布局对比 (A/B 均 row-major 自然存储): C[MxN] = A[MxK] × B[KxN]
1242 // - A: row-major [M, K], B: row-major [K, N]
1243 // - 按 N=col-major/T=row-major 约定, 二者 BLAS 视角均为 T → cuBLAS 等效 (T,T)
1244 // - 问题: ldmatrix 默认加载 col-major, B 是 row-major 需要 .trans
1245 //
1246 // TN 布局: C[MxN] = A[MxK] × B[KxN], B 以 B^T=[NxK] row-major 存储 (A 行优先, B 列优先)
1247 // - A [MxK]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1248 // - B^T [NxK]: row-major → 全局索引 B[n*K+k] 即访问原 B 元素 (k,n) (△ 内维连续的是 K)
1249 // - 优势: B^T 已是 row-major, ldmatrix 无需 .trans, 天然匹配 MMA row.col
1250 // MMA 指令: mma.sync.aligned.m16n8k16.row.col
1251 // - row.col: A 输入 row-major, B 输入 col-major → 与 TN 布局天然匹配
1252 //
1253 // WGMMMA (Warp Group MMA, Hopper+):
1254 // - warpgroup 级指令 (128 threads = 4 warps), 异步执行
1255 // - m64n128k16: 一次处理 64x128x16 的矩阵乘 (总 tile 量 131072, 是 MMA m16n8k16 的 64 倍)
1256 // - Warp Specialization: Producer(128 threads) 做 TMA 搬运, Consumer(128
1257 // threads) 做计算
1258 // - TMA: 硬件 DMA, ~零寄存器开销, 支持 1D~5D 寻址
1259

```

```

1260 // =====
1261 // Phase 7b-1: MMA PTX 宏定义
1262 // =====
1263
1264 // ---- gmem → smem: cp.async ----
1265 // cp.async.commit_group / wait_group / wait_all 语义 (PTX ISA §9.7.9.25.3) :
1266 //   - commit_group: 将此前所有未提交的 cp.async 归入一个新的 async-group (per-thread) 。
1267 //   - wait_group N: 阻塞直到最多 N 个 async-group 尚未完成 (即 pending ≤ N) 。
1268 //     N=0 → 等所有 group 完成。与 wgmma.wait_group 语义一致。
1269 //   - wait_all: 等价于 commit_group + wait_group 0。
1270 #define CP_ASYNC_COMMIT_GROUP() asm volatile("cp.async.commit_group;\n" ::)
1271 #define CP_ASYNC_WAIT_ALL() asm volatile("cp.async.wait_all;\n" ::)
1272 #define CP_ASYNC_WAIT_GROUP(n) \
1273     asm volatile("cp.async.wait_group %0;\n" :: "n"(n))
1274
1275 // 注意: cg 只支持 16 bytes, ca 支持 4/8/16 bytes
1276 #define CP_ASYNC_CG(dst, src, bytes) \
1277     asm volatile( \
1278         "cp.async.cg.shared.global.L2::128B [%0], [%1], %2;\n" :: "r"(dst), \
1279         "l"(src), "n"(bytes))
1280
1281 // ---- ldmatrix: smem → register (Tensor Core 专用) ----
1282 // ldmatrix.sync.aligned.xN.m8n8.shared.b16
1283 // 每次加载 8x8 的 half 矩阵片段到 1/2/4 条 32-bit 寄存器
1284 // aligned: 要求 128-bit 对齐, trans: 转置加载
1285 #define LDMATRIX_X4(R0, R1, R2, R3, addr) \
1286     asm volatile( \
1287         "ldmatrix.sync.aligned.x4.m8n8.shared.b16 {%0, %1, %2, %3}, [%4];\n" \
1288         : "=r"(R0), "=r"(R1), "=r"(R2), "=r"(R3) \
1289         : "r"(addr))
1290
1291 #define LDMATRIX_X2(R0, R1, addr) \
1292     asm volatile("ldmatrix.sync.aligned.x2.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1293         : "=r"(R0), "=r"(R1) \
1294         : "r"(addr))
1295
1296 // ldmatrix.x2.trans: 转置加载 (用于 NN 布局中需要 col-major B 矩阵的场景)
1297 // FA 中 V[Bc,d] 为 row-major, 但 P@V 的 MMA 需要 col-major 的 B → 使用 trans
1298 #define LDMATRIX_X2_T(R0, R1, addr) \
1299     asm volatile( \
1300         "ldmatrix.sync.aligned.x2.trans.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1301         : "=r"(R0), "=r"(R1) \
1302         : "r"(addr))
1303
1304 // ---- mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 ----
1305 // m16n8k16: M=16, N=8, K=16 (Ampere Tensor Core 的基本 tile)
1306 // row.col: A 是 row-major, B 是 col-major
1307 // f16.f16.f16.f16: A/B 是 f16, C/D 是 f16 (f32 累加版本用 f32.f16.f16.f32)
1308 // 2 个输出寄存器 (RD0, RD1), 4 个 A 寄存器 + 2 个 B 寄存器
1309 // C 矩阵大小 16x8=128 元素 = 128 个 half; 32 线程分担, 每线程 4 half = 2 个 uint32
1310 #define HMMA16816(RD0, RD1, RA0, RA1, RA2, RA3, RB0, RB1, RC0, RC1) \
1311     asm volatile( \
1312         "mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 {%0, %1}, {%2, %3, " \
1313         "%4, %5}, {%6, %7}, {%8, %9};\n" \
1314         : "=r"(RD0), "=r"(RD1) \
1315         : "r"(RA0), "r"(RA1), "r"(RA2), "r"(RA3), "r"(RB0), "r"(RB1), "r"(RC0), \
1316         "r"(RC1))
1317
1318 // ---- MMA 辅助函数 ----
1319 // div_ceil: 整数除法向上取整
1320 #define HOST_DEVICE_INLINE __device__ __host__ inline
1321 HOST_DEVICE_INLINE int div_ceil(int a, int b) {
1322     return (a % b != 0) ? (a / b + 1) : (a / b);

```

```

1323 }
1324
1325 // =====
1326 // Phase 7b-2: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 (统一循环版)
1327 // =====
1328 // 面试重点 — Tile Hierarchy:
1329 //   MMA Atom:          m16n8k16 (1 条 MMA 指令处理的最小 tile)
1330 //   MMA Tile (more warps): 2x4=8 个 MMA atom → [2x16, 8x4]=[32,32]
1331 //   VAL Tile (more values): 4x4=16 expand → [32x4, 32x4]=[128,128]
1332 //   实际: kMmaTileM=2, kMmaTileN=4, kValTileM=4, kValTileN=4
1333 //           → BM=16x2x4=128, BN=8x4x4=128, Warps=2x4=8, Threads=8x32=256
1334 //
1335 // ★ TN 布局 (T=A 行优先, N=B 列优先):
1336 //   - A[M][K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1337 //   - B[K][N]: col-major (等价于 B^T[N][K] row-major) → 全局索引 B[n*K+k]
1338 //   - ldmatrix A: 用 x4 (非转置), A row-major 原生匹配
1339 //   - ldmatrix B: 用 x2 (非转置), B^T row-major 逐行加载 = B 的列, 天然匹配 MMA row.col
1340 //   - MMA 指令: mma.sync.aligned.m16n8k16.row.col → 天然匹配 TN 布局
1341 //
1342 // ★ 统一循环设计 (k 从 0 开始, 消除尾端重复代码):
1343 //   - k 从 0 开始: 每次迭代 k 直接对应 tile k, sel = k % kStages (零偏移)
1344 //   - 加载语义为"预取未来": 迭代 k 加载 tile (k+kStages-1) 供后续使用
1345 //   - cp.async 条件化: 仅当 k+kStages-1 < NUM_K_TILES 时加载
1346 //   - WAIT_GROUP 自适应: 满载期用 kStages-2, 尾部排空用 0
1347 //
1348 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1349 // Block: (256, 1, 1), 8 warps
1350 // source: LeetCUDA/kernels/hgemm/mma/basic/hgemm_mma_stage_tn.cu
1351 // =====
1352 template <const int kMmaM = 16,           // MMA atom M dim (m16n8k16)
1353           const int kMmaN = 8,           // MMA atom N dim
1354           const int kMmaK = 16,          // MMA atom K dim, also BK tile = kMmaK
1355           const int kMmaTileM = 2,       // warps along M, 2 → warp tile M = 32
1356           const int kMmaTileN = 4,       // warps along N, 4 → warp tile N = 32
1357           const int kValTileM = 4,       // value-repeat along M, BM = 16*2*4 = 128
1358           const int kValTileN = 4,       // value-repeat along N, BN = 8*4*4 = 128
1359           const int kStages = 3,         // cp.async pipeline depth
1360           const bool kBlockSwizzle = false> // enable 3D grid swizzle for L2 locality
1361 __global__ void __launch_bounds__(256)
1362   hgemm_mma_stages_tn(half *A, half *B, half *C, int M, int N, int K) {
1363   // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1364   const int bx = ((int)kBlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
1365   const int by = blockIdx.y;
1366   constexpr int BM = kMmaM * kMmaTileM * kValTileM; // 16*2*4=128
1367   constexpr int BN = kMmaN * kMmaTileN * kValTileN; // 8*4*4=128
1368   constexpr int BK = kMmaK; // 16
1369
1370   // Dynamic shared memory: kStages 个 stage 的 A 和 B
1371   // TN 布局: s_a[BM][BK]=[128][16](A row-major), s_b[BN][BK]=[128][16](B^T
1372   // row-major, 即 B col-major [KxN] 在 smem 中按 B^T[NxK] 存储)
1373   // 原始实现会按配置决定是否给 A/B 的 K 维加 PAD; 尤其 B 在 TN 布局下常见会额外
1374   // 加 B_PAD 来打散 bank 映射, 避免按列访问时出现明显 bank conflict. 这里先保留最简 PAD=0 版本。
1375   extern __shared__ half smem[];
1376   half *s_a = smem;
1377   half *s_b = smem + kStages * BM * BK; // A 和 B 连续存放
1378   constexpr int s_a_stage_offset = BM * BK; // 128*16
1379   constexpr int s_b_stage_offset = BN * BK; // 128*16 △ BN(128)xBK(16) = B^T row-major
1380   const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1381   const int warp_id = tid / WARP_SIZE; // 0~7
1382   const int lane_id = tid % WARP_SIZE; // 0~31
1383   // warp_m变化快(0->0,1->1), warp_n变化慢([0,1]->0,[2,3]->1,...), 因此,
1384   // 这种2x4的MMA(Warp) layout是按照col major的顺序来排列MMA0~MMA7的
1385   const int warp_m = warp_id % 2; // 0,1 (M 方向 2 个 warp)

```

```

1386 const int warp_n = warp_id / 2; // 0,1,2,3 (N 方向 4 个 warp)
1387
1388 // 线程到 global memory 的映射 (用于加载 A 和 B) 共 256 个线程
1389 // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major (内维连续的是 K)
1390 int load_smem_a_m = tid / 2; // 0~127
1391 int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1392 int load_smem_b_n = tid / 2; // 0~127 → B^T 的 N 方向 (row-major 的行)
1393 int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8 → B^T 的 K 方向 (row-major 的列)
1394 int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号 = M 方向的 tile 起始 + 线程偏移
1395 int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号 = N 方向的 tile 起始 + 线程偏移
1396 if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1397     return;
1398
1399 // 8个Warps(MMAs)的排布是M方向2个, N方向4个, 则MMA Atom一次性能处理的tile大小是[2*16, 4*8]=[32, 32].
1400 // CUDA中每个block能放的线程数是有上限的 (warp数量有限), 一般为4或者8个warps。为了能处理更大的C tile,
1401 // 需要把MMA Atom的tile再M方向和N方向各自重复4次, 得到[128, 128]的C block tile, 这就是Value Tile的作用。
1402 // MMA Tile对应到cutlass cute中的TiledMMA的概念, Value Tile对应到cutlass cute中的PermuteMNK的概念。
1403 uint32_t RC[kValTileM][kValTileN][2] = {0}; // 初始化为 0
1404
1405 // CVTA: 一次转换 smem 基地址, 避免每次 cp.async 都做转换
1406 uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1407 uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1408
1409 // 预加载前 (kStages-1) 个 stage
1410 // TN 布局: A 的 gmem 索引用 m*K+k(row-major), B^T 用 n*K+k(row-major, 即 B col-major [K×N])
1411 #pragma unroll
1412 for (int k = 0; k < (kStages - 1); ++k) {
1413     int load_gmem_a_k = k * BK + load_smem_a_k;
1414     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: [m][k]
1415     uint32_t load_smem_a_ptr = (smem_a_base_ptr +
1416         (k * s_a_stage_offset + load_smem_a_m * BK + load_smem_a_k) * sizeof(half)
1417     );
1418     CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1419
1420     int load_gmem_b_k = k * BK + load_smem_b_k;
1421     // B^T: [n][k] row-major (即 B[k][n] col-major) Δ
1422     int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1423     uint32_t load_smem_b_ptr = (smem_b_base_ptr +
1424         (k * s_b_stage_offset + load_smem_b_n * BK + load_smem_b_k) * sizeof(half)
1425     );
1426     CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1427
1428     CP_ASYNC_COMMIT_GROUP();
1429 }
1430
1431 CP_ASYNC_WAIT_GROUP(kStages - 2); // 允许有 kStages-2 个group未完成
1432 __syncthreads();
1433
1434 // 统一循环: k 从 0 开始, 每次迭代负责 tile k (加载 + 计算合并为单循环)
1435 const int NUM_K_TILES = div_ceil(K, BK);
1436 #pragma unroll
1437 for (int k = 0; k < NUM_K_TILES; ++k) {
1438     int smem_sel = k % kStages; // 计算 tile k 的 stage
1439     int smem_sel_next = (k + kStages - 1) % kStages; // 预加载目标 stage
1440
1441     // 条件加载: 预加载 tile (k+kStages-1) 供将来使用。因为在prefetch loop中已经
1442     // loaded (kStages-1) 个 stage, 因此这里是从 tile (k+kStages-1) 开始预加载
1443     // TN 布局: A 的 gmem 地址用 m*K+k (row-major), B^T 的 gmem 地址用 n*K+k
1444     // (row-major, 内维连续的是 K)
1445     if (k + kStages - 1 < NUM_K_TILES) {
1446         int load_gmem_a_k = (k + kStages - 1) * BK + load_smem_a_k;
1447         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: row-major [m][k]
1448         int load_gmem_b_k = (k + kStages - 1) * BK + load_smem_b_k;

```

```

1449     int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k; // B^T: row-major [n][k]
1450
1451     uint32_t load_smem_a_ptr =
1452         (smem_a_base_ptr + (smem_sel_next * s_a_stage_offset +
1453             load_smem_a_m * BK + load_smem_a_k) *
1454             sizeof(half));
1455     CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1456
1457     uint32_t load_smem_b_ptr =
1458         (smem_b_base_ptr + (smem_sel_next * s_b_stage_offset +
1459             load_smem_b_n * BK + load_smem_b_k) *
1460             sizeof(half));
1461     CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1462     CP_ASYNC_COMMIT_GROUP();
1463 }
1464
1465 // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
1466 // TN 布局关键: A 用 x4 (非转置), 因为 A 是 row-major; B 用 x2 (非转置), smem 中
1467 // B^T 为 row-major, 逐行加载即得 B 的列, 天然匹配 col-major B
1468 uint32_t RA[kValTileM][4]; // [4][4], M方向4次repeat, A regs = 4 uint32_t per MMA
1469 uint32_t RB[kValTileN][2]; // [4][2], N方向4次repeat, B regs = 2 uint32_t per MMA
1470
1471 // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A, 非转置)
1472 #pragma unroll
1473 for (int i = 0; i < kValTileM; ++i) {
1474     // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1475     // 其中 warp_m {0,1}; warp_m_0 offsets {0,16,32,48}; warp_m_1 offsets {64,80,96,112}
1476     int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
1477     // {0,16,32,...,112} + {0~15} = {0~127}, 按照col-major的顺序访问A的4个8x8 matrix (16x16)
1478     // ldmatrix.{...}.x4.{...} 需要warp内32个线程都参与, 每个线程提供一个有效的, 不重叠的addr
1479     int lane_smem_a_m = warp_smem_a_m + lane_id % 16; // t{0...15}=0~15, t{16...31}=0~15
1480     int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8, t{0...15}=0, t{16...31}=8
1481     uint32_t lane_smem_a_ptr =
1482         (smem_a_base_ptr +
1483             (smem_sel * s_a_stage_offset + lane_smem_a_m * BK + lane_smem_a_k) *
1484             sizeof(half));
1485     LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1486 }
1487
1488 // ldmatrix.x2: 加载 B 的 k16n8 片段 (非转置)
1489 // 为什么不用 .trans? 因为 smem 中存的是 B^T row-major [N][K],
1490 // ldmatrix 逐行加载 B^T 的行 = B 的列, 天然给出 col-major B fragment → 直接匹配 MMA row.col
1491 #pragma unroll
1492 for (int j = 0; j < kValTileN; ++j) {
1493     // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1494     // warp_n_0 offsets {0,8,16,24}; warp_n_1 offsets {32,40,48,56}
1495     // warp_n_2 offsets {64,72,80,88}; warp_n_3 offsets {96,104,112,120}
1496     int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
1497     // {0,8,...,120} + {0~7} = {0~127}, 按照row-major的顺序访问B^T的2个8x8 matrix (8x16)
1498     // ldmatrix.{...}.x2.{...} 需要warp内前16个线程参与, 后16个线程传的addr会被忽略
1499     int lane_smem_b_n = warp_smem_b_n + lane_id % 8; // t{0...7}=0~7, t{8...15}=0~7
1500     int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // t{0...7}=0, t{8...15}=8
1501     uint32_t lane_smem_b_ptr =
1502         (smem_b_base_ptr +
1503             (smem_sel * s_b_stage_offset + lane_smem_b_n * BK + lane_smem_b_k) *
1504             sizeof(half));
1505     LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1506 }
1507
1508 // MMA compute: 每个Warp(MMA) 在M方向重复VAL_TILE_M(4)次, 在N方向重复VAL_TILE_N(4)次
1509 #pragma unroll
1510 for (int i = 0; i < kValTileM; ++i) {
1511     #pragma unroll

```

```

1512     for (int j = 0; j < kValTileN; ++j) {
1513         HMMA16816(RC[i][j][0], RC[i][j][1], // C fragment
1514                 RA[i][0], RA[i][1], RA[i][2], RA[i][3], // A fragment
1515                 RB[j][0], RB[j][1], // B fragment
1516                 RC[i][j][0], RC[i][j][1]);
1517     }
1518 }
1519
1520 // 自适应等待: 流水线满载期用 kStages-2, 尾部排空用 0
1521 if (k + kStages - 1 < NUM_K_TILES) {
1522     CP_ASYNC_WAIT_GROUP(kStages - 2);
1523 } else { // 对于尾部的 k, 等待所有剩余的 cp.async 完成
1524     CP_ASYNC_WAIT_GROUP(0);
1525 }
1526 __syncthreads();
1527 }
1528
1529 // Epilogue: 寄存器 → global memory (通过 warp shuffle + 128-bit store)
1530 {
1531     for (int i = 0; i < kValTileM; ++i) {
1532         // RC[kValTileM][kValTileN][2] 中 RC[...][...][2] 中保存了2个 uint32
1533         // 寄存器, 每个uint32 寄存器表示两个临近的fp16值: {c0,c1}, 然后RC[...][...][0]
1534         // 和RC[...][...][1]代表的是按照col-major排布的2个8x8子矩阵上 (不同物理行, 跨8x8)
1535         // 同一个位置上的元素, 也就是, 实际代表了2个不同的行的元素, 因此要分开RC0, RC1;
1536         // RC0表示第一行, 8个half, 可以用4个uint32寄存器来装; 同理, RC1表示第二行。
1537         uint32_t RC0[kValTileN][4]; // 32 bits x 4 = 128 bits = 8 half
1538         uint32_t RC1[kValTileN][4]; // 32 bits x 4 = 128 bits = 8 half
1539         // =====
1540         // MMA m16n8k16 C fragment — a single 16x8 matrix (registers per warp):
1541         // Thread t holds 4 half values → RC[0] for rows 0-7, RC[1] for rows 8-15.
1542         // Mapping: row = t/4, col-pair = t%4 (each uint32 = 2 half values).
1543         //
1544         //      cols: c0-1    c2-3    c4-5    c6-7
1545         //  r0:  T0{c0,c1}  T1{c2,c3}  T2{c4,c5}  T3{c6,c7}  |
1546         //  r1:  T4{c0,c1}  T5{c2,c3}  T6{c4,c5}  T7{c6,c7}  |
1547         //  r2:  T8{c0,c1}  T9{c2,c3}  T10{c4,c5} T11{c6,c7} | RC[0]
1548         //  ...
1549         //  r7:  T28{c0,c1} T29{c2,c3} T30{c4,c5} T31{c6,c7} |
1550         //  r8:  T0{c0,c1}  T1{c2,c3}  T2{c4,c5}  T3{c6,c7}  |
1551         //  r9:  T4{c0,c1}  T5{c2,c3}  T6{c4,c5}  T7{c6,c7}  |
1552         // r10:  T8{c0,c1}  T9{c2,c3}  T10{c4,c5} T11{c6,c7} | RC[1]
1553         //  ...
1554         // r15:  T28{c0,c1} T29{c2,c3} T30{c4,c5} T31{c6,c7} |
1555         //
1556         // Within a 4-lane group (e.g. T0-T3 for row 0):
1557         // lane+0 holds {c0,c1}, lane+1 holds {c2,c3},
1558         // lane+2 holds {c4,c5}, lane+3 holds {c6,c7}.
1559         // shfl 从 lane+1/+2/+3 各收 2 个 half 到 lane+0, 4 次 shuffle 凑齐
1560         // 一行 8 个 half = {c0,c1,c2,c3,c4,c5,c6,c7}, 再由 lane%4==0 128-bit store。
1561         // =====
1562 #pragma unroll
1563         for (int j = 0; j < kValTileN; ++j) {
1564             RC0[j][0] = RC[i][j][0];
1565             RC1[j][0] = RC[i][j][1];
1566             RC0[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
1567             RC0[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
1568             RC0[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
1569             RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
1570             RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
1571             RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
1572         }
1573         // 每 4 个 lane 中只有 lane 0 做 128-bit store
1574         // lane_id / 4 → 行号映射 (每 4 个连续 lane 负责上8x8矩阵和下8x8矩阵各一行):

```

```

1575 //
1576 // | lane_id | lane_id/4 | RC[0] 的行 | RC[1] 的行 |
1577 // |-----|-----|-----|-----|
1578 // | 0~3    | 0        | row 0     | row 8     |
1579 // | 4~7    | 1        | row 1     | row 9     |
1580 // | 8~11   | 2        | row 2     | row 10    |
1581 // | 12~15  | 3        | row 3     | row 11    |
1582 // | 16~19  | 4        | row 4     | row 12    |
1583 // | 20~23  | 5        | row 5     | row 13    |
1584 // | 24~27  | 6        | row 6     | row 14    |
1585 // | 28~31  | 7        | row 7     | row 15    |
1586 //
1587 if (lane_id % 4 == 0) {
1588     // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1589     int store_warp_smem_c_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
1590     int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
1591 #pragma unroll
1592     for (int j = 0; j < kValTileN; ++j) {
1593         // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1594         int store_warp_smem_c_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
1595         int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
1596         int store_gmem_c_addr_0 = store_lane_gmem_c_m * N + store_lane_gmem_c_n;
1597         int store_gmem_c_addr_1 = (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
1598         // 128-bit store: 一次写入 8 个 half
1599         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
1600             *reinterpret_cast<float4*>(&RC0[j][0]);
1601         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
1602             *reinterpret_cast<float4*>(&RC1[j][0]);
1603     }
1604 }
1605 }
1606 }
1607 }
1608
1609 // =====
1610 // Phase 7b-3: HGEMM MMA Swizzle — m16n8k16 + multistage pipeline + TN 布局
1611 // + XOR swizzle 消除 smem bank conflict (统一循环版)
1612 // =====
1613 // 面试要点 (Swizzle 原理 — smem Bank Conflict 消除) :
1614 // - 问题: ldmatrix 以 8x8 片段 (m8n8) 从 smem 加载 16x16 的矩阵。同一 warp 中
1615 //   不同 lane 访问的地址在 bank 上产生冲突 → 串行化 (most common: 2/4-way)。
1616 // - XOR swizzle 方案: 对列地址做 `((j>>3) ^ (i>>2)) % 2 << 3` 的 XOR 置换,
1617 //   将连续 4 行的 bank 映射打散。每 4 行为一组的 pattern 交替:
1618 //   rows 0~3: 列 0→地址偏移 0, 列 8→地址偏移 8
1619 //   rows 4~7: 列 0→地址偏移 8, 列 8→地址偏移 0 (XOR 翻转)
1620 //   rows 8~11: repeat rows 0~3 pattern
1621 //   rows 12~15: repeat rows 4~7 pattern
1622 // - 效果: 原来 n-way 的 bank conflict 降低到 1-way (fully conflict-free)。
1623 // - 优势: 无需 smem PAD (不浪费空间), 原理上完全消除特定 pattern 的 bank conflict。
1624 // - 局限性: 要求 kColStride ≤ 16 (即 BK ≤ 16), kStep ∈ {4,8}。
1625 //   对 MMA m16n8k16 的 BK=16 而言刚好满足。
1626 //
1627 // 参考: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1628 // 参考: https://zhuanlan.zhihu.com/p/4746910252
1629
1630 // ---- Swizzle 辅助函数 ----
1631
1632 // swizzle_permuted_j: 对 smem 列索引 j 做 XOR 置换, 消除 bank conflict。
1633 // i: row index; j: col index.
1634 // e.g kColStride = 16, kStep = 8 -> load 8 half as 128 bits memory issue.
1635 // 公式: ((j / kStep) ^ (i / 4)) % (kColStride / kStep) * kStep
1636 // 用位运算展开 (kStep=8) : (((j >> 3) ^ (i >> 2)) % (kColStride >> 3)) << 3
1637 // 限制: kColStride ≤ 16 (BK ≤ 16), kStep ∈ {4, 8}, kColStride % kStep == 0

```

```

1638 // source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1639 template <const int kColStride = 16, const int kStep = 8>
1640 static __device__ __forceinline__ int swizzle_permuted_j(int i, int j) {
1641     // for col_stride > 16, we have to permute it using col major ZigZag order.
1642     // e.g, A smem logical layout [Br,d]=[Br,64] -> store layout [4][Br][16].
1643     static_assert(kColStride <= 16, "kColStride must <= 16");
1644     // swizzle: ((int(j / kStep) ^ int(i / 4)) % int(kColStride / kStep)) * kStep;
1645     static_assert(kStep == 4 || kStep == 8, "kStep must be 8 or 4.");
1646     static_assert(kColStride % kStep == 0,
1647         "kColStride must be multiple of kStep.");
1648     if constexpr (kStep == 8) {
1649         // j >> 3: 表示8个half(=16 bytes)数据为一个chunk; i >> 2: 表示4行为一组
1650         // kColStride >> 3: 按照kStep=8计算chunk idx, kColStride=16 -> {0, 1}
1651         // 最后的 << 3: 将chunk idx转换为实际的列偏移量 (8个half/chunk), {0, 8}
1652         return (((j >> 3) ^ (i >> 2)) % (kColStride >> 3)) << 3;
1653     } else {
1654         static_assert(kStep == 4);
1655         return (((j >> 2) ^ (i >> 2)) % (kColStride >> 2)) << 2;
1656     }
1657 }
1658
1659 // swizzle_A: A 矩阵专用封装 (kMmaAtomK=16, kStep=8)。
1660 // 16 行 (一个 MMA atom 的 M 维) 内的 swizzle pattern:
1661 // 8个half=16 bytes=128 bits=4 banks (32 bits/bank) 组成一个phase,
1662 // 触发一次合并的memory transaction. 这里的col=16的swizzle, 相当于
1663 // TMA中的SWIZZLE_32B pattern.
1664 // -----
1665 // -col 0~16, step 8--
1666 // -----
1667 // | row 0 | (0, 8) |
1668 // | row 1 | (0, 8) |
1669 // | row 2 | (0, 8) |
1670 // | row 3 | (0, 8) |
1671 // -----
1672 // | row 4 | (8, 0) |
1673 // | row 5 | (8, 0) |
1674 // | row 6 | (8, 0) |
1675 // | row 7 | (8, 0) |
1676 // -----
1677 // | row 8 | (0, 8) |
1678 // | row 9 | (0, 8) |
1679 // | row 10 | (0, 8) |
1680 // | row 11 | (0, 8) |
1681 // -----
1682 // | row 12 | (8, 0) |
1683 // | row 13 | (8, 0) |
1684 // | row 14 | (8, 0) |
1685 // | row 15 | (8, 0) |
1686 // -----
1687 // source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1688 template <const int kMmaAtomK = 16>
1689 static __device__ __forceinline__ int swizzle_A(int i, int j) {
1690     return swizzle_permuted_j<kMmaAtomK, 8>(i, j);
1691 }
1692
1693 // swizzle_B: B 矩阵专用封装 (与 A 相同的 pattern)。
1694 // B^T smem 布局 [BN][BK]=[128][16] 在 BK 维做 swizzle,
1695 // pattern 与 A 完全相同 (kMmaAtomK=16, kStep=8)。
1696 // source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1697 template <const int kMmaAtomK = 16>
1698 static __device__ __forceinline__ int swizzle_B(int i, int j) {
1699     return swizzle_permuted_j<kMmaAtomK, 8>(i, j);
1700 }

```

```

1701 // =====
1702 // Phase 7b-3: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 + XOR swizzle
1703 //           + Register Double Buffering (kValTileK=2, BK=32 统一 tile)
1704 // =====
1705 // 在 Phase 7b-2 的 smem XOR swizzle 基础上增加寄存器双缓冲:
1706 // - kValTileK=2: 每个 BK tile 包含 2 个 kMmaK slice (BK = kMmaK * kValTileK = 32)
1707 // - BK=32 统一 tile: kMmaK=0 和 kMmaK=1 数据连续存放在同一个 BK=32 宽的 smem tile 中
1708 // - RA[2][kValTileM][4] / RB[2][kValTileN][2]: 双份寄存器乒乓切换
1709 // - ldmatrix 与 MMA 计算重叠: 加载 k_step=1 的同时用另一组寄存器做 k_step=0 计算
1710 //
1711 // ★ smem 布局:
1712 //   s_a: [stage0][stage1][stage2], 每个 stage = BM × BK = 128 × 32
1713 //   s_b: 紧接 s_a 之后
1714 //   每个 stage 内 k_step=0 列偏移 0, k_step=1 列偏移 +kMmaK(=16)
1715 //
1716 // ★ 寄存器双缓冲时间线 (每 k 迭代内):
1717 //   Step 1: G→S cp.async 预取 stage(k+kStages-1) 全部 k_step (条件)
1718 //   Step 2: MMA k_step=0 (用 reg[load], 已在上轮 post-wait 中 ldmatrix)
1719 //   Step 3: for k_step=1..kValTileK-1: ldmatrix(列偏移 k_step*kMmaK)→reg[store], flip, MMA→reg[load]
1720 //   Step 4: adaptive wait + __syncthreads
1721 //   Step 5: S→R ldmatrix 预加载 stage(k+1) k_step=0 → reg[store] (条件)
1722 //
1723 // 与 dsmem 版本的对比:
1724 // - dsmem: smem 分两个区域存放 kMmaK=0 和 kMmaK=1, smem 翻倍
1725 // - BK=32: kMmaK=0/1 连续存同一 tile 内, smem 不变, gmem 索引用 k*BK 直接定位
1726 //
1727 // 参考: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle_x2.cu
1728 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1729 // Block: (256, 1, 1), 8 warps
1730 template <const int kMmaM = 16,           // MMA atom M dim (m16n8k16)
1731           const int kMmaN = 8,           // MMA atom N dim
1732           const int kMmaK = 16,          // MMA atom K dim
1733           const int kMmaTileM = 2,        // warps along M, 2 → warp tile M = 32
1734           const int kMmaTileN = 4,        // warps along N, 4 → warp tile N = 32
1735           const int kValTileM = 4,        // value-repeat along M, BM = 16*2*4 = 128
1736           const int kValTileN = 4,        // value-repeat along N, BN = 8*4*4 = 128
1737           const int kValTileK = 2,        // MMA_K slices per BK tile, BK = kMmaK*2 = 32
1738           const int kStages = 3,          // cp.async pipeline depth
1739           const bool kBlockSwizzle = false> // enable 3D grid swizzle for L2 locality
1740 __global__ void __launch_bounds__(256)
1741   hgemm_mma_stages_tn_swizzle(half *A, half *B, half *C, int M, int N, int K) {
1742   // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1743   const int bx = ((int)kBlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
1744   const int by = blockIdx.y;
1745   constexpr int BM = kMmaM * kMmaTileM * kValTileM; // 16*2*4=128
1746   constexpr int BN = kMmaN * kMmaTileN * kValTileN; // 8*4*4=128
1747   constexpr int BK = kMmaK * kValTileK;             // 16*2=32
1748
1749   // kStages stages, each with BM×BK for A, BN×BK for B
1750   // smem: kValTileK 个 kMmaK slice 连续存放在同一个 BK=32 宽 tile 中
1751   extern __shared__ half smem[];
1752   half *s_a = smem;
1753   half *s_b = smem + kStages * BM * BK;
1754   constexpr int s_a_stage_offset = BM * BK;
1755   constexpr int s_b_stage_offset = BN * BK;
1756
1757   const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1758   const int warp_id = tid / WARP_SIZE;
1759   const int lane_id = tid % WARP_SIZE;
1760   const int warp_m = warp_id % 2; // 0,1 (M 方向 2 个 warp)
1761   const int warp_n = warp_id / 2; // 0,1,2,3 (N 方向 4 个 warp)
1762
1763

```

```

1764 // 线程到 global memory 的映射 (用于加载 A 和 B) 共 256 个线程
1765 // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major (内维连续的是 K)
1766 // 注意: smem_a_k 和 smem_b_k 依然使用 0/8, 虽然BK=16*2=32, 在后续的kValTileK循环中
1767 // 会加上 k_step*kMmaK=0/16, 最终得到 smem 中的列偏移 0/8/16/24, 正好覆盖 BK=32
1768 int load_smem_a_m = tid / 2; // 0~127
1769 int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1770 int load_smem_b_n = tid / 2; // 0~127 → B^T 的 N 方向 (row-major 的行)
1771 int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8 → B^T 的 K 方向 (row-major 的列)
1772 int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号 = M 方向的 tile 起始 + 线程偏移
1773 int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号 = N 方向的 tile 起始 + 线程偏移
1774 if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1775     return;
1776
1777 // 8个Warps(MMAs)的排布是M方向2个, N方向4个, 则MMA Atom一次性能处理的tile大小是[2*16, 4*8]=[32, 32].
1778 // CUDA中每个block能放的线程数是有上限的 (warp数量有限), 一般为4或者8个warps。为了能处理更大的C tile,
1779 // 需要把MMA Atom的tile再M方向和N方向各自重复4次, 得到[128, 128]的C block tile, 这就是Value Tile的作用。
1780 // MMA Tile对应到cutlass cute中的TiledMMA的概念, Value Tile对应到cutlass cute中的PermuteMNMK的概念。
1781 uint32_t RC[kValTileM][kValTileN][2] = {0}; // 初始化为 0
1782
1783 uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1784 uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1785
1786 // Prefetch (kStages-1) stages: load all k_step slices for each early stage
1787 #pragma unroll
1788 for (int k = 0; k < (kStages - 1); ++k) {
1789     #pragma unroll
1790     for (int k_step = 0; k_step < kValTileK; ++k_step) {
1791         int load_gmem_a_k = k * BK + k_step * kMmaK + load_smem_a_k;
1792         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1793         int load_gmem_b_k = k * BK + k_step * kMmaK + load_smem_b_k;
1794         int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1795
1796         uint32_t load_smem_a_ptr =
1797             (smem_a_base_ptr +
1798              (k * s_a_stage_offset + load_smem_a_m * BK +
1799               k_step * kMmaK +
1800               swizzle_A<kMmaK>(load_smem_a_m, load_smem_a_k)) *
1801              sizeof(half));
1802         CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1803
1804         uint32_t load_smem_b_ptr =
1805             (smem_b_base_ptr +
1806              (k * s_b_stage_offset + load_smem_b_n * BK +
1807               k_step * kMmaK +
1808               swizzle_B<kMmaK>(load_smem_b_n, load_smem_b_k)) *
1809              sizeof(half));
1810         CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1811     }
1812     CP_ASYNC_COMMIT_GROUP();
1813 }
1814
1815 CP_ASYNC_WAIT_GROUP(kStages - 2);
1816 __syncthreads();
1817
1818 // Register double buffers: RA[0/1] for k_step=0/1 A data, RB[0/1] for B data
1819 uint32_t RA[2][kValTileM][4];
1820 uint32_t RB[2][kValTileN][2];
1821 int reg_st_idx = 0; // write target for ldmatrix
1822 int reg_ld_idx = 1; // read source for MMA
1823
1824 // Initial ldmatrix: load stage 0, k_step=0 (kMmaK=0 列) → reg[0]
1825 #pragma unroll
1826 for (int i = 0; i < kValTileM; ++i) {

```

```

1827     int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
1828     int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
1829     int lane_smem_a_k = (lane_id / 16) * 8;
1830     uint32_t lane_smem_a_ptr =
1831         (smem_a_base_ptr +
1832          (0 * s_a_stage_offset + lane_smem_a_m * BK +
1833           swizzle_A<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *
1834           sizeof(half));
1835     LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
1836                RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
1837                lane_smem_a_ptr);
1838 }
1839 #pragma unroll
1840 for (int j = 0; j < kValTileN; ++j) {
1841     int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
1842     int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
1843     int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
1844     uint32_t lane_smem_b_ptr =
1845         (smem_b_base_ptr +
1846          (0 * s_b_stage_offset + lane_smem_b_n * BK +
1847           swizzle_B<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
1848           sizeof(half));
1849     LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
1850                lane_smem_b_ptr);
1851 }
1852
1853 // 统一循环: k 从 0 开始, 条件 G→S / S→R / wait 处理所有边界情况
1854 // BK = kMmaK * kValTileK = 32, 每个 k 迭代覆盖 BK=32 个 K 元素
1855 const int NUM_K_TILES = div_ceil(K, BK);
1856 #pragma unroll
1857 for (int k = 0; k < NUM_K_TILES; ++k) {
1858     int smem_sel = k % kStages;
1859     int smem_sel_next = (k + kStages - 1) % kStages;
1860
1861     // G→S: 条件预取 stage(k+kStages-1) 全部 k_step slice
1862     if (k + kStages - 1 < NUM_K_TILES) {
1863         #pragma unroll
1864         for (int k_step = 0; k_step < kValTileK; ++k_step) {
1865             int load_gmem_a_k =
1866                 (k + kStages - 1) * BK + k_step * kMmaK + load_smem_a_k;
1867             int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1868             int load_gmem_b_k =
1869                 (k + kStages - 1) * BK + k_step * kMmaK + load_smem_b_k;
1870             int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1871
1872             uint32_t load_smem_a_ptr =
1873                 (smem_a_base_ptr +
1874                  (smem_sel_next * s_a_stage_offset + load_smem_a_m * BK +
1875                   k_step * kMmaK +
1876                  swizzle_A<kMmaK>(load_smem_a_m, load_smem_a_k)) *
1877                  sizeof(half));
1878             CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1879
1880             uint32_t load_smem_b_ptr =
1881                 (smem_b_base_ptr +
1882                  (smem_sel_next * s_b_stage_offset + load_smem_b_n * BK +
1883                   k_step * kMmaK +
1884                  swizzle_B<kMmaK>(load_smem_b_n, load_smem_b_k)) *
1885                  sizeof(half));
1886             CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1887         }
1888         CP_ASYNC_COMMIT_GROUP();
1889     }

```

```

1890 // 内层 k_step 循环: flip → ldmatrix(k_step+1) → MMA, 乒乓交替
1891 // 初始: st=0, ld=1, reg[0]=preload k_step=0; reg[1]=未初始化
1892 // 每轮: flip→ldmatrix next k_step→reg[st], MMA reg[ld] (k_step+1 的 ldmatrix 条件化)
1893
1894 #pragma unroll
1895 for (int k_step = 0; k_step < kValTileK; ++k_step) {
1896     reg_st_idx ^= 1; // 0 -> 1; 1 -> 0
1897     reg_ld_idx ^= 1; // 1 -> 0; 0 -> 1
1898
1899     if (k_step + 1 < kValTileK) {
1900         int smem_k_offset = (k_step + 1) * kMmaK;
1901 #pragma unroll
1902         for (int i = 0; i < kValTileM; ++i) {
1903             int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
1904             int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
1905             int lane_smem_a_k = (lane_id / 16) * 8;
1906             uint32_t lane_smem_a_ptr =
1907                 (smem_a_base_ptr +
1908                  (smem_sel * s_a_stage_offset + lane_smem_a_m * BK +
1909                   smem_k_offset +
1910                   swizzle_A<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *
1911                  sizeof(half));
1912             LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
1913                         RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
1914                         lane_smem_a_ptr);
1915         }
1916 #pragma unroll
1917         for (int j = 0; j < kValTileN; ++j) {
1918             int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
1919             int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
1920             int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
1921             uint32_t lane_smem_b_ptr =
1922                 (smem_b_base_ptr +
1923                  (smem_sel * s_b_stage_offset + lane_smem_b_n * BK +
1924                   smem_k_offset +
1925                   swizzle_B<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
1926                  sizeof(half));
1927             LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
1928                         lane_smem_b_ptr);
1929         }
1930     }
1931 }
1932
1933 #pragma unroll
1934 for (int i = 0; i < kValTileM; ++i) {
1935 #pragma unroll
1936     for (int j = 0; j < kValTileN; ++j) {
1937         HMMA16816(RC[i][j][0], RC[i][j][1],
1938                  RA[reg_ld_idx][i][0], RA[reg_ld_idx][i][1],
1939                  RA[reg_ld_idx][i][2], RA[reg_ld_idx][i][3],
1940                  RB[reg_ld_idx][j][0], RB[reg_ld_idx][j][1],
1941                  RC[i][j][0], RC[i][j][1]);
1942     }
1943 }
1944 }
1945
1946 // 自适应等待: 满载期用 kStages-2, 尾部排空用 0
1947 if (k + kStages - 1 < NUM_K_TILES) {
1948     CP_ASYNC_WAIT_GROUP(kStages - 2);
1949 } else {
1950     CP_ASYNC_WAIT_GROUP(0);
1951 }
1952 __syncthreads();

```

```

1953 // 从下一阶段加载 k_step=0 数据, 供下一轮 k 迭代使用, 此时 reg_st_idx=0
1954 if (k + 1 < NUM_K_TILES) {
1955     smem_sel = (k + 1) % kStages; // old smem_sel + 1
1956 #pragma unroll
1957 for (int i = 0; i < kValTileM; ++i) {
1958     int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
1959     int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
1960     int lane_smem_a_k = (lane_id / 16) * 8;
1961     uint32_t lane_smem_a_ptr =
1962         (smem_a_base_ptr +
1963          (smem_sel * s_a_stage_offset + lane_smem_a_m * BK +
1964           swizzle_A<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *
1965           sizeof(half));
1966     LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
1967                 RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
1968                 lane_smem_a_ptr);
1969 }
1970 #pragma unroll
1971 for (int j = 0; j < kValTileN; ++j) {
1972     int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
1973     int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
1974     int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
1975     uint32_t lane_smem_b_ptr =
1976         (smem_b_base_ptr +
1977          (smem_sel * s_b_stage_offset + lane_smem_b_n * BK +
1978           swizzle_B<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
1979           sizeof(half));
1980     LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
1981                 lane_smem_b_ptr);
1982 }
1983 }
1984 }
1985 }
1986
1987 // Epilogue: 复用 RA 寄存器做 warp shuffle → 128-bit collective store
1988 for (int i = 0; i < kValTileM; ++i) {
1989     #pragma unroll
1990     for (int j = 0; j < kValTileN; ++j) {
1991         RA[0][j][0] = RC[i][j][0];
1992         RA[1][j][0] = RC[i][j][1];
1993         RA[0][j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
1994         RA[0][j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
1995         RA[0][j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
1996         RA[1][j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
1997         RA[1][j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
1998         RA[1][j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
1999     }
2000     if (lane_id % 4 == 0) {
2001         int store_warp_smem_c_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
2002         int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
2003         #pragma unroll
2004         for (int j = 0; j < kValTileN; ++j) {
2005             int store_warp_smem_c_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
2006             int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
2007             int store_gmem_c_addr_0 =
2008                 store_lane_gmem_c_m * N + store_lane_gmem_c_n;
2009             int store_gmem_c_addr_1 =
2010                 (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
2011             *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
2012                 *reinterpret_cast<float4*>(&RA[0][j][0]);
2013             *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
2014                 *reinterpret_cast<float4*>(&RA[1][j][0]);
2015         }

```

```

2016     }
2017 }
2018 }
2019
2020 // =====
2021 // Phase 7c: HGEMM CuTe — CUTLASS CuTe DSL 实现 (SM80+, Tensor Core 全自动调度)
2022 // =====
2023 // 面试要点 (CuTe vs 手写 MMA PTX) :
2024 // - CuTe (CUTLASS Templates) 是 NVIDIA CUTLASS 3.x 的核心 DSL, 提供编译期
2025 //   Tensor 抽象, 自动推导 MMA、Copy、Swizzle 的线程-数据映射
2026 // - 与手写 MMA PTX (Phase 7b) 的对比:
2027 //   - 手写 MMA: 手动计算 smem 地址、手动 ldmatrix/mma PTX、手动 swizzle、手动 epilogue shuffle
2028 //   - CuTe: 声明 TiledMMA / TiledCopy / SmemLayout, 编译器自动推导线程-数据映射,
2029 //     cute::copy / cute::gemm 自动展开为高效指令序列
2030 // - 核心抽象 ("CuTe 五要素") :
2031 //   1. Tensor: 全局/共享/寄存器数据的逻辑视图 = (ptr, shape, stride)
2032 //   2. TiledMMA: 描述 MMA 指令 + warp/warpgroup 排布 + value tile 的 compile-time type
2033 //   3. TiledCopy: 描述数据搬运 (G→S, S→R, R→S, S→G) 的线程-元素映射
2034 //   4. Layout + Swizzle: 描述 smem 的数据排布和 bank conflict 消除
2035 //   5. Pipeline: cp.async + multistage, 由 cute::copy 自动管理
2036 //
2037 // CuTe kernel 的参数推导链 (launch wrapper 负责实例化, kernel 只接收实例化后的类型) :
2038 //   MMA Atom (SM80_16x8x16_F16F16F16F16_TN)
2039 //     → TiledMMA (EURepeat=MxNxK, ValTile=MxNxK)
2040 //     → TiledCopy (G2S/S2R/R2S/S2G, 每种有 ThrLayout + ValLayout)
2041 //     → SmemLayout (Atom + Swizzle + tile_to_shape + Stage)
2042 //
2043 // 调参口诀 (面试速记) :
2044 //   BM/BN/BK 越大 → smem 越大 → occupancy 越低, 但 K 循环更少 → 指令开销更低
2045 //   Swizzle<B,M,S> 的核心不是改变逻辑 Tensor 的 shape, 而是把一维 offset
2046 //   重新解释为一个二维逻辑空间, 再把二维坐标映射回 bank-conflict-free 的物理 offset:
2047 //     1. 连续的 2^M 个元素组成二维空间中的一个基本元素;
2048 //     2. 连续的 2^S 个基本元素组成一行;
2049 //     3. 二维空间包含 2^B 行;
2050 //     4. 对二维坐标做列置换: icol' = irow ^ icol;
2051 //     5. 保留基本元素内部的低 M 位, 再将置换后的二维坐标编码回 offset。
2052 //   因此 M 描述基本元素宽度, S 描述二维空间的列数, B 描述二维空间的行数。
2053 //   CuTe 的位级实现等价于:
2054 //     offset' = offset ^ ((offset & YYY) >> S),
2055 //     YYY = ((1 << B) - 1) << (M + S)。
2056 //   对 Swizzle<3,3,3>: 每个基本元素有 2^3=8 个值, 每行有 2^3=8 个基本元素,
2057 //   二维空间有 2^3=8 行; 元素地址位 [6:8] XOR 到 [3:5], 低 3 位保持不变。
2058 //   一个完整 swizzle 周期覆盖 2^(M+S+B)=2^9=512 个元素 (FP16 下为 1024B)。
2059 //   kStage: 2=最低延迟, 3/4=更好隐藏延迟 → 权衡 smem 占用
2060 //   EURepeat: 在 M/N/K 方向重复 MMA atom 的执行单元布局, 决定 TiledMMA 的线程排布
2061 //   与逻辑 tile 组织; MMA atom 越多, 需要的线程数就越多
2062 //
2063 // ★ 与手写 MMA Swizzle (Phase 7b-3) 的本质区别:
2064 //   - 手写 swizzle: 在 smem 地址计算时手动 XOR 列索引
2065 //   - CuTe Swizzle: SmemLayout 声明式指定地址位的 XOR pattern, cute::copy 自动应用
2066 //   - CuTe 优势: 类型安全, 编译器可在编译期推导映射; 布局变更通常只需修改类型定义
2067 //
2068 // ★ 本实现的 Tile 配置:
2069 //   - gmem/smem tile: BM=128, BN=256, BK=32, kStage=2; 这是 A/B tile 的目标 shape。
2070 //   - MMA atom: SM80_16x8x16_F16F16F16F16_TN; EURepeat=2x2x1, MMA_P_T=32x32x16。
2071 //   - TiledMMA 的逻辑 MMA tile 是 32x32x16, G2S/S2R copy 再把它映射到更大的 BMxBN tile;
2072 //     BN=256 不是由 "8x2x16" 直接推导出的 MMA tile, 而是本 wrapper 选择的 B tile 尺寸。
2073 //   - Smem Swizzle<3,3,3>: 512-element address period 的 XOR 重排, 针对 ldmatrix
2074 //     的访问模式降低 bank conflict; 512 是 swizzle 周期, 不等于 A atom 的逻辑元素数。
2075 //   - Threads: size(MMA{}) = 128 (4 warps × 2x2 EU repeat)。
2076 //   - Dynamic smem: Stage=2 时 A[128x32] + B[256x32] 共 49,152 bytes (约 48 KiB)。
2077 //
2078 // 参考: LeetCUDA/kernels/hgemm/cutlass/hgemm_mma_stage_tn_cute.cu

```

```

2079 // 参考: https://zhuanlan.zhihu.com/p/671419093 (CuTe Swizzle 详解)
2080 // =====
2081
2082 #if defined(NOTES_V2_ENABLE_CUTE)
2083
2084 #include <cute/tensor.hpp>
2085
2086 // =====
2087 // Phase 7c-1: CuTe HGEMM Kernel — TN 布局 + Smem Swizzle + Multistage Pipeline
2088 // =====
2089 // TN 布局:  $C[M \times N] = A[M \times K] \times B^T[N \times K]$ 
2090 //   -  $A[M][K]$  row-major,  $B^T[N][K]$  row-major (等价  $B$  col-major  $[K \times N]$ )
2091 //   - CuTe 中通过 make_tensor(make_gmem_ptr(ptr), shape, stride) 描述
2092 //
2093 // Pipeline 流程 (stage 是  $K$  tile 的循环缓冲槽, 不是一条完整计算链):
2094 //   1. PREFETCH: 预加载  $kStage-1$  个  $K$  tile ( $G \rightarrow S$  via cp.async), 填满 stage。
2095 //   2. 主循环 over  $K$  tiles:
2096 //       a. 从当前 stage 做  $S \rightarrow R$ : cute::copy(...) → ldmatrix;
2097 //       b. 对当前  $K$  slice 做 MMA: cute::gemm(...) → mma.sync;
2098 //       c. 在处理当前 tile 的首个 MMA slice 时, 异步提交下一个 tile 的  $G \rightarrow S$ ;
2099 //       d. 当前 tile 的最后一个  $K$  slice 完成后等待并切换 smem stage。
2100 //   3. Epilogue:  $R \rightarrow S$  (UniversalCopy 写入  $C$  scratchpad)  $\rightarrow S \rightarrow G$  (128-bit store)。
2101 //
2102 // 面试对比 — 手写 MMA vs CuTe 代码量:
2103 //   手写: ldmatrix PTX 地址计算 + swizzle XOR + mma PTX + epilogue shuffle (~200行)
2104 //   CuTe: partition + copy + gemm (~80行), 其余由 launch wrapper 的类型推导完成
2105 template <typename T,                                // element type (half)
2106         int BM, int BN, int BK,                      // block tile: BM=128, BN=256, BK=32
2107         int kStage,                                  // cp.async pipeline depth (2)
2108         typename TiledMMA,                           // MMA: SM80_16x8x16_F16F16F16F16_TN, EURepeat=2x2x1
2109         typename G2SCopyA,                           // G→S A: SM80_CP_ASYNC_CACHEGLOBAL<uint128_t>
2110         typename G2SCopyB,                           // G→S B: same as G2SCopyA
2111         typename SmemLayoutA,                        // smem A: Swizzle<3,3,3> + 8×BK atom → (BM,BK,kStage)
2112         typename SmemLayoutB,                        // smem B: same atom → (BN,BK,kStage)
2113         typename SmemLayoutC,                        // C scratchpad: Swizzle<3,3,3> + 32×32 atom, pipe=4
2114         typename S2RCopyAtomA,                       // S→R A: SM75_U32x4_LDSM_N (ldmatrix.x4)
2115         typename S2RCopyAtomB,                       // S→R B: same as S2RCopyAtomA
2116         typename R2SCopyAtomC,                       // R→S C: UniversalCopy<int> (32-bit store)
2117         typename S2GCopyAtomC,                       // S→G C: UniversalCopy<uint128_t> (128-bit wide store)
2118         typename S2GCopyC,                           // S→G TiledCopy: ThrLayout{32,4} × ValLayout{1,8}
2119         const bool BlockSwizzle = false> // enable 3D grid swizzle for L2 locality
2120 __global__ void hgemm_mma_stages_tn_cute(T *Aptr, T *Bptr, T *Dptr, int m,
2121         int n, int k) {
2122     using namespace cute;
2123
2124     // 动态 shared memory:  $KStage$  个  $A/B$  tile;  $C$  epilogue 复用当前  $A$  stage 的空间。
2125     extern __shared__ T shm_data[];
2126     T *Ashm = shm_data;
2127     T *Bshm = shm_data + cute::cosize(SmemLayoutA{});
2128
2129     int idx = threadIdx.x;
2130     // BlockSwizzle: 0/1 控制是否启用 thread block swizzle, 改善 L2 cache 局部性
2131     int ix = ((int)BlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
2132     int iy = blockIdx.y; //  $M$  方向 block 索引
2133     // 当前 kernel 只有 CTA 级边界判断, 没有 tile 内的逐元素 predication; 调用方需保证
2134     //  $M \% BM == 0, N \% BN == 0, K \% BK == 0$ , 才能避免边界 tile 的越界访问或未写回。
2135     if (iy * BM >= m || ix * BN >= n) return;
2136
2137     // CuTe Tensor 抽象: (ptr, shape, stride) 三元组描述数据布局
2138     // make_stride(leading_dim, Int<1>{}) → row-major: 同行相邻元素步长=1, 跨行步长=leading_dim
2139     Tensor A = make_tensor(make_gmem_ptr(Aptr), make_shape(m, k),
2140         make_stride(k, Int<1>{}));
2141     Tensor B = make_tensor(make_gmem_ptr(Bptr), make_shape(n, k),

```

```

2142         make_stride(k, Int<1>{}));
2143 Tensor D = make_tensor(make_gmem_ptr(Dptr), make_shape(m, n),
2144         make_stride(n, Int<1>{}));
2145
2146 // local_tile: 按 tiler 切出当前 CTA 的逻辑 tile; 返回 Tensor 仍保留未切出的 remainder mode。
2147 // make_coord(iy, _): 固定 M/N 方向的 CTA 坐标, _ 保留 K 方向的 tile remainder:
2148 // gA: (BM, BK, num_tile_k), gB: (BN, BK, num_tile_k), gD: (BM, BN)。
2149 Tensor gA = local_tile(A, make_tile(Int<BM>{}, Int<BK>{}), make_coord(iy, _));
2150 Tensor gB = local_tile(B, make_tile(Int<BN>{}, Int<BK>{}), make_coord(ix, _));
2151 Tensor gD = local_tile(D, make_tile(Int<BM>{}, Int<BN>{}), make_coord(iy, ix));
2152
2153 // Shared memory Tensor: 按 SmemLayout 解读 smem 区域
2154 auto sA = make_tensor(make_smem_ptr(Ashm), SmemLayoutA{}); // (BM, BK, kStage)
2155 auto sB = make_tensor(make_smem_ptr(Bshm), SmemLayoutB{}); // (BN, BK, kStage)
2156
2157 // TiledMMA partition: 将 MMA 的 A/B/C tile 映射到各线程的寄存器 fragment
2158 TiledMMA tiled_mma;
2159 auto thr_mma = tiled_mma.get_slice(threadIdx.x);
2160 auto tCrA = thr_mma.partition_fragment_A(gA(_, _, 0)); // (MMA, MMA_M, MMA_K)
2161 auto tCrB = thr_mma.partition_fragment_B(gB(_, _, 0)); // (MMA, MMA_N, MMA_K)
2162 auto tCrD = thr_mma.partition_fragment_C(gD); // (MMA, MMA_M, MMA_N)
2163 clear(tCrD); // 累加器清零
2164
2165 // G2S TiledCopy: 描述 global → shared memory 的数据搬运 (128-bit cp.async)。
2166 G2SCopyA g2s_tiled_copy_a;
2167 auto g2s_thr_copy_a = g2s_tiled_copy_a.get_slice(idcx);
2168 auto tAgA_copy = g2s_thr_copy_a.partition_S(gA); // (CPY, CPY_M, CPY_K, num_tile_k)
2169 auto tAsA_copy = g2s_thr_copy_a.partition_D(sA); // (CPY, CPY_M, CPY_K, kStage)
2170
2171 G2SCopyB g2s_tiled_copy_b;
2172 auto g2s_thr_copy_b = g2s_tiled_copy_b.get_slice(idcx);
2173 auto tBgB_copy = g2s_thr_copy_b.partition_S(gB); // (CPY, CPY_N, CPY_K, num_tile_k)
2174 auto tBsB_copy = g2s_thr_copy_b.partition_D(sB);
2175
2176 // S2R TiledCopy: 描述 shared → register 的数据搬运 (使用 ldmatrix)
2177 // make_tiled_copy_A/B: 根据 TiledMMA 自动推导 S2R copy 的线程-数据映射
2178 auto s2r_tiled_copy_a = make_tiled_copy_A(S2RCopyAtomA{}, tiled_mma);
2179 auto s2r_thr_copy_a = s2r_tiled_copy_a.get_slice(idcx);
2180 auto tAsA = s2r_thr_copy_a.partition_S(sA); // (CPY, CPY_M, CPY_K, kStage)
2181 auto tCrA_view = s2r_thr_copy_a.retile_D(tCrA); // (CPY, CPY_M, CPY_K) — 与 rA 寄存器布局对齐
2182
2183 auto s2r_tiled_copy_b = make_tiled_copy_B(S2RCopyAtomB{}, tiled_mma);
2184 auto s2r_thr_copy_b = s2r_tiled_copy_b.get_slice(idcx);
2185 auto tBsB = s2r_thr_copy_b.partition_S(sB);
2186 auto tCrB_view = s2r_thr_copy_b.retile_D(tCrB);
2187
2188 // PREFETCH: 预加载前 (kStage - 1) 个 K tile, 填满循环缓冲; 每次 copy 提交一个异步 G→S 操作。
2189 int itile_to_read = 0, ismem_read = 0, ismem_write = 0;
2190 #pragma unroll
2191 for (int istage = 0; istage < kStage - 1; ++istage) {
2192     cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, istage),
2193         tAsA_copy(_, _, _, istage));
2194     cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, istage),
2195         tBsB_copy(_, _, _, istage));
2196     cp_async_fence();
2197     ++itle_to_read;
2198     ++ismem_write;
2199 }
2200 cp_async_wait<kStage - 2>();
2201 __syncthreads();
2202
2203 // K 维第一层循环 — 按 BK 大小将 K 切分为 num_k_tiles 个 tile。
2204 // 外层 k_tile 遍历这些 BK-tile, 内层 k_step 遍历 tile 内的 MMA_K slice。

```

```

2205 // 当前实现使用整除, 要求  $K \% BK == 0$ ; 没有为尾部  $K$  tile 做 predication/padding。
2206 int num_k_tiles = k / BK;
2207 // 循环前预加载首轮数据: 将第一个  $K$  tile 的第 0 个  $K$  slice 从 smem 加载到寄存器。
2208 cute::copy(s2r_tiled_copy_a, tAsA(_, _, 0, ismem_read), tCrA_view(_, _, 0));
2209 cute::copy(s2r_tiled_copy_b, tBsB(_, _, 0, ismem_read), tCrB_view(_, _, 0));
2210
2211 #pragma unroll 1
2212 for (int k_tile = 0; k_tile < num_k_tiles; ++k_tile) {
2213     // num_k_steps: BK tile 内  $K$  方向的 MMA 迭代次数, 取自 fragment 的第三 mode ( $K$ -mode)。
2214     //
2215     // 为什么只显式展开  $K$ ? MMA_M 和 MMA_N 去哪了?
2216     //   tCrA=(MMA, MMA_M, MMA_K), tCrB=(MMA, MMA_N, MMA_K), tCrD=(MMA, MMA_M, MMA_N)
2217     //   -  $K$  方向: 每个 k_step 需要从 smem 加载**不同的** A/B 数据 (ldmatrix),
2218     //     所以  $K$  循环必须显式写, 每次迭代做  $S \rightarrow R$  copy + cute::gemm。
2219     //   - M/N 方向: 同一个 k_step 内, 所有 M、N 位置的 MMA atom 共享
2220     //     同一批寄存器数据。cute::gemm 根据 tiled_mma 的类型信息 (EURepeat=2x2)
2221     //     在编译期自动展开为覆盖全部 (MMA_M, MMA_N) 对的 mma.sync 指令序列,
2222     //     无需手动写 M/N 循环。——这等价于手写 MMA 中的:
2223     //       for (i=0; i<kValTileM; ++i)
2224     //         for (j=0; j<kValTileN; ++j)
2225     //           HMMA16816(RC[i][j][0], RC[i][j][1], ...);
2226     //
2227     // CUTLASS 推导链 (mma_atom.hpp) :
2228     //   make_tiled_mma 将 MMA_P_T::<2> (即 kMmaPK) 存入 PermutationMnK
2229     //   → TiledMMA::permutation_mnk<2>() 返回 kMmaPK
2230     //   → thrfrg_A 对 A(M,K) 按 (permutation_mnk<0>(), permutation_mnk<2>())
2231     //     做 logical_divide, 即按 (kMmaPM, kMmaPK) tile 化  $K$  维:
2232     //        $K$  被分为  $BK/kMmaPK$  个 perm- $K$  slice
2233     //       随后按 AtomShape_MnK<2> (MMA_K_atom) 做 zipped_divide
2234     //       每个 perm- $K$  含 kMmaEURepeatK 个 atom- $K$  slice
2235     //   → partition_A 选当前线程后,  $K$ -mode size =  $BK / kMmaPK = 32 / 16 = 2$ 
2236     // 本配置: kMmaPK=16 (MMA_K_atom=16 × kMmaEURepeatK=1), BK=32 → num_k_steps=2
2237     int num_k_steps = size<2>(tCrA);
2238
2239 #pragma unroll
2240     for (int k_step = 0; k_step < num_k_steps; ++k_step) {
2241         int k_step_next = (k_step + 1) % num_k_steps;
2242
2243         if (k_step == num_k_steps - 1) {
2244             // 等待下一个  $K$  tile 的 smem 数据就绪, 然后切换到下一个 stage。
2245             cp_async_wait<kStage - 2>();
2246             __syncthreads();
2247             ismem_read = (ismem_read + 1) % kStage;
2248         }
2249
2250         //  $S \rightarrow R$ : 加载下一组 A/B fragment 到寄存器。
2251         //
2252         // cute::copy 原生支持多 mode——理论上可以一次加载全部  $K$  slice:
2253         //   cute::copy(s2r_tiled_copy_a, tAsA(_, _, _, ismem_read), tCrA_view);
2254         // 但这里刻意逐 k_step_next 加载, 目的是做**寄存器双缓冲**：
2255         //   当前 k_step 做 MMA 计算时, ldmatrix 预加载 k_step_next 的数据,
2256         //   让  $S \rightarrow R$  延迟与 MMA 计算重叠。
2257         // 如果一次性加载全部  $K$  slice,  $S \rightarrow R$  和 MMA 就会彻底串行。
2258         cute::copy(s2r_tiled_copy_a, tAsA(_, _, k_step_next, ismem_read),
2259                   tCrA_view(_, _, k_step_next));
2260         cute::copy(s2r_tiled_copy_b, tBsB(_, _, k_step_next, ismem_read),
2261                   tCrB_view(_, _, k_step_next));
2262
2263         if (k_step == 0) {
2264             //  $G \rightarrow S$ : 提交下一个  $K$  tile (整个 BK) 的异步拷贝。
2265             if (itile_to_read < num_k_tiles) {
2266                 cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, itile_to_read),
2267                           tAsA_copy(_, _, _, ismem_write));

```

```

2268         cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, itile_to_read),
2269                   tBsB_copy(_, _, _, ismem_write));
2270         ++itile_to_read;
2271         ismem_write = (ismem_write + 1) % kStage;
2272     }
2273     cp_async_fence();
2274 }
2275
2276 // MMA: cute::gemm 内部根据 tiled_mma 的类型信息, 自动展开为覆盖
2277 // 全部 (MMA_M, MMA_N) 对的 mma.sync 指令序列, 累加到 tCrD。
2278 cute::gemm(tiled_mma, tCrD, tCrA(_, _, k_step), tCrB(_, _, k_step), tCrD);
2279 }
2280 }
2281
2282 // Epilogue: D 寄存器 → shared memory → global memory。
2283 // 使用当前 A stage 作为 C scratchpad, 避免为 epilogue 单独分配完整的 C shared memory。
2284 // 这里是“复用 storage”, 不是把 A Tensor 转换成 C Tensor:
2285 //   sA 的逻辑 shape 是 (BM, BK, kStage)=(128,32,2), 而 sA(_,_,ismem_read)
2286 //   只选出其中一个 A-stage 的 storage 起点; sC 随后以完全独立的 SmemLayoutC
2287 //   解释这同一段地址。当前 wrapper 的固定配置下:
2288 //       一个 A stage: 128 x 32 = 4096 个 half;
2289 //       C scratchpad: 32 x 32 x 4 = 4096 个 half。
2290 //   两者容量刚好相等, 但 logical shape 和 layout 不是同一个: A 的 layout atom
2291 //   是 8x32, C 的 layout atom 是 32x32, 二者都含 Swizzle<3,3,3>, 却不能把
2292 //   C 数据再按 SmemLayoutA 读取。launch wrapper 的 static_assert 用当前配置
2293 //   检查 C scratchpad 能放进一个 A pipe; 此处只别名该单个 stage, 不是整块双缓冲 sA。
2294 //
2295 // mainloop 已结束, 不会再通过 sA 的 A/B load view 消费这个 pipe 的旧 A 数据。
2296 // 从这一行开始, 对这块地址的所有访问都通过 SmemLayoutC 派生的 sC 完成: R2S
2297 // 用它写 C, S2G 用它读 C。因此 C 的 swizzle/address mapping 在写和读两侧一致。
2298 auto sC = make_tensor(sA(_, _, ismem_read).data(), SmemLayoutC{});
2299
2300 // R2S TiledCopy: 寄存器 → shared memory (使用 UniversalCopy 做类型转换)
2301 // R2SCopyAtomC = Copy_Atom<UniversalCopy<int>, T>; 以 32-bit 粒度搬运 T 数据
2302 auto r2s_tiled_copy_c = make_tiled_copy_C(R2SCopyAtomC{}, tiled_mma);
2303 auto r2s_thr_copy_c = r2s_tiled_copy_c.get_slice(idx);
2304 // tCrD 是 TiledMMA 产生的 accumulator fragment, 逻辑 shape 为
2305 // (MMA, MMA_M, MMA_N)。它的寄存器元素已经就位, 但该 layout 是为 MMA
2306 // accumulator 服务的, 并不直接按 R2S copy atom 的 source-value 顺序表达。
2307 //
2308 // retile_S 中的 S 指“这个 TiledCopy 的 source side”, 不是 shared memory。
2309 // 它基于 r2s_tiled_copy_c 的 reference layout 创建一个共享 tCrD.data() 的
2310 // zero-copy layout view, 将逻辑坐标表示为 (CPY, CPY_M, CPY_N)。这一步不读取
2311 // 或写入寄存器/共享内存, 不进行 dtype conversion, 也不交换 lane 间数据; 真正的
2312 // R2S 数据搬运发生在下面的 cute::copy(r2s_tiled_copy_c, ..., tCsC_r2s(...))。
2313 // 可将它与主循环的 s2r_thr_copy_a.retile_D(tCrA) 对照: 二者都是为了让已有
2314 // fragment 的 per-thread value ordering 匹配某个 TiledCopy 的 source/destination
2315 // 角色, 而不是另一次 memory transfer。
2316 // tCrD: (MMA, MMA_M, MMA_N) -> retile -> tCrC_r2s: (CPY, CPY_M, CPY_N)
2317 auto tCrC_r2s = r2s_thr_copy_c.retile_S(tCrD); // (CPY, CPY_M, CPY_N)
2318 // get_slice(idx) 已固定 CTA 内当前 logical copy thread; partition_D 再按
2319 // R2S TiledCopy 的 destination mapping 切分 sC。推导链:
2320 //   MMA_P_T = Tile<kMmaPM, kMmaPN, kMmaPK> = Tile<32, 32, 16>
2321 //   → TiledMMA::tile_size<0>(mma) = 32, tile_size<1>(mma) = 32
2322 //   → make_tiled_copy_C 的 tiler = (32, 32)
2323 //   → SmemLayoutC 的逻辑 M×N = (kMmaPM, kMmaPN) = (32, 32) 恰好等于该 tiler
2324 //   → partition_D: zipped_divide(sC, tiler=(32,32)) 后 M/N 维无 remainder
2325 //   → 结果 shape 的 M/N remainder = _1, _1
2326 // 最后一维 pipe=4 来自 SmemLayoutC 的第三维 kSmemLayoutCBatch, 而不是
2327 // “thread-level tensor 自动只剩 CPY”这一条规则。_1 表示该逻辑 mode 的 extent
2328 // 是 1, 并非该 mode 被删除或 C tile 没有 M/N 覆盖。
2329 auto tCsC_r2s = r2s_thr_copy_c.partition_D(sC); // (CPY, _1, _1, pipe)
2330

```

```

2331 // S2G TiledCopy: shared memory → global memory (128-bit store)
2332 // S2GCopyAtomC(Copy_Atom<UniversalCopy<cute::uint128_t>, T>) -> S2GCopyC
2333 S2GCopyC s2g_tiled_copy_c;
2334 auto s2g_thr_copy_c = s2g_tiled_copy_c.get_slice(idx);
2335 // tCsC_s2g 与 tCsC_r2s 都从同一个 sC 产生, 因而保留相同的 pipe=4; 前者是
2336 // S2G source mapping, 后者是 R2S destination mapping。tCgC_s2g 则面向完整
2337 // gD=(BM,BN)=(128,256), 相对于 S2G copy tiler 仍有非平凡的 M/N repetition,
2338 // 所以它保留 CPY_M、CPY_N。这里的 CPY/CPY_M/CPY_N 都是编译期 copy-partition
2339 // 的逻辑 mode, 不应直接理解为固定的 lane 数、warp 数或物理连续维度。
2340 auto tCsC_s2g = s2g_thr_copy_c.partition_S(sC); // (CPY, _1, _1, pipe)
2341 auto tCgC_s2g = s2g_thr_copy_c.partition_D(gD); // (CPY, CPY_M, CPY_N)
2342
2343 // group_modes<B,E> 将 layout 的半开区间 [B,E) 组合成一个嵌套 mode:
2344 //   group_modes<1,3>(Tensor(CPY, CPY_M, CPY_N))
2345 //   -> Tensor(CPY, (CPY_M, CPY_N))
2346 // 它只重组 Tensor 的 layout, 不分配内存、不移动 data(), 也不改变元素访问的物理地址。
2347 // 第二个 mode 仍然保存原来 CPY_M/CPY_N 的层次关系, 只是现在可以用一个坐标
2348 // i+j 访问这个组合 mode; 因此这里的 group_modes 更准确地说“合并 mode”,
2349 // 而不是把底层数据拷贝或无条件 flatten 成普通一维数组。
2350 // 从 type/shape 结构看, 结果确实是 Tensor(CPY, (CPY_M, CPY_N)); 但 grouped
2351 // mode 的 cardinality 是两个子 mode 的乘积, 因此 size<1>(…) 可作为一个标量
2352 // 范围遍历其全部 logical coordinate。于是 (_, i+j) 中的 i+j 是该 nested mode
2353 // 的线性 logical coordinate, 不是在 C++ 中对二维 tuple 做加法, 更不是 raw
2354 // pointer offset; 最终物理地址仍由各自 Tensor 的 grouped layout 计算。
2355 //
2356 // 这里为什么要对 tCrC_r2s 和 tCgC_s2g 同时 group?
2357 //   - tCrC_r2s: 每个线程持有的寄存器 C fragment, 原始形状为 (CPY, CPY_M, CPY_N);
2358 //   - tCgC_s2g: 该线程对应的 global-memory C 输出位置, 形状与源 Tensor 对齐;
2359 //   - 两者使用相同的 [1,3) 合并后, 源/目的 Tensor 仍保持相同的坐标空间,
2360 //     cute::copy 可以按相同的 (CPY, i+j) 坐标完成寄存器到 global 的对应搬运。
2361 //
2362 // tCsC_r2s/tCsC_s2g 没有 group 第 3 个 pipe mode, 因为它们还需要显式保留
2363 // shared-memory C scratchpad 的 pipeline 维度; step=size<3>(tCsC_r2s) 表示
2364 // 每一轮可以复用的 C shared-memory pipeline 深度。外层 i 在合并后的 C 元素空间中
2365 // 按 step 前进, 内层 j 选择当前 pipeline slot: 寄存器先写入 sC(_,0,0,j),
2366 // 再从同一个 slot 写回 global memory。
2367 // 当前 kSmemLayoutCBatch=4, 因此 step=4。代码未对 i+j 做尾部 predication,
2368 // 这个循环依赖当前静态 layout 中 grouped extent 能被 pipe depth 整除; 不应将
2369 // “每轮正好处理 4 个 fragment”误认为任意 tile/copy 配置都自动成立的通用规则。
2370 auto tCgC_s2gx = group_modes<1, 3>(tCgC_s2g);
2371 auto tCrC_r2sx = group_modes<1, 3>(tCrC_r2s);
2372
2373 int step = size<3>(tCsC_r2s); // C scratchpad 的 pipe 数 (由 kSmemLayoutCBatch=4 指定)
2374 // 双层循环的语义 (注意 step 与 CPY_N 无关):
2375 //   group_modes<1,3> 之后, tCrC_r2sx 的 shape 为 (CPY, (CPY_M, CPY_N))。
2376 //   size<1>(tCrC_r2sx) = CPY_M * CPY_N, 即该线程需处理的 C fragment 总数。
2377 //   外循环以 step=4 为步长, 内循环 j=0..3 每次处理 1 个 fragment, 将其写入
2378 //   第 j 号 C scratchpad pipe slot, 再读出写回 global。
2379 //
2380 // 这里 step=4 是 scratchpad pipeline 深度, 不是 CPY_N。CPY_N 的值取决于
2381 // TiledMMA 的 C-layout 如何将 (128,256) 的 gD tile 分配到 128 个线程上,
2382 // 通常 ≠ 4。循环之所以成立, 是因为 grouped mode 用线性坐标 i+j 遍历整个
2383 // CPY_M * CPY_N 的扁平空间——它不再区分 M 和 N 方向, 也不要求 CPY_N == step。
2384 // 唯一前提是 CPY_M * CPY_N 能被 step 整除 (当前静态 layout 编译期保证)。
2385 //
2386 // 第 j 号 pipe slot 在 sC 上的物理地址由 R2S partition_D / S2G partition_S
2387 // 各自推导; 因为二者都从同一个 sC (SmemLayoutC) 出发, pipe mode 保持一致,
2388 // 写入和读取命中同一段 shared memory。
2389 #pragma unroll
2390 for (int i = 0; i < size<1>(tCrC_r2sx); i += step) {
2391     // 每轮处理 step 个 fragment, 内层 j 选 pipe slot。
2392 #pragma unroll
2393     for (int j = 0; j < step; ++j) {

```

```

2394 // 这两行是 R2R staging: make_tensor_like<T> 分配当前线程私有、拥有 storage
2395 // 的 register Tensor; 普通 cute::copy 将 accumulator fragment materialize
2396 // 成输出元素类型 T 的临时 payload。
2397 //
2398 // cute::copy (无 copy-atom 的普通版) 内部按元素做
2399 // dst(i) = static_cast<T>(static_cast<SrcType>(src(i)))
2400 // 因此当 accumulator 与 T 类型不同时, 它自动完成 dtype 转换; 当二者相同
2401 // (如本 kernel 的 f16 accumulator + T=half), cast 退化为 no-op。
2402 //
2403 // 即使类型一致, t 还有一个不可省略的作用: layout 归一化。
2404 // tCrC_r2sx(_, i+j) 的 layout 是 retile_S → group_modes → slice 层层
2405 // 叠加的 composed layout, 而 make_tensor_like<T> + cute::copy 把它
2406 // 物化到一个拥有简单 strides 的 owning register Tensor 中。后续
2407 // r2s_tiled_copy_c 的 copy_unpack 需要按 copy-atom 的 val-layout
2408 // 拆分 source 元素, 简单 owning layout 比复杂的 composed layout 更容易
2409 // 被编译器推导内联。上游同源实现将其概括为"cope with accumulator and
2410 // output data type difference", 实际承担了类型适配和 layout 归一化两个职责。
2411 //
2412 // 这不是 retile_S 的一部分, 也不是 warp shuffle: 源/目的都在当前线程的
2413 // register 中, 代码没有显式 __shfl_sync。不能仅根据这段 C++ 断言最终 SASS
2414 // 绝不会含 shuffle; 但语义上的跨线程 regrouping 不由此 R2R copy 承担, 而是
2415 // 由随后的 R2S → shared memory → S2G 路径完成。若特定 accumulator/output
2416 // type 与 R2S copy-atom contract 已直接兼容, 可以设计直接 R2S 的变体; 本例
2417 // 保持同源实现的通用 staging 写法。
2418 auto t = make_tensor_like<T>(tCrC_r2sx(_, i + j));
2419 cute::copy(tCrC_r2sx(_, i + j), t);
2420 // R2S 的 copy atom 才在此处将 thread-local payload 写入 j 号 C scratchpad
2421 // slot; SmemLayoutC 定义写入后的跨线程地址重组。
2422 //
2423 // cute::copy(r2s_tiled_copy_c, src, dst) 的调用链路:
2424 // r2s_tiled_copy_c 是 TiledCopy, 但继承自 Copy_Atom<UniversalCopy<int>, T>
2425 // → 匹配 copy(Copy_Atom<...> const&, src, dst) 重载 [copy.hpp:~L190]
2426 // → src(t) 与 dst(tCsC_r2s(_, 0, 0, j)) 都是 rank-1 → 直接 copy_atom.call
2427 // → copy_atom.call 内部: 若 size(src)==NumValSrc (atom 编译期 val-count),
2428 // 走 copy_unpack; 否则递归剥 mode, 最终都落到 UniversalCopy<int>::copy
2429 // → 硬件层面就是逐 uint32_t 的 register-to-smem store [arch/copy.hpp:~L46]
2430 //
2431 // 这里没有任何运行时"查找表": t 的第 i 个元素之所以对应 dst 的第 i 个
2432 // shared memory offset, 是因为 pre-partition 阶段已经把映射烧进 layout 了:
2433 // - t 的 layout 来自 retile_S → group_modes → slice → make_tensor_like
2434 // → 元素顺序已按 R2S atom 的 source-side value ordering 排好
2435 // - tCsC_r2s 的 layout 来自 r2s_thr_copy_c.partition_D(sC)
2436 // → TiledCopy::tidfrg_D 用 LayoutCopy_TV + Tiler_MN + ValLayoutDst
2437 // 计算了当前线程每个 val 在 smem 中的物理 offset
2438 // 两者共享同一个 copy atom 的 ValLayoutSrc/ValLayoutDst 定义,
2439 // 因此逐元素 copy 天然把正确的数据写到了正确的地址。
2440 cute::copy(r2s_tiled_copy_c, t, tCsC_r2s(_, 0, 0, j));
2441 }
2442 // 这是 CTA 范围的 rendezvous: 所有线程完成当前批 R2S 写入后, S2G 才能从
2443 // sC 的重组布局读取完整数据。它承担了手写版中显式跨 lane 拼接所需的协作边界。
2444 __syncthreads();
2445
2446 // S→G: 从同一个 pipe slot 读回 global memory, 保持源/目的坐标一一对应。
2447 #pragma unroll
2448 for (int j = 0; j < step; ++j) {
2449 // S2GCopyC 的 atom 为 UniversalCopy<uint128_t>: 与 R2S 的 UniversalCopy<int>
2450 // (32-bit) 不同, 它一次搬运 128-bit (8 个 half), 映射为一条 wide store
2451 // (如 st.global.v4.f32 或 st.global.b128)。
2452 //
2453 // cute::copy(s2g_tiled_copy_c, src, dst) 的调用链路:
2454 // s2g_tiled_copy_c 是 TiledCopy, 继承自 Copy_Atom<UniversalCopy<uint128_t>, T>
2455 // → 匹配 copy(Copy_Atom<...> const&, src, dst) 重载 [copy.hpp:~L190]
2456 // → src=tCsC_s2g(_, 0, 0, j) 与 dst=tCgC_s2gx(_, i+j) 都是 rank-1

```

```

2457 // → 直接 copy_atom.call(src, dst)
2458 // → copy_atom.call 内部: size(src)==NumValSrc → copy_unpack
2459 // → 逐 128-bit chunk 调用 UniversalCopy<uint128_t>::copy
2460 // → 硬件层面就是 shared-memory-to-global wide store [arch/copy.hpp:~L46]
2461 //
2462 // S2G 的 pre-partition 与 R2S 对称, 但 source/destination 角色互换:
2463 // - tCsC_s2g 来自 s2g_thr_copy_c.partition_S(sC): TiledCopy::tidfrg_S
2464 // 用 LayoutCopy_TV + Tiler_MN + ValLayoutSrc 计算了当前线程每个 val
2465 // 在 sC (shared memory) 中的物理 offset。
2466 // S2GCopyC 的 tiler = product(ThrLayout{32,4}, ValLayout{1,8})
2467 // = (32×1, 4×8) = (32, 32) — 恰好与 R2S tiler 相同,
2468 // 因此 tCsC_s2g 也是 (CPY, _1, _1, pipe), _1,_1 来自 sC=(32,32,4) 无
2469 // M/N remainder。
2470 // - tCgC_s2gx(_, i+j) 来自 s2g_thr_copy_c.partition_D(gD) → group_modes
2471 // → slice。gD=(128,256) 相对于 tiler (32,32) 有 4×8 个 tile repetition,
2472 // 因此 tCgC_s2g 为 (CPY, CPY_M, CPY_N), group_modes<1,3> 合并 M/N
2473 // repetition 后得到 (CPY, (CPY_M,CPY_N)), 再用线性坐标 i+j 切片。
2474 // 两者共享同一个 copy atom 的 ValLayoutSrc/ValLayoutDst 定义,
2475 // 因此逐 chunk copy 天然从正确的 smem 地址读到正确的 gmem 地址。
2476 cute::copy(s2g_tiled_copy_c, tCsC_s2g(_, 0, 0, j), tCgC_s2gx(_, i + j));
2477 }
2478 // 所有线程都完成当前 pipe slots 的 S2G 读取后, 下一轮 R2S 才能覆盖这 4 个
2479 // scratchpad slots, 避免一部分线程仍在读取旧数据时被其他线程提前重写。
2480 __syncthreads();
2481 }
2482
2483 // 与 hgemm_mma_stages_tn 的手写 epilogue 对照: 手写版必须显式处理 MMA fragment
2484 // 的物理分布, 使用 RC0/RC1 暂存, 再用 __shfl_sync 从同一 warp 的相邻 lane 收齐
2485 // 8 个 half, 并由 lane_id % 4 == 0 的线程做 float4 (128-bit) global store。
2486 // CuTe 版没有把这些 lane 映射写死在 kernel 源码中: retile_S 表达 accumulator
2487 // fragment 与 R2S copy 的对应, R2S/sC/S2G 通过 shared-memory scratchpad 完成
2488 // CTA 范围的数据重组, S2GCopyC 以 uint128_t 表达 wide store。两者的共同目标都是
2489 // 将分散在计算线程寄存器中的 C fragment 变为适合连续 global-memory 写回的布局;
2490 // 区别是手写版直接编排 shuffle/store, CuTe 版将映射交给 TiledMMA/TiledCopy 类型推导。
2491 }
2492
2493 // =====
2494 // Phase 7c-2: CuTe HGEMM Launch Wrapper — 类型实例化 + Grid/Block 配置
2495 // =====
2496 // CuTe 的核心设计理念: "类型即配置" (Type-level configuration)
2497 // 所有的 MMA atom、TiledCopy、SmemLayout、Swizzle 都是 **编译期类型**,
2498 // kernel 接收这些类型的实例 (通常为 struct), 在编译期完成全部映射推导。
2499 //
2500 // 以下每个类型定义后面都标注了它在 kernel body 中的对应变量/用法, 形成 1:1 对照。
2501 //
2502 // 面试常问: 「CuTe 为什么比手写 PTX 简洁?」
2503 // 答: 手写需要:
2504 // 1) 手动计算每线程的 smem 地址偏移
2505 // 2) 手动计算 ldmatrix 的 lane 映射
2506 // 3) 手动插入 swizzle XOR 到每个地址计算点
2507 // 4) 手动做 epilogue 的 warp shuffle 编排
2508 // CuTe 的 partition + copy + gemm 通过 TiledCopy/TiledMMA 的类型信息
2509 // 在编译期自动完成上述全部推导, 生成与手写等价的 PTX 指令序列。
2510 // =====
2511 template <typename T, const int Stages = 2>
2512 void launch_hgemm_mma_stages_tn_cute(T *a, T *b, T *c, int M, int N, int K) {
2513     using namespace cute;
2514
2515     // — Tile 尺寸 (kernel 模板参数 BM/BN/BK/kStage 的值) —
2516     //
2517     // kernel 用法对照:
2518     // BM=128: local_tile(A, make_tile(Int<BM>{}, Int<BK>{}), ...) → gA=(BM,BK,num_k_tiles)
2519     //          local_tile(D, make_tile(Int<BM>{}, Int<BN>{}), ...) → gD=(BM,BN)

```

```

2520 // BN=256: local_tile(B, make_tile(Int<BN>{}, Int<BK>{}), ...) → gB=(BN,BK,num_k_tiles)
2521 // local_tile(D, ...) → gD 的列方向
2522 // BK=32: num_k_tiles = k / BK; 每个 BK tile 内 num_k_steps = BK/kMmaPK = 2 个 MMA_K slice
2523 // kStage=2: sA/sB 的第三维 = (BM,BK,kStage); cp_async_wait<kStage-2>() 流水线同步
2524 // kSmemLayoutCBatch=4: step = size<3>(tCsC_r2s) → C scratchpad pipe 深度 = 4
2525 auto BM = Int<128>{};
2526 auto BN = Int<256>{};
2527 auto BK = Int<32>{};
2528 auto KStage = Int<Stages>{};
2529 auto kSmemLayoutCBatch = Int<4>{};
2530
2531 // — SmemLayoutA / SmemLayoutB —
2532 // kernel 用法:
2533 // auto sA = make_tensor(make_smem_ptr(Ashm), SmemLayoutA{}); // (BM, BK, kStage)
2534 // auto sB = make_tensor(make_smem_ptr(Bshm), SmemLayoutB{}); // (BN, BK, kStage)
2535 // SmemLayout 由三部分组成: Swizzle<3,3,3> + atom layout(8×BK) + tile_to_shape 扩展到完整 tile+stage。
2536 //
2537 // 对当前 base layout shape=(8,32), stride=(32,1), T=half 的例子:
2538 // - M=3: 连续 2^3=8 个 half 组成一个基本元素, 即 16 bytes;
2539 // - S=3: 连续 2^3=8 个基本元素组成一行, 即 128 bytes;
2540 // - B=3: 共有 2^3=8 行。
2541 // 于是一个 8×32 的逻辑 tile 正好对应 8 行 × 128B, 覆盖全部 32 个 shared-memory bank。
2542 // Swizzle 对每个基本元素的列坐标执行 icol' = irow ^ icol,
2543 // 再将 (irow, icol') 和基本元素内部的 3 个低位编码回物理 offset。
2544 // 其位级形式为 offset' = offset ^ ((offset & (0b111 << 6)) >> 3);
2545 // 地址位 [6:8] XOR 到 [3:5], 地址位 [0:2] 不参与置换。
2546 // composition(Swizzle, base_layout) 的语义是 R(c) = Swizzle(base_layout(c)):
2547 // 逻辑坐标仍由 base_layout 给出, 只有最终的 shared-memory offset 被重排。
2548 // tile_to_shape: 通过 blocked product 重复这个带 swizzle 的 block layout,
2549 // 使结果 shape 匹配 BM×BK×kStage; 默认按目标 shape 的 mode order 重复各维。
2550 using SmemLayoutAtom = decltype(composition(
2551     Swizzle<3, 3, 3>{},
2552     make_layout(make_shape(Int<8>{}, Int<BK>{}),
2553         make_stride(Int<BK>{}, Int<1>{}))));
2554 using SmemLayoutA = decltype(tile_to_shape(
2555     SmemLayoutAtom{}, make_shape(Int<BM>{}, Int<BK>{}, Int<KStage>{})));
2556 using SmemLayoutB = decltype(tile_to_shape(
2557     SmemLayoutAtom{}, make_shape(Int<BN>{}, Int<BK>{}, Int<KStage>{})));
2558
2559 // — MMA (TiledMMA) —
2560 // kernel 用法:
2561 // TiledMMA tiled_mma;
2562 // auto thr_mma = tiled_mma.get_slice(threadIdx.x); // 每线程的 MMA slice
2563 // auto tCrA = thr_mma.partition_fragment_A(gA(_,_,0)); // (MMA, MMA_M, MMA_K)
2564 // auto tCrB = thr_mma.partition_fragment_B(gB(_,_,0)); // (MMA, MMA_N, MMA_K)
2565 // auto tCrD = thr_mma.partition_fragment_C(gD); // (MMA, MMA_M, MMA_N)
2566 // cute::gemm(tiled_mma, tCrD, tCrA(_,_,k_step), tCrB(_,_,k_step), tCrD);
2567 // MMA 还决定 S2R/R2S copy 的 tiler: make_tiled_copy_A/B/C 都依赖 tiled_mma 的
2568 // tile_size<0/1>(mma) = (32,32) 来推导线程→数据映射。
2569 //
2570 // 推导链: SM80_16x8x16_F16F16F16F16_TN → MMA_Atom
2571 // → make_tiled_mma(atom, EURepeat{2,2,1}, ValTile{32,32,16})
2572 // → TiledMMA: 128 threads = 4 warps × (2×2 EU slices), 逻辑 MMA tile = 32×32×16
2573 // TN = A row-major, B col-major; 因此传给本 kernel 的 B 指针实际指向
2574 // B^T[N,K] 的 row-major 存储 (等价于 GEMM 语义中的 B[K,N] col-major)。
2575 using mma_op = SM80_16x8x16_F16F16F16F16_TN;
2576 using mma_traits = MMA_Traits<mma_op>;
2577 using mma_atom = MMA_Atom<mma_traits>;
2578 using mma_atom_shape = mma_traits::Shape_MNK; // (Int<16>, Int<8>, Int<16>)
2579
2580 static constexpr int kMmaEURepeatM = 2;
2581 static constexpr int kMmaEURepeatN = 2;
2582 static constexpr int kMmaEURepeatK = 1;

```

```

2583 static constexpr int kMmaPM = 1 * kMmaEURepeatM * get<0>(mma_atom_shape{}); // 32
2584 static constexpr int kMmaPN = 2 * kMmaEURepeatN * get<1>(mma_atom_shape{}); // 32
2585 static constexpr int kMmaPK = 1 * kMmaEURepeatK * get<2>(mma_atom_shape{}); // 16
2586 // kMmaPK=16 → kernel 中 num_k_steps = size<2>(tCrA) = BK/kMmaPK = 32/16 = 2
2587
2588 using MMA_EU_RepeatT = decltype(make_layout(make_shape(
2589     Int<kMmaEURepeatM>{}, Int<kMmaEURepeatN>{}, Int<kMmaEURepeatK>{})));
2590 using MMA_P_T = Tile<Int<kMmaPM>, Int<kMmaPN>, Int<kMmaPK>>;
2591 using MMA = decltype(make_tiled_mma(mma_atom{}, MMA_EU_RepeatT{}, MMA_P_T{}));
2592
2593 // — G2SCopyA / G2SCopyB —
2594 // kernel 用法:
2595 // G2SCopyA g2s_tiled_copy_a; G2SCopyB g2s_tiled_copy_b;
2596 // cute::copy(g2s_tiled_copy_a, tAgA_copy(_,_,_,istage), tAsA_copy(_,_,_,istage));
2597 // cute::copy(g2s_tiled_copy_b, tBgB_copy(_,_,_,istage), tBsB_copy(_,_,_,istage));
2598 // G→S: 128-bit cp.async. ThrLayout{32,4} × ValLayout{1,8}:
2599 // 128 个线程, 每线程搬运 1×8=8 个 half = 128 bits.
2600 using g2s_copy_op = SM80_CP_ASYNC_CACHEGLOBAL<cute::uint128_t>;
2601 using g2s_copy_traits = Copy_Traits<g2s_copy_op>;
2602 using g2s_copy_atom = Copy_Atom<g2s_copy_traits, T>;
2603 using G2SCopyA = decltype(make_tiled_copy(
2604     g2s_copy_atom{},
2605     make_layout(make_shape(Int<32>{}, Int<4>{}),
2606         make_stride(Int<4>{}, Int<1>{}))),
2607     make_layout(make_shape(Int<1>{}, Int<8>{}))));
2608 using G2SCopyB = G2SCopyA;
2609
2610 // — S2RCopyAtomA / S2RCopyAtomB —
2611 // kernel 用法:
2612 // auto s2r_tiled_copy_a = make_tiled_copy_A(S2RCopyAtomA{}, tiled_mma);
2613 // auto s2r_tiled_copy_b = make_tiled_copy_B(S2RCopyAtomB{}, tiled_mma);
2614 // cute::copy(s2r_tiled_copy_a, tAsA(_,_,k_step_next,ismem_read), tCrA_view(_,_,k_step_next));
2615 // cute::copy(s2r_tiled_copy_b, tBsB(_,_,k_step_next,ismem_read), tCrB_view(_,_,k_step_next));
2616 // S→R: ldmatrix (SM75_U32x4_LDSM_N)。make_tiled_copy_A/B 根据 TiledMMA 自动推导
2617 // 与 MMA fragment 对齐的 shared→register 映射, 无需手动指定 ThrLayout/ValLayout.
2618 using s2r_copy_op = SM75_U32x4_LDSM_N;
2619 using s2r_copy_traits = Copy_Traits<s2r_copy_op>;
2620 using s2r_copy_atom = Copy_Atom<s2r_copy_traits, T>;
2621 using S2RCopyAtomA = s2r_copy_atom;
2622 using S2RCopyAtomB = s2r_copy_atom;
2623
2624 // — SmemLayoutC —
2625 // kernel 用法:
2626 // auto sC = make_tensor(sA(_,_,ismem_read).data(), SmemLayoutC{});
2627 // 复用当前 A stage 的空间作为 C scratchpad
2628 // sC shape = (kMmaPM, kMmaPN, kSmemLayoutCBatch) = (32, 32, 4)
2629 // pipe 数 4 由 kSmemLayoutCBatch 指定 → step = size<3>(tCsC_r2s) = 4
2630 // 与 A/B 相同的 Swizzle<3,3,3> 规则, 但 base layout 是 32×32 (MMA tile 大小),
2631 // 再扩展为 4 个 epilogue scratchpad pipe slots。这 4 个 slots 不等同于主循环
2632 // 的 kStage K-tile pipeline.
2633 using SmemLayoutAtomC = decltype(composition(
2634     Swizzle<3, 3, 3>{},
2635     make_layout(make_shape(Int<kMmaPM>{}, Int<kMmaPN>{}),
2636         make_stride(Int<kMmaPN>{}, Int<1>{}))));
2637 using SmemLayoutC = decltype(tile_to_shape(
2638     SmemLayoutAtomC{},
2639     make_shape(Int<kMmaPM>{}, Int<kMmaPN>{}, Int<kSmemLayoutCBatch>{})));
2640
2641 // — R2SCopyAtomC —
2642 // kernel 用法:
2643 // auto r2s_tiled_copy_c = make_tiled_copy_C(R2SCopyAtomC{}, tiled_mma);
2644 // cute::copy(r2s_tiled_copy_c, t, tCsC_r2s(_, 0, 0, j));
2645 // R→S: UniversalCopy<int> 以 32-bit 粒度将寄存器 payload 写入 C scratchpad.

```

```

2646 // make_tiled_copy_C 从 TiledMMA 的 tile_size<0/1>(mma)=(32,32) 推导 tiler,
2647 // 因此 R2S copy 的 tiler = (32,32), 与 SmemLayoutC 的 (32,32) 恰好匹配。
2648 using R2SCopyAtomC = Copy_Atom<UniversalCopy<int>, T>;
2649
2650 // — S2GCopyC —
2651 // kernel 用法:
2652 // S2GCopyC s2g_tiled_copy_c;
2653 // cute::copy(s2g_tiled_copy_c, tCsC_s2g(_, 0, 0, j), tCgC_s2gx(_, i+j));
2654 // S→G: UniversalCopy<uint128_t> 以 128-bit 粒度做 shared→global wide store。
2655 // ThrLayout{32,4} × ValLayout{1,8} → tiler = product((32,4),(1,8)) = (32,32),
2656 // 与 R2S tiler 相同, 因此 partition_S(sC) 得到的 pipe 维度一致。
2657 using S2GCopyAtomC = Copy_Atom<UniversalCopy<cute::uint128_t>, T>;
2658 using S2GCopyC = decltype(make_tiled_copy(
2659     S2GCopyAtomC{},
2660     make_layout(make_shape(Int<32>{}, Int<4>{}),
2661         make_stride(Int<4>{}, Int<1>{}))),
2662     make_layout(make_shape(Int<1>{}, Int<8>{}))));
2663
2664 // — Grid/Block 配置 —
2665 // kernel 用法:
2666 // int idx = threadIdx.x; // 0..127
2667 // int ix = ((int)BlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
2668 // int iy = blockIdx.y; // M 方向 CTA 坐标
2669 // if (iy * BM >= m || ix * BN >= n) return; // CTA 级边界保护
2670 // 由于 kernel 没有 tile 内的逐元素 predication, 调用方需保证
2671 // M % BM == 0, N % BN == 0, K % BK == 0。
2672 // 当不启用 BlockSwizzle 时, BZ=1 退化为纯 2D grid, 行为不变。
2673 constexpr bool kBlockSwizzle = false;
2674 constexpr int kSwizzleStride = 2048;
2675 int BX = (N + BN - 1) / BN;
2676 int BY = (M + BM - 1) / BM;
2677 int BZ = kBlockSwizzle ? (N + kSwizzleStride - 1) / kSwizzleStride : 1;
2678 BX = kBlockSwizzle ? (BX + BZ - 1) / BZ : BX;
2679 dim3 block(size(MMA{})); // 128 threads (= size(TiledMMA))
2680 dim3 grid(BX, BY, BZ);
2681
2682 // — Dynamic Shared Memory 大小 —
2683 // kernel 用法:
2684 // extern __shared__ T shm_data[];
2685 // T *Ashm = shm_data;
2686 // T *Bshm = shm_data + cute::cosize(SmemLayoutA{});
2687 // cosize(SmemLayoutA) = BM*BK*kStage = 128*32*2 = 8192 个 T
2688 // cosize(SmemLayoutB) = BN*BK*kStage = 256*32*2 = 16384 个 T
2689 // 合计 A+B = 24576 个 T; C epilogue 复用 A stage (128*32 = 4096 个 T)。
2690 // static_assert 保证 C scratchpad (kMmaPM*kMmaPN*kSmemLayoutCBatch = 32*32*4 = 4096)
2691 // 能放进一个 A stage (128*32 = 4096)。
2692 static constexpr int shm_size_AB =
2693     cute::cosize(SmemLayoutA{}) + cute::cosize(SmemLayoutB{});
2694 static constexpr int shm_size_C = cute::cosize(SmemLayoutC{});
2695 static_assert(size<0>(SmemLayoutA{}) * size<1>(SmemLayoutA{}) >= size(SmemLayoutC{}),
2696     "C shared memory must fit within one A pipe");
2697 static constexpr int kShmSize =
2698     cute::max(shm_size_AB, shm_size_C) * sizeof(T);
2699
2700 cudaFuncSetAttribute(
2701     hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
2702         SmemLayoutA, SmemLayoutB, SmemLayoutC,
2703         S2RCopyAtomA, S2RCopyAtomB, R2SCopyAtomC,
2704         S2GCopyAtomC, S2GCopyC, kBlockSwizzle>,
2705     cudaFuncAttributeMaxDynamicSharedMemorySize, kShmSize);
2706
2707 hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
2708     SmemLayoutA, SmemLayoutB, SmemLayoutC, S2RCopyAtomA,

```

```

2709         S2RCopyAtomB, R2SCopyAtomC, S2GCopyAtomC, S2GCopyC,
2710         kBlockSwizzle>
2711     <<<grid, block, kShmSize>>>(a, b, c, M, N, K);
2712 }
2713
2714 #endif /* NOTES_V2_ENABLE_CUTE */
2715
2716 // =====
2717 // Phase 7d: HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (Hopper)
2718 // =====
2719 // 面试要点 (WGMMMA vs MMA 对比) :
2720 //   - MMA: warp 级 (32 threads), 同步执行
2721 //   - WGMMMA: warpgroup 级 (128 threads = 4 warps), 异步执行 (fire-and-forget)
2722 //   - m64n128k16: M=64, N=128, K=16 → 一次处理 64×128×16=131K 个乘加 (MMA m16n8k16的 4×16=64倍)
2723 //   - TMA (Tensor Memory Accelerator): 硬件 DMA, 2D 寻址, 零寄存器开销
2724 //   - Warp Specialization: Producer 做 TMA, Consumer 做 WGMMMA, 通过 barrier 同步
2725 //   - 128B swizzle: shared memory 的 128B swizzle 模式, 避免 bank conflict
2726
2727 // ---- WGMMMA 辅助函数 ----
2728 // wgmma.commit_group / wait_group 语义 (PTX ISA §9.7.15.7) :
2729 //   - commit_group: 将此前所有未提交的 wgmma.mma_async 归入一个新的 wgmma-group
2730 //     (per-warpgroup, 故需 .sync.aligned 确保所有线程同步执行)。
2731 //   - wait_group N: 阻塞直到最多 N 个 wgmma-group 尚未完成 (pending ≤ N)。
2732 //     N=0 → 等所有 group 完成。★ 与 cp.async.wait_group 语义一致 (PTX ISA §9.7.9.25.3.3),
2733 //     都是 "wait until only N or fewer groups are pending"。
2734 #define WGMMMA_FENCE() asm volatile("wgmma.fence.sync.aligned;\n" ::: "memory")
2735 #define WGMMMA_COMMIT_GROUP() \
2736     asm volatile("wgmma.commit_group.sync.aligned;\n" ::: "memory")
2737 #define WGMMMA_WAIT_GROUP(n) \
2738     asm volatile("wgmma.wait_group.sync.aligned %0;\n" :: "n"(n) : "memory")
2739
2740 // SMEM_DESC_ENCODE: 将 byte 偏移编码为 WGMMMA descriptor 位域中的 14-bit 值。
2741 // 编码规则 (PTX ISA §9.7.15.5.1.2.2) :
2742 //   encoded = (x & 0x3FFFF) >> 4
2743 // 其中 0x3FFFF = 218 - 1 = 262143, 只保留低 18 bit。
2744 // >>4 是因为 shared memory 地址/偏移必须 16B 对齐 (24), 低 4 bit 恒为 0,
2745 // 可省略以扩大可表示范围。
2746 // 解码: decoded = encoded << 4 (回到原始 byte 值)。
2747 #define SMEM_DESC_ENCODE(x) (((uint64_t)(x)) & 0x3FFFF) >> 0x4
2748
2749 // make_smem_desc: 构造 WGMMMA (warpgroup MMA) 的 64-bit shared memory 矩阵描述符。
2750 // 硬件 WGMMMA (如 wgmma.mma_async.m64n128k16) 需要 descA/descB 两个 64-bit 描述符,
2751 // 来定位 shared memory 中的 A/B tile 并告知 swizzle 模式。
2752 //
2753 // 64-bit descriptor 位域布局 (参见 PTX ISA §9.7.15.5.1.2.2 & CUTLASS GmmaDescriptor) :
2754 // -----
2755 // | 位域          | 范围      | 大小 | 含义
2756 // |-----|-----|-----|-----|
2757 // | start_address  | [0, 14)   | 14   | smem 基址编码
2758 // | (unused)       | [14, 16)  | 2    | -
2759 // | leading_byte_offset | [16, 30) | 14   | 主要维度 (K) 的字节步长; swizzle 下硬件未使用 (assumed=1)
2760 // | (unused)       | [30, 32)  | 2    | -
2761 // | stride_byte_offset | [32, 46) | 14   | 跨越维度 (M/N) 的字节步长: K-Major 下为 8-row stripe 间距
2762 // | (unused)       | [46, 49)  | 3    | -
2763 // | base_offset    | [49, 52)  | 3    | swizzle pattern 偏移
2764 // | (unused)       | [52, 62)  | 10   | -
2765 // | layout_type    | [62, 64)  | 2    | swizzle 模式
2766 // -----
2767 // layout_type: 0=None, 1=128B swizzle, 2=64B swizzle, 3=32B swizzle
2768 //
2769 // K-Major + 128B swizzle + half(16-bit) 的几何推导 (PTX ISA §9.7.15.5.1.2) :
2770 //   - 128B swizzle atom 在 128-bit 单位下为 8×8
2771 //   - 归一化到 half(2B) 后: atom 覆盖 8×(128/16)=64 个 K 方向元素 × 8 个 M 方向元素

```

```

2772 // - 即每个 atom = 64(K) × 8(M) 个 half = 64×8×2 = 1024 bytes
2773 // - 这是 stride_byte_offset=1024 的来源
2774 //
2775 // - leading_byte_offset 在 K-Major swizzle 下 unused (hardware assumes 1) ,
2776 // 此处填 16 仅为占位, 编码后为 1。
2777 //
2778 // - base_offset=0 (bits [49,52] 未显式置位) , 表示 smem ptr 已 1024B 对齐。
2779 // 若未对齐需计算 (pattern_start_addr >> 7) & 0x7。
2780 //
2781 // 参考: CUTLASS GmmaDescriptor (cute/arch/mma_sm90_desc.hpp)
2782 // 注: descA/descB 共用此函数; 对 B 而言物理存储为 [BN, BK], 详见 WgmmaSMem 注释。
2783 __device__ inline uint64_t make_smem_desc(half *ptr) {
2784 // __cvta_generic_to_shared: 将通用地址空间中的指针转换为 shared memory 地址
2785 // (一个 32-bit 的 smem byte 偏移量, 相对于当前 CTA 的 shared memory 基址)。
2786 uint32_t addr = static_cast<uint32_t>(__cvta_generic_to_shared(ptr));
2787
2788 // 从全零开始: 所有位域初始化为 0。
2789 uint64_t desc = 0x0000000000000000;
2790
2791 // — bits [0, 14): start_address —
2792 // SMEM_DESC_ENCODE(addr) 将 addr 右移 4 位 (丢弃低 4 个 0 bit) ,
2793 // 只保留 14 位。硬件解码时左移 4 位恢复完整的 16B-aligned 地址。
2794 // 可寻址范围: 2^(14+4) = 2^18 = 256K 个 half = 512 KB 共享内存。
2795 desc |= SMEM_DESC_ENCODE(addr);
2796
2797 // — bits [16, 30): leading_byte_offset —
2798 // 原始值 16 (字节), 编码后为 (16 & 0x3FFFF) >> 4 = 1。
2799 // << 16 将编码值放到 bits [16, 30)。
2800 // K-Major + 128B swizzle 下该字段硬件未使用 (assumed to be 1, 参见 PTX ISA §9.7.15.5.1.2.1.1) ,
2801 // 此处填 16 仅为占位, 使编码值为 1 满足硬件预期。
2802 // 若为 MN-Major 或 INTERLEAVE 布局, LBO 有实际含义:
2803 // 对 INTERLEAVE: 同一 8×2 brick 内第一列到第二列的字节偏移。
2804 // 对 MN-Major swizzle: 偏移从首 (swizzle-byte-size/16) 行到下一组行。
2805 desc |= SMEM_DESC_ENCODE((uint64_t)16) << 16;
2806
2807 // — bits [32, 46): stride_byte_offset —
2808 // 原始值 1024 (字节), 编码后为 (1024 & 0x3FFFF) >> 4 = 64。
2809 // << 32 将编码值放到 bits [32, 46)。
2810 // K-Major 下定义为"从首 8 行到次 8 行的字节偏移" (PTX ISA §9.7.15.5.1.2.1.2) 。
2811 // 1024 的来源 (K-Major + 128B swizzle + half) :
2812 // 一个 128B swizzle atom 覆盖 64(K) × 8(M) 个 half。
2813 // 从 1 个 8-row stripe 到下一个 stripe 需要跨越 8 × 64 × 2 = 1024 bytes。
2814 // 若为 MN-Major: SBO 是从首列到下一列的偏移 (8 列一组) 。
2815 desc |= SMEM_DESC_ENCODE((uint64_t)1024) << 32;
2816
2817 // — bits [62, 64): layout_type (swizzle mode) —
2818 // 1llu << 62 = 二进制 01 在 bits [62, 64) = layout_type = 1。
2819 // 1 = 128B swizzle mode。
2820 // 其他取值: 0=None, 2=64B swizzle, 3=32B swizzle。
2821 // 128B swizzle 将 smem 中连续的行按 128B 粒度进行 XOR 重映射,
2822 // 确保同一 warp 内各线程访问不同 bank 的同一偏移时不会产生 bank conflict。
2823 desc |= 1llu << 62;
2824
2825 return desc;
2826 }
2827
2828 // ---- WGMMMA PTX 指令宏 ----
2829 // wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16
2830 // m64n128k16: M=64, N=128, K=16。一次 WGMMMA 计算 D[64×128] += A[64×16] × B[16×128]。
2831 // f16.f16.f16: A=f16, B=f16, D=f16 (累加器也是 f16 精度) 。
2832 // 每线程 32 个 uint32 输出: D=64×128=8192 half, warpgroup 128 线程分担,
2833 // 每线程 64 half = 32 uint32。
2834 // descA/descB: shared memory 描述符 (由 make_smem_desc 生成, 64-bit 编码) 。

```

```

2835 // ScaleD: 0=清零累加器再写入, 1=与累加器中的旧值累加 (K 维迭代必须用 1)。
2836 // ScaleA/ScaleB: 1=正常符号, -1=翻转符号 (此处始终用 1)。
2837 // TransA/TransB: 0=K-major (K 维连续), 1=MN-major (M/N 维连续)。本实现始终用 0。
2838 // 参考: PTX ISA §9.7.15.4 (wgmma.mma_async)
2839 #define WGMMMA_M64N128K16_F16F16F16(d, sA, sB, ScaleD, ScaleA, ScaleB, TransA, \
2840                                     TransB) \
2841 { \
2842     uint64_t desc_a = make_smem_desc(&(sA)[0]); \
2843     uint64_t desc_b = make_smem_desc(&(sB)[0]); \
2844     asm volatile( \
2845         "{\n" \
2846         "wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16 " \
2847         "{%0,  %1,  %2,  %3,  %4,  %5,  %6,  %7,  " \
2848         " %8,  %9,  %10, %11, %12, %13, %14, %15, " \
2849         " %16, %17, %18, %19, %20, %21, %22, %23, " \
2850         " %24, %25, %26, %27, %28, %29, %30, %31}, " \
2851         " %32, " \
2852         " %33, " \
2853         " %34, %35, %36, %37, %38;\n" \
2854         "}\n" \
2855         : "+r"((d)[0][0]), "+r"((d)[0][1]), "+r"((d)[0][2]), "+r"((d)[0][3]), \
2856         "+r"((d)[1][0]), "+r"((d)[1][1]), "+r"((d)[1][2]), "+r"((d)[1][3]), \
2857         "+r"((d)[2][0]), "+r"((d)[2][1]), "+r"((d)[2][2]), "+r"((d)[2][3]), \
2858         "+r"((d)[3][0]), "+r"((d)[3][1]), "+r"((d)[3][2]), "+r"((d)[3][3]), \
2859         "+r"((d)[4][0]), "+r"((d)[4][1]), "+r"((d)[4][2]), "+r"((d)[4][3]), \
2860         "+r"((d)[5][0]), "+r"((d)[5][1]), "+r"((d)[5][2]), "+r"((d)[5][3]), \
2861         "+r"((d)[6][0]), "+r"((d)[6][1]), "+r"((d)[6][2]), "+r"((d)[6][3]), \
2862         "+r"((d)[7][0]), "+r"((d)[7][1]), "+r"((d)[7][2]), "+r"((d)[7][3]) \
2863         : "l"(desc_a), "l"(desc_b), "n"(int32_t(ScaleD)), \
2864         "n"(int32_t(ScaleA)), "n"(int32_t(ScaleB)), "n"(int32_t(TransA)), \
2865         "n"(int32_t(TransB))); \
2866 }
2867
2868 // ---- TMA Shared Memory Layout ----
2869 // Multi-stage pipeline: kStages 个 stage, 每个 stage 存储 A[BM×BK] + B[BK×BN]
2870 //
2871 // 每个 stage 包含两块 smem:
2872 //   A tile: [BM, BK] row-major → BK×BM 个 half, 地址连续
2873 //   B tile: TMA 按 [BN, BK] 物理写入 (详见下方 ★), BN×BK 个 half
2874 //
2875 // ★ 关键区别 — WGMMMA 的 B 物理存储 vs 逻辑视图:
2876 //
2877 //   上层数据: 源矩阵 B^T [N, K] row-major (TN 布局, B 的列以行优先形式存储)。
2878 //   物理存储: TMA 将 B^T 的 tile 写入 smem, 物理布局为 [BN, BK] row-major
2879 //               (BN=128 行, BK=64 列, 每行 64 个连续的 half)。
2880 //   TMA 的 smem_box_shape = (BK=64, BN=128) 定义了每个 tile 的 shape,
2881 //   TMA 按行写入, 行内 BK 个 half 连续 → 物理上就是 [BN][BK]。
2882 //   逻辑视图: WGMMMA 指令 (wgmma.mma_async.m64n128k16, PTX ISA §9.7.15.4)
2883 //               通过 imm-trans-b=0 指定 B 为 K-major, 即逻辑上按 [BK, BN] 寻址。
2884 //               实际 [BN, BK] → 逻辑 [BK, BN] 的"转置"是通过 descB 中的 128B
2885 //               swizzle (layout_type=1) 在硬件地址重映射层完成的, 无需软件转置。
2886 //
2887 //   stride_byte_offset=1024 的来源 (K-major + 128B swizzle + f16, PTX ISA §9.7.15.5.1.2.1.2) :
2888 //       128B swizzle atom = 8(N) × 8(K) × 128-bit = 8 rows × 64 个 half。
2889 //       stride_byte_offset = 从当前 8-row stripe 到下一个 8-row stripe 的字节偏移
2890 //                           = 8 rows × (BK=64) halves × 2 bytes = 1024。
2891 //       这个值恰好等于物理 [BN, BK] 布局中连续 8 行的跨度 ← 进一步证实物理存储是 [BN, BK]。
2892 //
2893 //   与 MMA(TN) 的对比:
2894 //       MMA (TN): smem 存 B^T [N, K] row-major, ldmatrix 按列解出 col-major B fragment。
2895 //       WGMMMA:   smem 物理存 [BN, BK] (TMA 直写), WGMMMA 通过 swizzle 硬件以 K-major [BK, BN] 逻辑视图读取。
2896 //       简言之: swizzle 桥接了 "物理 row-major [BN, BK]" 和 "逻辑 K-major [BK, BN]" 的差异。
2897 template <int BM, int BN, int BK, int QSIZE> struct WgmmaSMem {

```

```

2898 alignas(128) half A[BM * BK * QSIZE]; // A tile: row-major [BM, BK]
2899 // B tile 注记:
2900 // - 物理存储: TMA 从 B^T[N,K] row-major 搬入, 物理上是 [BN, BK] row-major (BN 行, BK 列)
2901 // - 逻辑视图: WGMMMA 通过 descB 的 128B swizzle (PTX ISA §9.7.15.5.1.2) 将地址重映射,
2902 //   以 K-major [BK, BN] 逻辑视图供 wgmma.mma_async (imm-trans-b=0, PTX ISA §9.7.15.4) 读取
2903 // - B[BN * BK * QSIZE] 与 B[BK * BN * QSIZE] 数值等价 (BN×BK = BK×BN),
2904 //   但写 BN×BK 更能体现物理存储形态
2905 alignas(128) half B[BN * BK * QSIZE];
2906 };
2907
2908 // ---- WGMMMA Kernel: Warp Specialization + TMA ----
2909 // 面试重点 — Warp Specialization (Hopper 最核心的编程模型变化):
2910 //
2911 // 背景: 传统 GEMM kernel 中, 所有线程同步地"加载→计算→存回",
2912 // 数据搬运和计算无法重叠。Hopper 的 TMA + WGMMMA 支持将工作拆分为两个
2913 // warpgroup, 让数据搬运和矩阵乘完全异步执行:
2914 //
2915 // WG0 (128 threads, Producer): 仅 thread 0 提交 TMA 2D 拷贝指令,
2916 //   将 A/B tile 从 HBM 搬到 shared memory。
2917 // WG1 (128 threads, Consumer): 所有 128 个线程参与 WGMMMA 矩阵乘。
2918 //
2919 // Producer 和 Consumer 通过 cuda::barrier (CTA 级别) 同步:
2920 // - full[qidx]: Producer 发信号表示 stage qidx 的数据已就绪, 可被使用
2921 // - empty[qidx]: Consumer 发信号表示 stage qidx 的使用完毕, 可以被覆盖
2922 //
2923 // 本节重点理解:
2924 // 1) 为什么 Producer 只需要 thread 0? TMA 是硬件 DMA 指令, 一次提交
2925 //   即可搬运整个 2D tile, 无需所有线程参与。
2926 // 2) barrier 的 arrive count = 129: 128 个 Consumer 线程 + 1 个 Producer 提交线程。
2927 // 3) 多 stage pipeline 使 Consumer 计算 stage qidx 的同时,
2928 //   Producer 可以搬运 stage (qidx+1), 隐藏 HBM→SMEM 延迟。
2929 //
2930 // Tile Hierarchy (与 MMA m16n8k16 kernel 对比):
2931 // WGMMMA Atom: m64n128k16 (一次处理 64×128×16, 是 MMA 的 64 倍)
2932 // K Tile: BK=64, 每个 K tile 包含 BK/kWgmmaK=4 个 WGMMMA atom
2933 // M Tile: BM=128, 每个 M tile 包含 BM/kWgmmaM=2 个 WGMMMA atom
2934 // N Tile: BN=128, 单个 kWgmmaN=128 即可覆盖, 无需在 N 方向分块
2935 // Block Tile: C[128,128] = A[128,64] × B[64,128]
2936 // Threads: 256 = 2 warpgroups × 128 threads/warpgroup
2937 // Producer(WG0): 128 threads (仅 thread 0 做 TMA 提交)
2938 // Consumer(WG1): 128 threads = 4 warps (全部参与 WGMMMA 和写回)
2939 //
2940 // kStages=3: 3 级流水线。Consumer 滞后 Producer 最多 2 步, 确保 HBM→SMEM
2941 // 的延迟被计算完全掩盖。
2942 //
2943 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
2944 // - grid.z = S 个 swizzle 分区, 将连续 block 打散到不同 N 区域改善 L2 命中
2945 // Block: (256, 1, 1), 2 warpgroups
2946 // source: LeetCUDA/kernels/hgemm/wgmma/hgemm_wgmma_fp16acc_stages_tn.cu
2947 template <const int kWgmmaM = 64, // WGMMMA atom M dim (m64n128k16)
2948           const int kWgmmaN = 128, // WGMMMA atom N dim
2949           const int kWgmmaK = 16, // WGMMMA atom K dim
2950           const int BM = 128, // block tile M, 2 × kWgmmaM
2951           const int BN = 128, // block tile N, 1 × kWgmmaN
2952           const int BK = 64, // block tile K, 4 × kWgmmaK
2953           const int kNumThreads = 256, // 2 warpgroups × 128 threads
2954           const int kStages = 3, // TMA pipeline depth (full/empty barriers)
2955           const bool kBlockSwizzle = false> // enable 3D grid swizzle for L2 locality
2956 __global__ void __launch_bounds__(kNumThreads)
2957   hgemm_wgmma_stages_tn(
2958     int M, int N, int K, half *C,
2959     const CUtensorMap *__restrict__ tensorMapA,
2960     const CUtensorMap *__restrict__ tensorMapB) {

```

```

2961 // 注意: tensorMapA/tensorMapB 需要由 host 侧按当前 tile 布局预先创建;
2962 // 对 row-major [M, K] 矩阵, TMA shape 参数写的是 (K, M) 而不是 (M, K),
2963 // 也就是TMA descriptor中把连续的维度写在最内层, 非连续的维度写在最外层。
2964 // 这是 TMA descriptor 最容易背错的地方之一。notes 这里只保留 kernel 主体,
2965 // 不展开宿主侧 create_tensor_map 细节。
2966
2967 // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
2968 // bx = blockIdx.z * gridDim.x + blockIdx.x, 将相邻 block 打散到不同 N 区域
2969 const int bx = ((int)kBlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
2970 const int by = blockIdx.y;
2971 // num_consumers = (kNumThreads / 128) - 1 = 2 - 1 = 1 (1 个 consumer WG)
2972 // 这里的 -1 是因为 2 个 warpgroup 中 1 个是 producer, 剩余都是 consumer
2973 constexpr int num_consumers = (kNumThreads / 128) - 1; // 1 consumer WG
2974 // B_WG_M = BM / num_consumers: 每个 consumer warpgroup 负责的 M 行数
2975 // 当只有一个 consumer 时, 它负责全部 BM=128 行
2976 constexpr int B_WG_M = BM / num_consumers; // 128
2977
2978 // 边界检查: 确保当前 block 不超出 M/N 范围
2979 if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
2980     return;
2981
2982 // ---- Shared Memory 分配 ----
2983 // 动态 shared memory (由 host 侧通过 kernel launch 的 smem 参数指定大小)
2984 // __align__(128) 满足 TMA 和 WGMMMA 的 16B 对齐 + 128B swizzle 对齐要求
2985 extern __shared__ __align__(128) uint8_t smem_AB[];
2986 WgmmaSMem<BM, BN, BK, kStages> &s =
2987     *reinterpret_cast<WgmmaSMem<BM, BN, BK, kStages> *>(smem_AB);
2988 half *s_a = s.A;
2989 half *s_b = s.B;
2990
2991 // ---- cuda::barrier 初始化 ----
2992 //
2993 // ★ Barrier 机制速查 (arrive / wait 核心语义):
2994 //
2995 // cuda::barrier 是一个 CTA 级同步原语, 基于 **phase (奇偶交替) ** 工作:
2996 //
2997 //   init(bar, N): 设置 arrive_count = N。
2998 //       含义: 每 phase 需要 N 个线程各调用一次 arrive(), barrier 才翻转 phase。
2999 //
3000 //   arrive(): 线程声明"我已到达此 barrier 点"。
3001 //       - 非阻塞, 立即返回一个 token (包含当前 phase 值)。
3002 //       - 不关心"谁"到达, 只关心"到达次数"是否达到 arrive_count。
3003 //
3004 //   wait(token): 线程阻塞, 直到 barrier 的当前 phase ≠ token 中的 phase。
3005 //       - 即: 等待 phase 翻转。
3006 //       - Phase 翻转条件: (1) arrive 次数达到 arrive_count
3007 //                           (2) 若使用了 barrier_arrive_tx, 还需 TMA 字节全部写完
3008 //
3009 //   典型用法: b.wait(b.arrive())
3010 //       - arrive() 注册自己到达, 拿到 token (当前 phase = P)
3011 //       - wait(token) 阻塞直到 phase 翻转 (P → P+1)
3012 //       - 如果自己是第 N 个到达者 → phase 立即翻转 → wait 立即返回 (不阻塞)
3013 //       - 如果自己不是最后一个 → wait 阻塞, 直到第 N 个到达者触发翻转
3014 //
3015 // ★ 本 kernel 的 Pipeline 同步协议 (kStages=3, arrive_count=129):
3016 //
3017 //   Producer (1 thread)           Consumer (128 threads)
3018 //   ────────────                   ────────────
3019 //   C0: arrive(empty[*]) ×128 [init: 标记所有 stage 为空]
3020 //   ┌ for each k_tile: ┐           ┌ for each k_tile: ┐
3021 //   │ P1: arrive+wait(empty[q]) │   │ C1: arrive+wait(full[q]) │
3022 //   │   ↓ 等 stage q 被消费完   │   │   ↓ 等 TMA 数据就绪   │
3023 //   │ P2: TMA(A[q]) + TMA(B[q]) │   │ C2: WGMMMA 计算   │

```

```

3024 // |      ↓ 异步拷贝      | C3: wait WGMMA 完成
3025 // | P3: arrive_tx(full[q]) | C4: arrive(empty[q])
3026 // |      ↑ 通知: stage q 数据就绪      |      ↑ 通知: stage q 已消费完
3027 // |-----|             |-----|
3028 //
3029 // 关键不变式 (invariant) :
3030 //   - Producer 不会覆盖 Consumer 正在读的 stage
3031 //   - Consumer 不会读 Producer 还没写完的 stage
3032 //   - 通过 full/empty 两个 barrier 的 phase 交替来保证
3033 //
3034 // 每个 stage 有两个 barrier:
3035 //   full[qidx]: TMA 数据就绪信号。Producer 发 (arrive_tx), Consumer 等 (wait)。
3036 //   empty[qidx]: Stage 空闲信号。Consumer 发 (arrive), Producer 等 (wait)。
3037 //
3038 // arrive_count = 128 (consumer) + 1 (producer) = 129:
3039 // 每 phase, 128 个 consumer 线程 + 1 个 producer 线程都要 arrive 一次。
3040 #pragma nv_diag_suppress static_var_with_dynamic_init
3041 __shared__ cuda::barrier<cuda::thread_scope_block> full[kStages];
3042 __shared__ cuda::barrier<cuda::thread_scope_block> empty[kStages];
3043 #pragma nv_diag_default static_var_with_dynamic_init
3044
3045 // K 方向总 tile 数。要求 K 能被 BK 整除, 否则尾 tile 被丢弃。
3046 const int num_blocks_k = K / BK;
3047 const int wg_idx = threadIdx.x / 128; // 0=Producer, 1=Consumer
3048 const int tid = threadIdx.x % 128;    // 0~127 within warpgroup
3049
3050 // 初始化 barriers (仅 thread 0 执行)
3051 if (threadIdx.x == 0) {
3052     for (int i = 0; i < kStages; ++i) {
3053         // arrive_count = 129: 128 consumer + 1 producer
3054         // init 是 CUDA barrier 的 hidden friend 函数 (定义在 cuda::barrier 类体内),
3055         // 只能通过 ADL (Argument-Dependent Lookup) 调用。因为第一个参数 &full[i] 的类型是
3056         // cuda::barrier<cuda::thread_scope_block>*, 编译器通过 ADL 在 cuda 命名空间中自动找到它。
3057         init(&full[i], num_consumers * 128 + 1); // 128 consumer + 1 producer
3058         init(&empty[i], num_consumers * 128 + 1); // same
3059     }
3060     // fence_proxy_async_shared_cta: 确保 barrier 在 smem 中的初始化
3061     // 对 async proxy (TMA/WGMMA) 可见。
3062     cuda::device::experimental::fence_proxy_async_shared_cta();
3063 }
3064 __syncthreads();
3065
3066 // =====
3067 // ★ Warp Specialization 并发模型 — 为什么只有 if/else 没有 while?
3068 // =====
3069 //
3070 // 256 个线程在同一个 SM 上**并发执行**。if/else 只是分工, 不是先后顺序:
3071 //   - WG0 (threadIdx.x 0~127): 全部走 if 分支, 做 Producer。
3072 //   - WG1 (threadIdx.x 128~255): 全部走 else 分支, 做 Consumer。
3073 //
3074 // SM 的 warp scheduler 会交替调度来自 WG0 和 WG1 的 warp, 两者**同时**推进:
3075 //   Producer: for (k_iter = 0 .. num_blocks_k) { P1→P2→P3 } ← 自己的循环
3076 //   Consumer: for (k_iter = 0 .. num_blocks_k) { C1→C2→C3→C4 } ← 自己的循环
3077 //
3078 // 两个 warpgroups 各自独立迭代 K-tiles, 通过 full[] / empty[] barrier 同步:
3079 //   - Producer 领先 Consumer 太多 → wait(empty[q]) 阻塞, 等 Consumer 消费完
3080 //   - Consumer 领先 Producer 太多 → wait(full[q]) 阻塞, 等 TMA 搬运完
3081 //
3082 // 这是生产者-消费者模型的 GPU 实现: 不需要共享循环变量, barrier 的 phase
3083 // 交替天然保证了流水线串行化 (at most kStages-1 steps ahead)。
3084 // =====
3085 // Producer Warpgroup (WG0, threadIdx.x 0~127)
3086 // 职责: 提交 TMA 2D 拷贝, 将 A/B tile 从 HBM 异步搬运到 SMEM。

```

```

3087 // 只有 tid==0 执行实际拷贝提交, 其余 127 个线程空闲。
3088 // =====
3089 if (wg_idx == 0) {
3090     if (tid == 0) {
3091         // qidx: 当前操作的 stage 索引 (round-robin 0 -> 1 -> 2 -> 0 -> ...)
3092         int qidx = 0;
3093         for (int block_k_iter = 0; block_k_iter < num_blocks_k; ++block_k_iter, ++qidx) {
3094             if (qidx == kStages)
3095                 qidx = 0;
3096
3097             // Step P1: 等待 stage qidx 变为"空" (可被覆盖写入)
3098             //
3099             // 模式 empty[qidx].wait(empty[qidx].arrive()):
3100             //   - arrive(): Producer (1 个线程) 在 empty barrier 上注册到达,
3101             //     获得当前 phase 的 token。如果 Consumer 已经在 C4 步骤积累了
3102             //     128 次 arrive (来自上一轮迭代或 C0 初始化), 则加上这次共 129 次
3103             //     → phase 立即翻转。
3104             //   - wait(token): 阻塞直到 barrier phase ≠ token 中的 phase。
3105             //     由于 arrive() 是第 129 次到达, phase 在 arrive 时已翻转,
3106             //     wait 看到 phase 已变 → 立即返回 (首轮不阻塞)。
3107             //
3108             // 语义: Producer 说"我准备好写 stage qidx 了, Consumer 用完没有?"
3109             //     如果 Consumer 还没用完 → phase 未翻转 → wait 阻塞等待。
3110             //     如果 Consumer 已用完 → phase 已翻转 → wait 立即返回。
3111             empty[qidx].wait(empty[qidx].arrive());
3112
3113             // Step P2: 提交 TMA 2D 拷贝指令
3114             // cp_async_bulk_tensor_2d_global_to_shared:
3115             //   参数1(dst): smem 目标地址 (当前 stage 的 A/B tile 起始位置)
3116             //   参数2(tensorMap): Host 预创建的 TMA descriptor (描述源矩阵的 shape/stride/dtype)
3117             //   参数3/4(coords): (k_offset, m_offset) 或 (k_offset, n_offset) 的全局坐标
3118             //   参数5(barrier): 拷贝完成后自动 arrive 到此 barrier
3119             //
3120             // TMA 是硬件 DMA 引擎: 一次指令提交即可搬运整个 2D tile,
3121             // 无需线程逐元素搬运, 零寄存器开销。
3122             // TMA 2D 加载 A tile: coords = (k_offset, m_offset)
3123             cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
3124                 &s_a[qidx * BK * BM], tensorMapA, block_k_iter * BK, by * BM,
3125                 full[qidx]);
3126
3127             // TMA 2D 加载 B tile: coords = (k_offset, n_offset)
3128             cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
3129                 &s_b[qidx * BK * BN], tensorMapB, block_k_iter * BK, bx * BN,
3130                 full[qidx]);
3131
3132             // Step P3: 通知 Consumer: stage qidx 的 TMA 数据已就绪
3133             //
3134             // barrier_arrive_tx(bar, arrive_count_update, byte_count):
3135             //   - 在 bar 上注册 1 次到达 (arrive_count_update=1)
3136             //   - 同时声明预期有 byte_count 字节将通过 async copy (TMA) 写入 smem
3137             //   - Phase 翻转条件:
3138             //     (a) 总 arrive 次数达到 129 (128 consumer + 1 producer)
3139             //     (b) 所有声明的 async 字节已写入 smem
3140             //   两个条件都满足后 phase 才翻转, Consumer 的 wait() 才返回。
3141             //   - 即: Consumer 不会在 TMA 数据完整到达前就开始读 smem。
3142             [[maybe_unused]] auto token = cuda::device::barrier_arrive_tx(
3143                 full[qidx], 1, (BK * BN + BK * BM) * sizeof(half));
3144         }
3145     }
3146 }
3147 // =====
3148 // Consumer Warpgroup (WG1, threadIdx.x 128~255)
3149 // 职责: 等待 TMA 数据就绪 -> 发射 WGMMMA 做矩阵乘 -> 积累结果 -> 写回 C

```

```

3150 // 所有 128 个线程 (4 warps) 全部参与。
3151 // =====
3152 else {
3153     // Step C0: Consumer 初始化 — 标记所有 stage 为"空" (可被 Producer 写入)
3154     //
3155     // 所有 128 个 Consumer 线程对每个 stage 的 empty barrier 调用 arrive()。
3156     // 这是 Pipeline 的"预热"步骤——没有它, Producer 的 empty[qidx].wait()
3157     // 在第一轮会永远阻塞 (因为 Producer 的 1 次 arrive 不足以凑够 129)。
3158     //
3159     // 注意: 此时每个 empty[i] 只有 128 次 arrive, 未达 129, phase 不翻转。
3160     // Producer 后续的 empty[qidx].arrive() 作为第 129 次, 触发 phase 翻转。
3161     for (int i = 0; i < kStages; ++i) {
3162         [[maybe_unused]] auto token = empty[i].arrive();
3163     }
3164
3165     // 累加器寄存器声明
3166     // d[B_WG_M / kWgmmaM][kWgmmaN / 16][4]:
3167     //   - d[0][*][*]: M 方向第 1 个 WGMMMA atom (rows 0~63)
3168     //   - d[1][*][*]: M 方向第 2 个 WGMMMA atom (rows 64~127)
3169     //   - d[*][g][*]: N 方向第 g 组 16 列
3170     //   - d[*][*][0..3]: 4 条 uint32 寄存器, 共 8 个 half (覆盖 16×16 子块)
3171     // 每个线程总共 2 * 8 * 4 = 2 * 32 = 64 uint32 = 128 half (每次WGMMMA Atom 32 uint32 输出)
3172     // 128 线程 * 128 half = 16384 half = 128 * 128 = BM*BN (刚好覆盖整个 C tile)
3173     uint32_t d[B_WG_M / kWgmmaM][kWgmmaN / 16][4] = {};
3174
3175     int qidx = 0;
3176     // K 维外循环: 沿 K tile 迭代 (BK=64, 每个 K tile 做 4 次 WGMMMA 累加)
3177     for (int block_k_iter = 0; block_k_iter < num_blocks_k; ++block_k_iter, ++qidx) {
3178         if (qidx == kStages)
3179             qidx = 0;
3180
3181         // Step C1: 等待 TMA 数据就绪 (full 信号)
3182         //
3183         // 模式 full[qidx].wait(full[qidx].arrive()):
3184         //   - 128 个 Consumer 线程各调用 arrive(), 共 128 次到达。
3185         //   - 当前 phase 累积: 128/129, phase 尚未翻转。
3186         //   - wait(token) 阻塞, 直到 Producer 的 barrier_arrive_tx (Step P3)
3187         //     贡献第 129 次到达 + TMA 字节全部写完 → phase 翻转 → wait 返回。
3188         //
3189         // 语义: Consumer 说"我准备好读 stage qidx 了, 数据到了没有?"
3190         full[qidx].wait(full[qidx].arrive());
3191
3192         // Step C2: 发射 WGMMMA 指令序列
3193         //
3194         // WGMMMA 是异步指令 (fire-and-forget), 发射后立即返回, 不等待计算完成。
3195         // 标准流程: FENCE → 发射 WGMMMA → COMMIT → WAIT。
3196         //
3197         // WGMMMA_FENCE (wgmma.fence.sync.aligned) :
3198         //   - 确保 TMA 写入 smem 的数据对 async proxy (WGMMMA) 可见
3199         //   - 确保累加器寄存器 d[] 已准备好接收 WGMMMA 输出
3200         //   - 本质是 proxy 间的内存序 (memory ordering), 不是传统 __syncthreads
3201         //
3202         // 每个 WGMMMA_M64N128K16_F16F16F16 指令:
3203         //   D[64×128] += A[64×16] × B[16×128]
3204         //   A/B 均为 K-major (imm-trans=0), 由 descA/descB 描述 smem 中的布局。
3205         //   ScaleD=1: 累加。K 维有 BK/kWgmmaK=4 次迭代, 每次都用 ScaleD=1。
3206         WGMMMA_FENCE();
3207
3208         // M 维迭代: BM/kWgmmaM = 128/64 = 2 个 WGMMMA atom
3209         // 每个 atom 处理 64 行 M 方向数据。
3210 #pragma unroll
3211         for (int m_it = 0; m_it < B_WG_M / kWgmmaM; ++m_it) {
3212             // wgmma_sA 指向当前 stage 中 A tile 的第 m_it 个 64*BK 子块

```

```

3213 // s_a 布局: [kStages][BM][BK] -> qidx * BK*BM + BK * (m_it*64)
3214 half *wgmma_sA = s_a + qidx * BK * BM + BK * m_it * kWgmmaM;
3215
3216 // K 维迭代: BK/kWgmmaK = 64/16 = 4 次 WGMMMA (累加)
3217 // 每次处理 K=16 维的矩阵乘, 4 次累加后覆盖完整的 BK=64。
3218 #pragma unroll
3219 for (int k_it = 0; k_it < BK / kWgmmaK; ++k_it) {
3220     // 第 k_it 次 WGMMMA:
3221     //   A: wgmma_sA + k_it * kWgmmaK (A 的 K 维起始位置)
3222     //   B: s_b + qidx * BK * BN + k_it * kWgmmaK (B 的 K 维起始位置)
3223     //   注意 B 的 smem 布局是 [BK, BN] row-major, K-major 读取时从
3224     //   第 k_it*16 行开始取 16 行, 每行 BN=128 列。
3225     //   ScaleD=1: 累加 (K 维迭代需要累积结果)
3226     WGMMMA_M64N128K16_F16F16F16(d[m_it], wgmma_sA + k_it * kWgmmaK,
3227                                   s_b + qidx * BK * BN + k_it * kWgmmaK,
3228                                   1, // ScaleD=1: accumulate (不清零)
3229                                   1, 1, 0, 0);
3230 }
3231 }
3232
3233 // Step C3: 提交并等待 WGMMMA 完成
3234 //
3235 // WGMMMA_COMMIT_GROUP (wgmma.commit_group.sync.aligned):
3236 //   将自上一次 COMMIT 以来发射的所有 WGMMMA 归为一组, 提交到 async proxy 执行。
3237 //   类比 cp.async.commit_group, 但 scope 是 warpgroup 而非 per-thread。
3238 //
3239 // WGMMMA_WAIT_GROUP(0) (wgmma.wait_group.sync.aligned 0):
3240 //   阻塞直到最多 N 个 group 尚未完成 (pending ≤ N)。N=0 即等待所有 group。
3241 //   ★ 语义与 cp.async.wait_group 完全一致 (PTX ISA §9.7.15.7.3 vs §9.7.9.25.3.3) :
3242 //   两者都是 "wait until only N or fewer of the most recent groups are pending"。
3243 //   此处用 0 是因为 Consumer 在本次迭代中只 commit 了 1 个 group,
3244 //   必须等它全部完成才能进入下一步 (读下一 stage 的数据)。
3245 WGMMMA_COMMIT_GROUP();
3246 WGMMMA_WAIT_GROUP(0);
3247
3248 // Step C4: 释放 stage qidx — 通知 Producer 可以覆盖写入
3249 //
3250 // 128 个 Consumer 线程在 empty[qidx] 上 arrive(), 为 **下一 phase** 积累到达次数。
3251 // 这 128 次 arrive 不会立即翻转 phase (还需 Producer 在下一轮的 1 次 arrive)。
3252 //
3253 // 语义: Consumer 说 "stage qidx 我已经读完了, 你可以放心覆盖了。"
3254 [[maybe_unused]] auto token = empty[qidx].arrive();
3255 }
3256
3257 // =====
3258 // Epilogue: 将寄存器中的累加结果写回 global memory C
3259 // =====
3260 //
3261 // WGMMMA m64n128k16.f16.f16.f16 的输出寄存器映射:
3262 //
3263 // 对 warpgroup 内每个线程, 输出 64 个 half = 32 个 uint32。
3264 // 这些 half 分布在 d[2][8][4] 数组中:
3265 //   d[m_it][g][0]: (row, col) 和 (row, col+2) 位置的 2 个 half
3266 //   d[m_it][g][1]: (row+8, col) 和 (row+8, col+2) 位置的 2 个 half
3267 //   d[m_it][g][2]: (row, col+8) 和 (row, col+10) 位置的 2 个 half
3268 //   d[m_it][g][3]: (row+8, col+8) 和 (row+8, col+10) 位置的 2 个 half
3269 //
3270 // 其中:
3271 //   row = warp * 16 + lane / 4    (0~63, 4 warps * 16 rows)
3272 //   col = g * 16 + 2 * (lane % 4) (0~126, step 2, 8 g's * 16 cols)
3273 //
3274 // 每个线程覆盖一个 16x16 子块中的 16 个 half:
3275 //   +-----+-----+

```

```

3276 // | reg[0] | reg[2] | <- rows [row, row+8)
3277 // |(col,+2)|(col+8)|
3278 // +-----+-----+
3279 // | reg[1] | reg[3] | <- rows [row+8, row+16)
3280 // |(col,+2)|(col+8)|
3281 // +-----+-----+
3282 // 每个 reg 包含 2 个连续 half (col 和 col+2) , 用 uint32 存储。
3283 //
3284 // 三维遍历:
3285 // m_it=0: rows 0~63, m_it=1: rows 64~127
3286 // g=0..7: 每个覆盖 16 列, 8*16=128 列 ok
3287 // 每个线程写 4 uint32 * 2 half/uint32 * 8 g * 2 m_it = 128 half。
3288 // 128 线程 * 128 half = 16384 half = 128 * 128 ok
3289
3290 const int lane = tid % 32;
3291 const int warp = tid / 32;
3292 // row: 当前线程在当前 WGMMMA atom 中负责的起始行。
3293 // warp 0~3 各负责 16 行: rows [warp*16, warp*16+15]。
3294 // lane/4: 每 4 个连续 lane 负责同一行 (因为 WGMMMA fragment 中每行有 4 个 uint32,
3295 // 分别覆盖列对 {c0,c1}、{c2,c3}、{c8,c9}、{c10,c11}, 由 lane%4 区分) 。
3296 // 因此 lane/4 给出该行在 16-row block 内的行号 (0~7 对应 rows 0~7,
3297 // 同一 warp 中 lane/4==0 的有 lane 0,1,2,3, 都负责 row 0) 。
3298 const int row = warp * 16 + lane / 4;
3299 // block_C: 当前 C tile 在 global memory 中的起始地址
3300 // by*BM 是 M 方向偏移, bx*BN 是 N 方向偏移
3301 half *block_C = C + by * BM * N + bx * BN;
3302
3303 // 2 * (8 * 4) = 2 * 32 = 64 uint32 = 128 half
3304 #pragma unroll
3305 for (int m_it = 0; m_it < B_WG_M / kWgmmaM; ++m_it) {
3306     int yo = m_it * kWgmmaM; // M 方向行偏移 (0 或 64)
3307     #pragma unroll
3308     for (int g = 0; g < kWgmmaN / 16; ++g) {
3309         int col = g * 16 + 2 * (lane % 4); // N 方向列偏移 (0,2,4,...,14 then 16,18,...)
3310         // 一次 uint32 store 写入 2 个 half (连续列 col 和 col+2)
3311         // 左上象限: (row, col)
3312         *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col]) =
3313             d[m_it][g][0];
3314         // 左下象限: (row+8, col)
3315         *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col]) =
3316             d[m_it][g][1];
3317         // 右上象限: (row, col+8)
3318         *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col + 8]) =
3319             d[m_it][g][2];
3320         // 右下象限: (row+8, col+8)
3321         *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col + 8]) =
3322             d[m_it][g][3];
3323     }
3324 }
3325 }
3326 }
3327
3328 #if (defined(NOTES_V2_ENABLE_WGMMMA))
3329 // ---- Host-side TMA Tensor Map helpers (WGMMMA test) ----
3330 // 面试要点 (TMA descriptor 创建 — cuTensorMapEncodeTiled):
3331 // - TMA descriptor 描述 global memory 中矩阵的 shape/stride/dtype,
3332 // 以及硬件搬运的 box (tile) 大小和 smem swizzle 模式
3333 // - 关键易错点: TMA shape 参数写的是 (W,H) 而不是 (H,W)!
3334 // 对 row-major [M,K] 矩阵, TMA shape = (K,M), minor=K, major=M
3335 // - cuTensorMapEncodeTiled 是 CUDA Driver API, 需 #include <cuda.h> 并链接 -lcuda
3336 // - smem_box 维度顺序也是 (minor, major) = (BK, BM) 或 (BK, BN)
3337 // - Swizzle 模式通常选 CU_TENSOR_MAP_SWIZZLE_128B 配合 WGMMMA 的 128B swizzle
3338 //

```

```

3339 // * gmem_prob_stride + 1 的含义:
3340 // 语法层面 — 数组名退化 + 指针算术:
3341 //     gmem_prob_stride 类型为 uint64_t[5], 在表达式中退化为 uint64_t*
3342 //     (指向首元素 gmem_prob_stride[0])。+ 1 偏移 1 个 uint64_t, 指向
3343 //     gmem_prob_stride[1], 等价于 &gmem_prob_stride[1]。
3344 // 语义层面 — 跳过隐式的最内维 stride:
3345 //     cuTensorMapEncodeTiled 的 globalStrides 参数只接收 tensorRank-1 个
3346 //     stride 值。最内维 (fastest-changing dimension) 的 stride 是隐式的,
3347 //     固定等于 sizeof(element_type), 不需要显式传入。
3348 //     对于 tensorRank=2 的 FP16 矩阵:
3349 //         dim 0 (内维, K) : stride = sizeof(half)      ← 隐式, API 不需要
3350 //         dim 1 (外维, M) : stride = sizeof(half)*K    ← 需要传给 API
3351 //     gmem_prob_stride 数组的布局是内维在前:
3352 //         [0] = sizeof(half)                          ← 内维, 不传
3353 //         [1] = sizeof(half) * BlockMinorSize * blocks_width ← 外维, 传给 API
3354 //     所以 + 1 的作用是跳过 [0], 让 API 从 [1] 开始读取, 正好得到外维 stride。
3355 //
3356 // * smem_box_stride 为什么不需要 +1?
3357 //     与 globalStrides 不同, smemBoxStrides 参数接收完整的 tensorRank 个 stride,
3358 //     最内维 stride 也必须显式传入 (不隐式)。
3359 //     smem_box_stride[5] = {1, 1, 1, 1, 1} 表示 smem tile 中所有维度元素
3360 //     都是连续存放的, 维度间没有 padding/stride。
3361 //     对于 tensorRank=2: strides[0]=1 (内维 stride), strides[1]=1 (外维 stride),
3362 //     即 smem 中第 (i,j) 元素的偏移 = i*1 + j*1 = i+j (元素连续排列)。
3363 //     所以这里直接传 smem_box_stride (即 &smem_box_stride[0]), API 会读到全部
3364 //     两个 stride 值, 不需要跳过任何一个。
3365 //
3366 // 参考: CUDA Programming Guide §TMA, PTX ISA §9.7.15.5, CUTLASS GmmaDescriptor
3367
3368 template <int BlockMajorSize, int BlockMinorSize>
3369 __host__ static inline void create_tensor_map(CUtensorMap *tma_map,
3370                                               half *gmem_ptr,
3371                                               int blocks_height,
3372                                               int blocks_width) {
3373     void *gmem_address = (void *)gmem_ptr;
3374     uint64_t gmem_prob_shape[5] = {(uint64_t)BlockMinorSize * blocks_width,
3375                                     (uint64_t)BlockMajorSize * blocks_height,
3376                                     1, 1, 1};
3377     uint64_t gmem_prob_stride[5] = {
3378         sizeof(half), sizeof(half) * BlockMinorSize * blocks_width, 0, 0, 0};
3379     uint32_t smem_box_shape[5] = {uint32_t(BlockMinorSize),
3380                                   uint32_t(BlockMajorSize), 1, 1, 1};
3381     uint32_t smem_box_stride[5] = {1, 1, 1, 1, 1};
3382     CUresult result = cuTensorMapEncodeTiled(
3383         tma_map, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, 2, gmem_address,
3384         gmem_prob_shape, gmem_prob_stride + 1, smem_box_shape, smem_box_stride,
3385         CU_TENSOR_MAP_INTERLEAVE_NONE, CU_TENSOR_MAP_SWIZZLE_128B,
3386         CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE);
3387     if (result != CUDA_SUCCESS)
3388         printf("cuTensorMapEncodeTiled failed: %d\n", (int)result);
3389 }
3390
3391 __host__ static inline CUtensorMap *allocate_and_create_tensor_map(
3392     half *src, int blocks_height, int blocks_width) {
3393     CUtensorMap *tma_map_d;
3394     cudaMalloc(&tma_map_d, sizeof(CUtensorMap));
3395     CUtensorMap tma_map_host;
3396     create_tensor_map<128, 64>(&tma_map_host, src, blocks_height, blocks_width);
3397     cudaMemcpy(tma_map_d, &tma_map_host, sizeof(CUtensorMap),
3398               cudaMemcpyHostToDevice);
3399     return tma_map_d;
3400 }
3401 #endif /* NOTES_V2_ENABLE_WGMMMA */

```

```

3402 // =====
3403 // Phase 8: FlashAttention-2 (Split-Q + MMA m16n8k16)
3404 // =====
3405 // 面试要点 (FlashAttention 算法) :
3406 // 1. 核心问题: 标准 Attention 的  $O(N^2)$  中间矩阵 ( $S=QK^T$ ) 必须写入 HBM,
3407 //    但 HBM 带宽是瓶颈 → FlashAttention 用 tiling + online softmax 避免写回
3408 // 2. FA 三板斧:
3409 //    a) Tiling: Q 分块 [Br,d], K/V 沿 seqLen 分块 [Bc,d]
3410 //    b) Online Softmax: 迭代更新 m(行max) 和 l(行sum), 无需全局同步
3411 //    c) Recomputation(反向): 反向传播时重新计算 S/P, 而非存储中间矩阵
3412 // 3. Split-Q 设计: 所有 warp 共享同一块 K, 各 warp 处理 Q 的不同行片段
3413 //    - warp_KV=0 (所有 warp 共享 K), warp_QP=warp_id (各 warp 不同 Q 行)
3414 //    - 优点: 减少 warp 间通信和 shuffle
3415 // 4. Online rescaling 公式 (FA 核心, arXiv:2307.08691) :
3416 //    for each K,V tile:
3417 //        S_cur = Q @ K^T // 未缩放, 存入 R_S
3418 //        m_new = max(m_old, row_max(S_cur * scale))
3419 //        P_cur = exp(S_cur * scale - m_new) // ← 写回 R_S 寄存器!
3420 //        l_new = exp(m_old - m_new) * l_old + row_sum(P_cur)
3421 //        O_new = diag(exp(m_old - m_new)) * O_old + P_cur @ V
3422 //        O_final = O_new / l_final
3423 // 5. 为什么 R_S 可以直接用作 P@V 的 A 矩阵?
3424 //    - R_S 经过 softmax 后, 存储的是  $P = \exp(S - m)$ , 数据仍然是 half 精度
3425 //    - 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局, 使 softmax
3426 //      后的 P 可以继续留在 R_S 中供后面的 P@V 直接消费
3427 //    - 这是此实现的寄存器布局复用技巧, 不要背成“所有 MMA A/C fragment
3428 //      都天然同构”的通用结论
3429 //
3430 // 本实现参考: FlashAttention-2 (Dao et al., arXiv:2307.08691)
3431 // 从 LeetCUDA flash-attn/mma/basic/flash_attn_mma_split_q.cu 提取
3432 // Grid: ((QKV_seqLen + 63) / 64, QKV_batch * QKV_head, 1), Br=64
3433 // Block: (128, 1, 1), kNumThreads=WARP_SIZE*kMmaTileSeqLenQ*kMmaTileSeqLenK=128
3434 // source: LeetCUDA/kernels/flash-attn/mma/basic/flash_attn_mma_split_q.cu
3435 //
3436 // ---- 寄存器填充辅助函数 ----
3437 template <typename T, int M, const int N, const int K = 2>
3438 __device__ inline void fill_3D_regs(T (&R)[M][N][K], T val) {
3439     #pragma unroll
3440     for (int i = 0; i < M; ++i)
3441         #pragma unroll
3442         for (int j = 0; j < N; ++j)
3443             #pragma unroll
3444             for (int k = 0; k < K; ++k)
3445                 R[i][j][k] = val;
3446 }
3447
3448 template <typename T, int M, const int N = 2>
3449 __device__ inline void fill_2D_regs(T (&R)[M][N], T val) {
3450     #pragma unroll
3451     for (int i = 0; i < M; ++i)
3452         #pragma unroll
3453         for (int j = 0; j < N; ++j)
3454             R[i][j] = val;
3455 }
3456
3457 // =====
3458 // FlashAttention-2 Split-Q Kernel (完整实现)
3459 // =====
3460 // Q,K,V,O: [batch_size, num_heads, seq_len, head_dim], [B,H,N,d]
3461 //
3462 // Tile 设计 (以 kHeadDim=64 为例) :
3463 // Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ = 16*4*1 = 64

```

```

3465 // Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK = 8*1*8 = 64
3466 // Warp 布局: 4 warps, warp_QP 0~3 各处理 16 行, warp_KV=0 共享 K
3467 //
3468 // 执行流程:
3469 // 1) 预加载 Q[Br,d] 到 smem (只加载一次, split-Q 的核心优势)
3470 // 2) 外循环: 沿 K seqLen 分块迭代 (Tc = seqLen/Bc)
3471 // 3) 3a: cp.async 加载当前 V + 预加载下一个 K (多 stage pipeline)
3472 // 4) 3b: Q@K^T — 沿 head dim 内循环 → ldmatrix Q/K → HMMMA16816
3473 // 5) 3c: Online Safe Softmax — warp reduce max/row → exp(S*scale - max)
3474 // → 关键: 将 P 写回 R_S 寄存器 (替换 S), R_S 现在存储 P matrix
3475 // 6) 3d: P@V — 沿 V_Bc 内循环 → ldmatrix V (transposed) → 直接用 R_S 做 A
3476 // → 当前实现依赖同一组寄存器布局约定, 避免为 P@V 额外重组 P fragment
3477 // 7) 3e: Online rescaling — O_new = exp(m_old-m_new)*O_old + P@V
3478 // 8) 最终 rescale: O_final = (1/l_final) * O_final
3479 // 9) Epilogue: warp shuffle + 128-bit collective store
3480
3481 template <
3482     const int kHeadDim,          // head dim: 32, 64, 128
3483     const int kMmaAtomM,         // 16 (MMA instruction M dimension)
3484     const int kMmaAtomN,         // 8 (MMA instruction N dimension)
3485     const int kMmaAtomK,         // 16 (MMA instruction K dimension)
3486     const int kMmaTileSeqLenQ,   // MMA tiles along Q's M dim, 4 → Br=16*4=64
3487     const int kMmaTileSeqLenK,   // MMA tiles along K's N dim, 1 → Bc basis=8
3488     const int kMmaTileSeqLenP,   // MMA tiles for P@V M dim, must equal kMmaTileSeqLenQ
3489     const int kMmaTileHeadDimV,  // MMA tiles for P@V N dim (head dim direction)
3490     const int kValTileSeqLenQ,   // value tiles along Q's M, 1 → Br per warp=16
3491     const int kValTileSeqLenK,   // value tiles along K's N, 8 → Bc_warp=8*8=64
3492     const int kValTileSeqLenP,   // value tiles for P@V M dim, 1
3493     const int kValTileHeadDimV,  // value tiles for P@V N dim, kHeadDim/(8*kMmaTileHeadDimV)
3494     const int kStage,            // pipeline stages for K: 1 or 2
3495     const int kPad>              // padding for bank conflict avoidance
3496 __global__ void __launch_bounds__(WARP_SIZE * kMmaTileSeqLenQ * kMmaTileSeqLenK)
3497     flash_attn_mma_stages_split_q(half *Q, half *K, half *V, half *O,
3498                                     int QKV_seqLen, int QKV_head) {
3499     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 64
3500     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 64
3501     constexpr int kNumThreads = WARP_SIZE * kMmaTileSeqLenQ * kMmaTileSeqLenK; // 128
3502     const int Tc = (QKV_seqLen + Bc - 1) / Bc;
3503     // 原始实现默认 seqLen 与 Bc 对齐; 最后一个不完整 tile 需要额外 pad/边界处理。
3504     // 这里保留 ceil 写法是为了说明 tile 划分方式, 不等于当前实现已经完整处理了尾 tile。
3505     const float scale = 1.0f / sqrtf((float)kHeadDim);
3506
3507     // Block indexing
3508     const int QKV_batch_id = blockIdx.y / QKV_head;
3509     const int QKV_head_id = blockIdx.y % QKV_head;
3510     const int Q_tile_id = blockIdx.x;
3511     const int tid = threadIdx.x;
3512     const int warp_id = tid / WARP_SIZE;
3513     const int lane_id = tid % WARP_SIZE;
3514     const int warp_QP = warp_id; // Split-Q: 每个 warp 处理不同的 Q 行片段
3515     const int warp_KV = 0; // 所有 warp 共享 K (减少跨 warp 通信)
3516
3517     // Global memory base offsets for this (batch, head)
3518     // 这里默认 Q/K/V 共享同一 per-head 基址布局, 对应 self-attention 场景
3519     const int Q_gmem_offset =
3520         (QKV_batch_id * QKV_head * QKV_seqLen + QKV_head_id * QKV_seqLen) *
3521         kHeadDim;
3522     const int K_gmem_offset = Q_gmem_offset;
3523     const int V_gmem_offset = Q_gmem_offset;
3524     const int O_gmem_offset = Q_gmem_offset;
3525
3526     // Thread-to-smem mapping for cooperative load
3527     int load_smem_Q_Br = tid / (kNumThreads / Br);

```

```

3528 int load_smem_Q_d =
3529     (tid % (kNumThreads / Br)) * (kHeadDim / (kNumThreads / Br));
3530 int load_smem_K_Bc = tid / (kNumThreads / Bc);
3531 int load_smem_K_d =
3532     (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
3533 int load_smem_V_Bc = tid / (kNumThreads / Bc);
3534 int load_smem_V_d =
3535     (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
3536
3537 int load_gmem_Q_Br = Q_tile_id * Br + load_smem_Q_Br;
3538 if (load_gmem_Q_Br >= QKV_seqlen)
3539     return;
3540
3541 // ---- Shared memory layout ----
3542 extern __shared__ half smem[];
3543 constexpr int Q_tile_size = Br * (kHeadDim + kPad); // Q tile: [Br, d+kPad]
3544 constexpr int KV_tile_size = Bc * (kHeadDim + kPad); // K/V tile: [Bc, d+kPad]
3545 half *Q_tile_smem = smem;
3546 half *K_tile_smem = Q_tile_smem + Q_tile_size;
3547 half *V_tile_smem = K_tile_smem + kStage * KV_tile_size; // kStage copies of K
3548 // 原始 kernel 还留了一个优化点: 若 kStage=1, K 和 V 在时序上并不重叠,
3549 // 理论上可以复用同一块 KV shared memory 来进一步压缩 smem 占用。
3550
3551 uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_tile_smem);
3552 uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_tile_smem);
3553 uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_tile_smem);
3554
3555 // ---- Online Softmax persistent state ----
3556 // lane_block_row_max_old[i][r]: running max for row r of warp tile i
3557 // lane_block_row_sum_old[i][r]: running denominator l for row r of warp tile i
3558 float lane_block_row_max_old[kValTileSeqLenQ][2];
3559 float lane_block_row_sum_old[kValTileSeqLenQ][2];
3560 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_max_old, -INFINITY);
3561 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);
3562
3563 // ---- Register allocation ----
3564 uint32_t R_Q[kValTileSeqLenQ][4]; // Q regs
3565 uint32_t R_K[kValTileSeqLenK][2]; // K regs
3566 uint32_t R_V[kValTileHeadDimV][2]; // V regs
3567 // R_S / R_O / R_D 都按 mma.sync.aligned.m16n8k16 的 fragment 约定存储。
3568 // 对单个 m16n8k16 tile 而言:
3569 //   - reg[0] 持有该 tile 前 8 行里的两个 half 值
3570 //   - reg[1] 持有该 tile 后 8 行里的两个 half 值
3571 // 后续 softmax、P@V、online rescale 都直接围绕这组 fragment 布局做寄存器内变换。
3572 uint32_t R_S[kValTileSeqLenQ][kValTileSeqLenK][2]; // S=Q@K^T / P=softmax(S)
3573 uint32_t R_O[kValTileSeqLenP][kValTileHeadDimV][2]; // O for current tile
3574 uint32_t R_D[kValTileSeqLenP][kValTileHeadDimV]
3575     [2]; // O accumulator (final output)
3576
3577 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
3578 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_D, 0);
3579
3580 // =====
3581 // Step 1: 加载 Q[Br, d] 到 shared memory (整个外循环只加载一次)
3582 // =====
3583 {
3584     int load_gmem_Q_addr =
3585         Q_gmem_offset + load_gmem_Q_Br * kHeadDim + load_smem_Q_d;
3586     uint32_t load_smem_Q_ptr =
3587         smem_Q_base_ptr +
3588         (load_smem_Q_Br * (kHeadDim + kPad) + load_smem_Q_d) * sizeof(half);
3589 #pragma unroll
3590     for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {

```

```

3591     CP_ASYNC.CG(load_smem_Q_ptr + i * 2, &Q[load_gmem_Q_addr + i], 16);
3592 }
3593 CP_ASYNC_COMMIT_GROUP();
3594 }
3595
3596 // =====
3597 // Step 2: 预加载前 (kStage-1) 个 K tile (多 stage pipeline 预热)
3598 // 注意: Q 由 blockIdx.x 固定到当前 Q tile; 而 K/V 的 seqlen 遍历始终从 tile 0 开始,
3599 // 后续在外循环里不断递增至 tile 1/2/3/.../Tc-1.
3600 // =====
3601 if constexpr (kStage > 1) {
3602 #pragma unroll
3603     for (int stage = 0; stage < (kStage - 1); ++stage) {
3604         int load_gmem_K_Bc = stage * Bc + load_smem_K_Bc;
3605         int load_gmem_K_addr =
3606             K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
3607         uint32_t load_smem_K_ptr =
3608             smem_K_base_ptr +
3609             (stage * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
3610              load_smem_K_d) *
3611             sizeof(half);
3612 #pragma unroll
3613         for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
3614             CP_ASYNC.CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
3615         }
3616         CP_ASYNC_COMMIT_GROUP();
3617     }
3618     CP_ASYNC_WAIT_GROUP(kStage - 2);
3619     __syncthreads();
3620 }
3621
3622 // =====
3623 // Step 3: 外循环 — 沿 K seqlen 迭代 (Tc = ceil(seqlen/Bc))
3624 // 每次迭代处理一个 K[Bc,d] + V[Bc,d] tile
3625 // =====
3626 #pragma unroll 1
3627 for (int tile_K_seqlen = 0; tile_K_seqlen < Tc; ++tile_K_seqlen) {
3628     int smem_sel = tile_K_seqlen % kStage;
3629     int smem_sel_next = (tile_K_seqlen + (kStage - 1)) % kStage;
3630
3631     // ---- 3a: 异步加载 K/V tile (多 stage pipeline) ----
3632     if constexpr (kStage > 1) {
3633         // 只有 kStage>1 才能真正做 K 的 pipeline:
3634         // smem_sel 负责 “当前正在计算” 的 K tile, smem_sel_next 负责 “下一轮预取” 的 K tile。
3635         // 若 kStage=1, 这两个槽位永远都等于 0, 当前 K 还没算完就无法安全覆盖同一块 smem。
3636         // Load current V tile (no pipeline for V — one stage is enough)
3637         {
3638             int load_gmem_V_Bc = tile_K_seqlen * Bc + load_smem_V_Bc;
3639             int load_gmem_V_addr =
3640                 V_gmem_offset + load_gmem_V_Bc * kHeadDim + load_smem_V_d;
3641             uint32_t load_smem_V_ptr =
3642                 smem_V_base_ptr +
3643                 (load_smem_V_Bc * (kHeadDim + kPad) + load_smem_V_d) * sizeof(half);
3644 #pragma unroll
3645             for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
3646                 CP_ASYNC.CG(load_smem_V_ptr + i * 2, &V[load_gmem_V_addr + i], 16);
3647             }
3648             CP_ASYNC_COMMIT_GROUP();
3649         }
3650
3651         // Prefetch next K tile (pipelined)
3652         if ((tile_K_seqlen + 1) < Tc) {
3653             int load_gmem_K_Bc = (tile_K_seqlen + 1) * Bc + load_smem_K_Bc;

```

```

3654     int load_gmem_K_addr =
3655         K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
3656     uint32_t load_smem_K_ptr =
3657         smem_K_base_ptr +
3658         (smem_sel_next * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
3659         load_smem_K_d) *
3660         sizeof(half);
3661 #pragma unroll
3662     for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
3663         CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
3664     }
3665     CP_ASYNC_COMMIT_GROUP();
3666 }
3667 }
3668
3669 // ---- 3b: Q@K^T = S[Br, Bc] — 沿 head dim (d/kMmaAtomK=16) 内循环 ----
3670 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
3671 #pragma unroll
3672 for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
3673     // ldmatrix.x4: 加载 Q 的 m16k16 片段到 R_Q
3674 #pragma unroll
3675     for (int i = 0; i < kValTileSeqLenQ; ++i) {
3676         int warp_smem_Q_Br =
3677             warp_QP * (kMmaAtomM * kValTileSeqLenQ) + i * kMmaAtomM;
3678         int lane_smem_Q_Br =
3679             warp_smem_Q_Br + lane_id % 16; // ldmatrix uses 16 lanes
3680         int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8; // 0, 8
3681         uint32_t lane_smem_Q_ptr =
3682             smem_Q_base_ptr +
3683             (lane_smem_Q_Br * (kHeadDim + kPad) + lane_smem_Q_d) * sizeof(half);
3684         LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
3685             lane_smem_Q_ptr);
3686     }
3687
3688     // ldmatrix.x2: 加载 K 的 k16n8 片段到 R_K
3689     // K[Bc,d] row-major = K^T[d,Bc] col-major (NT 布局的 B 矩阵)
3690 #pragma unroll
3691     for (int j = 0; j < kValTileSeqLenK; ++j) {
3692         int warp_smem_K_Bc =
3693             warp_KV * (kMmaAtomN * kValTileSeqLenK) + j * kMmaAtomN;
3694         int lane_smem_K_Bc =
3695             warp_smem_K_Bc + lane_id % 8; // ldmatrix B uses 8 lanes
3696         int lane_smem_K_d =
3697             tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8; // 0, 8
3698         uint32_t lane_smem_K_ptr =
3699             smem_K_base_ptr +
3700             (smem_sel * KV_tile_size + lane_smem_K_Bc * (kHeadDim + kPad) +
3701             lane_smem_K_d) *
3702             sizeof(half);
3703         LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
3704     }
3705
3706     // MMA: S[tile] += Q[tile] @ K^T[tile]
3707 #pragma unroll
3708     for (int i = 0; i < kValTileSeqLenQ; ++i) {
3709 #pragma unroll
3710         for (int j = 0; j < kValTileSeqLenK; ++j) {
3711             HMMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1], R_Q[i][2],
3712                 R_Q[i][3], R_K[j][0], R_K[j][1], R_S[i][j][0],
3713                 R_S[i][j][1]);
3714         }
3715     }
3716 } // end loop over d

```

```

3717 __syncthreads();
3718
3719 // =====
3720 // 3c: Online Safe Softmax — row-wise max + exp + sum, then store P back to R_S
3721 // =====
3722 // MMA C fragment layout for m16n8k16 (PTX ISA 对应的线程寄存器分布):
3723 //   - R_S[i][j][0] 对应当前 16x8 tile 的 rows 0~7 里的两个 half 值 {c0,c1}
3724 //   - R_S[i][j][1] 对应 rows 8~15 里的两个 half 值 {c2,c3}
3725 //   - lane 0~3 持有 row 0 的片段, lane 4~7 持有 row 1, ..., lane 28~31 持有 row 7
3726 //   - 对于 rows 8~15 也是同样的 lane 分组, 只是读取的是 reg[1]
3727 // 这就是为什么后面做 row max / row sum 时, warp 内真正参与同一行归约的是
3728 // {0,4,8,...,28} 这一类 4-lane 子组, 而不是整 warp 32 个线程。
3729 // Each (i, j) pair = one 16x8 MMA tile; there are kValTileSeqLenQ x
3730 // kValTileSeqLenK tiles.
3731
3732 float lane_row_max_new[kValTileSeqLenQ][2];
3733 float lane_row_sum_new[kValTileSeqLenQ][2];
3734 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);
3735 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);
3736
3737 // Pass 1: Thread-level reduce max across all columns (kValTileSeqLenK tiles)
3738 #pragma unroll
3739 for (int i = 0; i < kValTileSeqLenQ; ++i) {
3740 #pragma unroll
3741 for (int j = 0; j < kValTileSeqLenK; ++j) {
3742     // Extract half values from R_S registers (C matrix fragment layout)
3743     float2 t_reg_S_0 =
3744         __half2float2(HALF2(R_S[i][j][0])); // rows 0~7: {c0, c1}
3745     float2 t_reg_S_1 =
3746         __half2float2(HALF2(R_S[i][j][1])); // rows 8~15: {c2, c3}
3747     // S = (Q@K^T) * scale
3748     float tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
3749     float tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
3750     lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
3751     lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
3752 }
3753 // Warp-level reduce max (warp_size = 4 for Q@K^T — only lanes
3754 // {0,4,8,...,28} hold valid data)
3755 lane_row_max_new[i][0] =
3756     warp_reduce_max<4, float>(lane_row_max_new[i][0]);
3757 lane_row_max_new[i][1] =
3758     warp_reduce_max<4, float>(lane_row_max_new[i][1]);
3759 }
3760
3761 // Pass 2: Compute P = exp(S*scale - m_new), store back to R_S
3762 // 面试关键点: 这里将 P 写回 R_S 寄存器!
3763 // 为什么可以? 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局,
3764 // 使 softmax 后的 P 能继续留在 R_S 中供后面的 P@V 直接消费, 无需额外重组。
3765 #pragma unroll
3766 for (int i = 0; i < kValTileSeqLenQ; ++i) {
3767     // m_new = max(m_old, m_cur)
3768     float block_row_max_new_0 =
3769         max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
3770     float block_row_max_new_1 =
3771         max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);
3772
3773 #pragma unroll
3774 for (int j = 0; j < kValTileSeqLenK; ++j) {
3775     float2 t_reg_S_0 = __half2float2(HALF2(R_S[i][j][0]));
3776     float2 t_reg_S_1 = __half2float2(HALF2(R_S[i][j][1]));
3777
3778     // P = exp(S * scale - m_new), 用 fma 保证精度
3779     t_reg_S_0.x =

```

```

3780     __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
3781     t_reg_S_0.y =
3782     __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
3783     t_reg_S_1.x =
3784     __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
3785     t_reg_S_1.y =
3786     __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));
3787
3788     // Accumulate row sums
3789     lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
3790     lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);
3791
3792     // 关键: 将 P 写回 R_S! R_S 现在存储的是 P = softmax(S), 不是 S
3793     HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
3794     HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);
3795 }
3796
3797 // Warp-level reduce sum (warp_size = 4, same as max)
3798 lane_row_sum_new[i][0] =
3799     warp_reduce_sum<4, float>(lane_row_sum_new[i][0]);
3800 lane_row_sum_new[i][1] =
3801     warp_reduce_sum<4, float>(lane_row_sum_new[i][1]);
3802 }
3803 __syncthreads();
3804
3805 // =====
3806 // 3d: P@V — P[Br,Bc] @ V[Bc,d] = O[Br,d]
3807 // =====
3808 // Wait for V to be ready before computing P@V
3809 if constexpr (kStage > 1) {
3810     if ((tile_K_seqLen + 1) < Tc) {
3811         CP_ASYNC_WAIT_GROUP(1);
3812     } else {
3813         CP_ASYNC_WAIT_GROUP(0);
3814     }
3815 } else {
3816     CP_ASYNC_WAIT_GROUP(0);
3817 }
3818 __syncthreads();
3819
3820 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_0, 0);
3821
3822 // tile_V_Bc: iterate over chunks of Bc/K=16 columns in P matrix
3823 // Bc=kMmaAtomK=16 → 1 iteration for kHeadDim≤64 configurations
3824 #pragma unroll
3825 for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
3826     // ldmatrix.x2.trans: load V[Bc,d] with transposition
3827     // V is row-major [Bc,d], but NN matmul needs B matrix in col-major → use
3828     // transposed ldmatrix
3829 #pragma unroll
3830 for (int j = 0; j < kValTileHeadDimV; ++j) {
3831     int warp_smem_V_d =
3832         warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
3833     int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
3834     int lane_smem_V_d = warp_smem_V_d;
3835     uint32_t lane_smem_V_ptr =
3836         smem_V_base_ptr +
3837         (lane_smem_V_Bc * (kHeadDim + kPad) + lane_smem_V_d) * sizeof(half);
3838     LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
3839 }
3840
3841 // P matrix layout in R_S[i][j][2]:
3842 // MMA = m16n8k16, Br=16x4=64, Bc=8x8=64, layout: 4 warps

```

```

3843 // | 64x64 | warp_KV 0 |
3844 // | warp_QP 0 | MMA 0 ... MMA 0 (x8) |
3845 // | warp_QP 1 | MMA 1 ... MMA 1 (x8) |
3846 // | warp_QP 2 | MMA 2 ... MMA 2 (x8) |
3847 // | warp_QP 3 | MMA 3 ... MMA 3 (x8) |
3848 // tile_V_Bc selects which 16-column slice of P to use:
3849 // tile_V_Bc=0 → cols 0:16 → MMA indices 0,1 → w=0
3850 // tile_V_Bc=1 → cols 16:32 → MMA indices 2,3 → w=2
3851 // tile_V_Bc=2 → cols 32:48 → MMA indices 4,5 → w=4
3852 // tile_V_Bc=3 → cols 48:64 → MMA indices 6,7 → w=6
3853 // 对应的 MMA A fragment 布局可以这样记:
3854 // - rows 0~7: lane 0~3 → {a0,a1} 与 {a4,a5}, lane 4~7 → 下一行, 依次类推
3855 // - rows 8~15: lane 0~3 → {a2,a3} 与 {a6,a7}, lane 4~7 → 下一行, 依次类推
3856 // 当前实现正是利用这一路径下 A fragment 与前面生成的 P fragment 可以直接对接,
3857 // 才能把 R_S 中的 P 直接喂给 HMMA16816 做 P@V; 复习时不要把它背成对所有 MMA
3858 // fragment 都无条件成立的通用结论。layout转换逻辑:
3859 // C layout: 8 x (16 x 8) layout → A layout: 4 x (16 x 16) layout
3860 int w = tile_V_Bc * 2;
3861 #pragma unroll
3862 for (int i = 0; i < kValTileSeqLenP; ++i) {
3863 #pragma unroll
3864 for (int j = 0; j < kValTileHeadDimV; ++j) {
3865     HMMA16816(R_0[i][j][0], R_0[i][j][1], // C fragment output
3866              R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][w + 1][1], // A fragment = P
3867              R_V[j][0], R_V[j][1], // B fragment = V
3868              R_0[i][j][0], R_0[i][j][1]); // C fragment output
3869     }
3870 }
3871 } // end for tile_V_Bc
3872 __syncthreads();
3873
3874 // =====
3875 // 3e: Online rescaling —  $O_{new} = \exp(m_{old} - m_{new}) * O_{old} + P@V$ 
3876 // =====
3877 // 公式来源: FA2 paper Eq.(7-8), 使用  $\exp(m_{old} - m_{new})$  做 0 与 1 的 rescale
3878 #pragma unroll
3879 for (int i = 0; i < kValTileSeqLenP; ++i) {
3880     float block_row_max_new_0 = lane_row_max_new[i][0];
3881     float block_row_max_new_1 = lane_row_max_new[i][1];
3882     float block_row_sum_new_0 = lane_row_sum_new[i][0];
3883     float block_row_sum_new_1 = lane_row_sum_new[i][1];
3884
3885     float block_row_max_old_0 = lane_block_row_max_old[i][0];
3886     float block_row_max_old_1 = lane_block_row_max_old[i][1];
3887
3888     block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0);
3889     block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1);
3890
3891     // Handle first iteration: m_old = -inf, need to use m_new directly
3892     block_row_max_old_0 =
3893         (tile_K_seqlen > 0 ? block_row_max_old_0 : block_row_max_new_0);
3894     block_row_max_old_1 =
3895         (tile_K_seqlen > 0 ? block_row_max_old_1 : block_row_max_new_1);
3896
3897     float rescale_o_factor_0 =
3898         __expf(block_row_max_old_0 - block_row_max_new_0);
3899     float rescale_o_factor_1 =
3900         __expf(block_row_max_old_1 - block_row_max_new_1);
3901
3902     // Rescale O_old + Add P@V in one fused step
3903 #pragma unroll
3904 for (int j = 0; j < kValTileHeadDimV; ++j) {
3905     // R_0 / R_D 与前面的 R_S 一样, 都按 MMA C fragment 布局解释:

```

```

3906 // reg[0] -> rows 0~7 的 {c0,c1}
3907 // reg[1] -> rows 8~15 的 {c2,c3}
3908 float2 t_reg_0_0 = __half22float2(HALF2(R_0[i][j][0]));
3909 float2 t_reg_0_1 = __half22float2(HALF2(R_0[i][j][1]));
3910 float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
3911 float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
3912
3913 // O_new = exp(m_old - m_new) * O_old + P@V (fused multiply-add)
3914 t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x, t_reg_0_0.x);
3915 t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y, t_reg_0_0.y);
3916 t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x, t_reg_0_1.x);
3917 t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y, t_reg_0_1.y);
3918
3919 HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
3920 HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
3921 }
3922
3923 // Update l: l_new = exp(m_old - m_new) * l_old + row_sum(P)
3924 float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
3925 float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
3926 lane_block_row_sum_old[i][0] = __fmaf_rn(
3927     rescale_o_factor_0, block_row_sum_old_0, block_row_sum_new_0);
3928 lane_block_row_sum_old[i][1] = __fmaf_rn(
3929     rescale_o_factor_1, block_row_sum_old_1, block_row_sum_new_1);
3930
3931 // Update m
3932 lane_block_row_max_old[i][0] = block_row_max_new_0;
3933 lane_block_row_max_old[i][1] = block_row_max_new_1;
3934 }
3935
3936 // Wait for next K tile to be ready in smem before next iteration
3937 if constexpr (kStage > 1) {
3938     if ((tile_K_seqlen + 1) < Tc) {
3939         CP_ASYNC_WAIT_GROUP(0);
3940     }
3941     __syncthreads();
3942 }
3943 } // end outer loop over K seqlen
3944 __syncthreads();
3945
3946 // =====
3947 // Step 4: 最终 rescale — O_final = (1/l_final) * O_final
3948 // =====
3949 #pragma unroll
3950 for (int i = 0; i < kValTileSeqLenP; ++i) {
3951     float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);
3952     float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
3953 #pragma unroll
3954 for (int j = 0; j < kValTileHeadDimV; ++j) {
3955     float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
3956     float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
3957     t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
3958     t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
3959     t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
3960     t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
3961     HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
3962     HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
3963 }
3964 }
3965
3966 // =====
3967 // Step 5: Epilogue — Collective store via warp shuffle + 128-bit store
3968 // =====

```

```

3969 // 利用 warp shuffle 将分散在各 lane 的寄存器数据收集到 lane 0~3,
3970 // 然后用 LDST128BITS (st.global.v4.f32) 一次性写入 16 bytes
3971 #pragma unroll
3972 for (int i = 0; i < kValTileSeqLenP; ++i) {
3973 #pragma unroll
3974 for (int j = 0; j < kValTileHeadDimV; ++j) {
3975     uint32_t R_Z[2][4];
3976     R_Z[0][0] = R_D[i][j][0];
3977     R_Z[1][0] = R_D[i][j][1];
3978     R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
3979     R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
3980     R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
3981     R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
3982     R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
3983     R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);
3984
3985     // st.global.v4.f32: 128-bit store, 4 lanes × 32-bit
3986     if (lane_id % 4 == 0) {
3987         int store_warp_regs_0_Br =
3988             warp_QP * (kMmaAtomM * kValTileSeqLenP) + i * kMmaAtomM;
3989         int store_lane_gmem_0_Br =
3990             Q_tile_id * Br + store_warp_regs_0_Br + lane_id / 4;
3991         int store_warp_regs_0_d =
3992             warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
3993         int store_lane_gmem_0_d = store_warp_regs_0_d;
3994         int store_gmem_0_addr_0 = 0_gmem_offset +
3995             (store_lane_gmem_0_Br + 0) * kHeadDim +
3996             store_lane_gmem_0_d;
3997         int store_gmem_0_addr_1 = 0_gmem_offset +
3998             (store_lane_gmem_0_Br + 8) * kHeadDim +
3999             store_lane_gmem_0_d;
4000         // LDST128BITS = reinterpret_cast<float4*>
4001         *reinterpret_cast<float4*>(&0[store_gmem_0_addr_0]) =
4002             *reinterpret_cast<float4*>(&R_Z[0][0]);
4003         *reinterpret_cast<float4*>(&0[store_gmem_0_addr_1]) =
4004             *reinterpret_cast<float4*>(&R_Z[1][0]);
4005     }
4006 }
4007 }
4008 }
4009
4010 // =====
4011 // 以下是测试代码, 验证 Phase 1 - Phase 8 的kernel的正确性, 不评估性能。
4012 // =====
4013
4014 static inline void check(cudaError_t err, const char *msg) {
4015     if (err != cudaSuccess) {
4016         fprintf(stderr, "[ERROR] %s: %s\n", msg, cudaGetErrorString(err));
4017         exit(EXIT_FAILURE);
4018     }
4019 }
4020
4021 static void test_block_reduce(int N) {
4022
4023     srand(42);
4024     float *h_a = (float *)malloc((size_t)N * sizeof(float));
4025     for (int i = 0; i < N; i++)
4026         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4027
4028     // CPU reference: sum of all elements
4029     double ref = 0.0;
4030     for (int i = 0; i < N; i++) ref += (double)h_a[i];

```

```

4032
4033 float *d_a, *d_y;
4034 check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "blockreduce alloc A");
4035 check(cudaMalloc(&d_y, sizeof(float)), "blockreduce alloc Y");
4036
4037 check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "blockreduce H2D A");
4038 check(cudaMemset(d_y, 0, sizeof(float)), "blockreduce zero Y");
4039
4040 dim3 block(128);
4041 dim3 grid((N + 127) / 128);
4042 block_reduce_all<128><<<grid, block>>>(d_a, d_y, N);
4043 check(cudaGetLastError(), "blockreduce launch");
4044 check(cudaDeviceSynchronize(), "blockreduce sync");
4045
4046 float result;
4047 check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "blockreduce D2H");
4048
4049 float err = fabsf(result - (float)ref);
4050 printf("| %-35s | %.6e | %-4s |\n", "BlockReduce", err,
4051        err < 1e-2f ? "PASS" : "FAIL");
4052
4053 free(h_a);
4054 cudaFree(d_a); cudaFree(d_y);
4055 }
4056
4057
4058 static void test_dot(int N) {
4059
4060     srand(42);
4061     float *h_a = (float *)malloc((size_t)N * sizeof(float));
4062     float *h_b = (float *)malloc((size_t)N * sizeof(float));
4063     for (int i = 0; i < N; i++) {
4064         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4065         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4066     }
4067
4068     // CPU reference
4069     double ref = 0.0;
4070     for (int i = 0; i < N; i++) ref += (double)h_a[i] * (double)h_b[i];
4071
4072     float *d_a, *d_b, *d_y;
4073     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "dot alloc A");
4074     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "dot alloc B");
4075     check(cudaMalloc(&d_y, sizeof(float)), "dot alloc Y");
4076
4077     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D A");
4078     check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D B");
4079     check(cudaMemset(d_y, 0, sizeof(float)), "dot zero Y");
4080
4081     dim3 block(128);
4082     dim3 grid((N + 127) / 128);
4083     dot<128><<<grid, block>>>(d_a, d_b, d_y, N);
4084     check(cudaGetLastError(), "dot launch");
4085     check(cudaDeviceSynchronize(), "dot sync");
4086
4087     float result;
4088     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot D2H");
4089
4090     float err = fabsf(result - (float)ref);
4091     printf("| %-35s | %.6e | %-4s |\n", "Dot", err,
4092            err < 1e-2f ? "PASS" : "FAIL");
4093
4094     // ---- Dot Vec4 ----

```

```

4095 check(cudaMemset(d_y, 0, sizeof(float)), "dot_vec4 zero Y");
4096 dim3 block_v4(32);
4097 dot_vec4<<<grid, block_v4>>>(d_a, d_b, d_y, N);
4098 check(cudaGetLastError(), "dot_vec4 launch");
4099 check(cudaDeviceSynchronize(), "dot_vec4 sync");
4100
4101 check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot_vec4 D2H");
4102 float err_v4 = fabsf(result - (float)ref);
4103 printf("| %-35s | %.6e | %-4s |\n", "Dot-Vec4", err_v4, err_v4 < 1e-2f ? "PASS" : "FAIL");
4104
4105 free(h_a); free(h_b);
4106 cudaFree(d_a); cudaFree(d_b); cudaFree(d_y);
4107 }
4108
4109
4110 static void test_relu(int N) {
4111     srand(42);
4112     float *h_x = (float *)malloc((size_t)N * sizeof(float));
4113     float *h_y = (float *)malloc((size_t)N * sizeof(float));
4114     for (int i = 0; i < N; i++)
4115         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4116
4117     float *d_x, *d_y;
4118     check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "relu alloc X");
4119     check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "relu alloc Y");
4120     check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "relu H2D");
4121
4122     for (int i = 0; i < N; i++) h_y[i] = fmaxf(0.0f, h_x[i]);
4123     dim3 block256(256);
4124     dim3 grid256((N + 255) / 256);
4125     relu<<<grid256, block256>>>(d_x, d_y, N);
4126     check(cudaGetLastError(), "relu launch");
4127     check(cudaDeviceSynchronize(), "relu sync");
4128     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu D2H");
4129     float max_err = 0.0f;
4130     for (int i = 0; i < N; i++) {
4131         float expected = fmaxf(0.0f, h_x[i]);
4132         float err = fabsf(h_y[i] - expected);
4133         if (err > max_err) max_err = err;
4134     }
4135     printf("| %-35s | %.6e | %-4s |\n", "ReLU", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4136
4137     dim3 block64(64);
4138     relu_vec4<<<grid256, block64>>>(d_x, d_y, N);
4139     check(cudaGetLastError(), "relu_vec4 launch");
4140     check(cudaDeviceSynchronize(), "relu_vec4 sync");
4141     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu_vec4 D2H");
4142     max_err = 0.0f;
4143     for (int i = 0; i < N; i++) {
4144         float expected = fmaxf(0.0f, h_x[i]);
4145         float err = fabsf(h_y[i] - expected);
4146         if (err > max_err) max_err = err;
4147     }
4148     printf("| %-35s | %.6e | %-4s |\n", "ReLU-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4149
4150     free(h_x); free(h_y);
4151     cudaFree(d_x); cudaFree(d_y);
4152 }
4153
4154
4155 static void test_elementwise(int N) {
4156     srand(42);
4157     float *h_a = (float *)malloc((size_t)N * sizeof(float));

```

```

4158 float *h_b = (float *)malloc((size_t)N * sizeof(float));
4159 for (int i = 0; i < N; i++) {
4160     h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4161     h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4162 }
4163
4164 float *d_a, *d_b, *d_c;
4165 check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "eadd alloc A");
4166 check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "eadd alloc B");
4167 check(cudaMalloc(&d_c, (size_t)N * sizeof(float)), "eadd alloc C");
4168 check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D A");
4169 check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D B");
4170
4171 dim3 block256(256);
4172 dim3 grid256((N + 255) / 256);
4173 elementwise_add<<<grid256, block256>>>(d_a, d_b, d_c, N);
4174 check(cudaGetLastError(), "eadd launch");
4175 check(cudaDeviceSynchronize(), "eadd sync");
4176 float *h_c = (float *)malloc((size_t)N * sizeof(float));
4177 check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd D2H");
4178 float max_err = 0.0f;
4179 for (int i = 0; i < N; i++) {
4180     float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
4181     if (err > max_err) max_err = err;
4182 }
4183 printf("| %-35s | %.6e | %-4s |\n", "ElemwiseAdd", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4184
4185 dim3 block64(64);
4186 check(cudaMemset(d_c, 0, (size_t)N * sizeof(float)), "eadd_vec4 zero C");
4187 elementwise_add_vec4<<<grid256, block64>>>(d_a, d_b, d_c, N);
4188 check(cudaGetLastError(), "eadd_vec4 launch");
4189 check(cudaDeviceSynchronize(), "eadd_vec4 sync");
4190 check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd_vec4 D2H");
4191 max_err = 0.0f;
4192 for (int i = 0; i < N; i++) {
4193     float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
4194     if (err > max_err) max_err = err;
4195 }
4196 printf("| %-35s | %.6e | %-4s |\n", "ElemwiseAdd-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4197
4198 free(h_a); free(h_b); free(h_c);
4199 cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
4200 }
4201
4202
4203 static void test_histogram(int N) {
4204     srand(42);
4205     int BINS = 16;
4206     int *h_hist = (int *)calloc(BINS, sizeof(int));
4207     int *h_hist_ref = (int *)calloc(BINS, sizeof(int));
4208     int *h_idx = (int *)malloc((size_t)N * sizeof(int));
4209     for (int i = 0; i < N; i++) h_idx[i] = rand() % BINS;
4210     for (int i = 0; i < N; i++) h_hist_ref[h_idx[i]]++;
4211
4212     int *d_idx, *d_hist;
4213     check(cudaMalloc(&d_idx, (size_t)N * sizeof(int)), "hist alloc idx");
4214     check(cudaMalloc(&d_hist, BINS * sizeof(int)), "hist alloc hist");
4215     check(cudaMemcpy(d_idx, h_idx, (size_t)N * sizeof(int), cudaMemcpyHostToDevice), "hist H2D idx");
4216     check(cudaMemset(d_hist, 0, BINS * sizeof(int)), "hist zero");
4217
4218     dim3 block256(256);
4219     dim3 grid256((N + 255) / 256);
4220     histogram<<<grid256, block256>>>(d_idx, d_hist, N);

```

```

4221 check(cudaGetLastError(), "histogram launch");
4222 check(cudaDeviceSynchronize(), "histogram sync");
4223 check(cudaMemcpy(h_hist, d_hist, BINS * sizeof(int), cudaMemcpyDeviceToHost), "hist D2H");
4224
4225 float max_err = 0.0f;
4226 for (int i = 0; i < BINS; i++) {
4227     float err = fabsf((float)(h_hist[i] - h_hist_ref[i]));
4228     if (err > max_err) max_err = err;
4229 }
4230 printf("| %-35s | %.6e | %-4s |\n", "Histogram", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4231
4232 free(h_hist); free(h_hist_ref); free(h_idx);
4233 cudaFree(d_idx); cudaFree(d_hist);
4234 }
4235
4236
4237 static void test_merge_attn_states(int num_tokens, int num_heads,
4238                                   int head_size) {
4239
4240     size_t out_size = (size_t)num_tokens * num_heads * head_size * sizeof(float);
4241     size_t lse_size = (size_t)num_heads * num_tokens * sizeof(float);
4242
4243     float *h_prefix_out = (float *)malloc(out_size);
4244     float *h_suffix_out = (float *)malloc(out_size);
4245     float *h_prefix_lse = (float *)malloc(lse_size);
4246     float *h_suffix_lse = (float *)malloc(lse_size);
4247     float *h_output_ref = (float *)malloc(out_size);
4248
4249     srand(42);
4250     for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
4251         h_prefix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4252         h_suffix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4253     }
4254     for (int i = 0; i < num_heads * num_tokens; i++) {
4255         h_prefix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
4256         h_suffix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
4257     }
4258
4259     // CPU reference: 与 kernel 完全相同的浮点计算
4260     for (int t = 0; t < num_tokens; t++) {
4261         for (int h = 0; h < num_heads; h++) {
4262             float p_lse = h_prefix_lse[h * num_tokens + t];
4263             float s_lse = h_suffix_lse[h * num_tokens + t];
4264             p_lse = isinf(p_lse) ? -INFINITY : p_lse;
4265             s_lse = isinf(s_lse) ? -INFINITY : s_lse;
4266
4267             float max_lse = fmaxf(p_lse, s_lse);
4268             p_lse -= max_lse;
4269             s_lse -= max_lse;
4270             float p_se = expf(p_lse);
4271             float s_se = expf(s_lse);
4272             float p_scale = p_se / (p_se + s_se);
4273             float s_scale = s_se / (p_se + s_se);
4274
4275             int head_off = t * num_heads * head_size + h * head_size;
4276             for (int d = 0; d < head_size; d++) {
4277                 h_output_ref[head_off + d] =
4278                     h_prefix_out[head_off + d] * p_scale +
4279                     h_suffix_out[head_off + d] * s_scale;
4280             }
4281         }
4282     }
4283 }

```

```

4284 float *d_prefix_out, *d_suffix_out, *d_prefix_lse, *d_suffix_lse, *d_output;
4285 check(cudaMalloc(&d_prefix_out, out_size), "merge_attn alloc p_out");
4286 check(cudaMalloc(&d_suffix_out, out_size), "merge_attn alloc s_out");
4287 check(cudaMalloc(&d_prefix_lse, lse_size), "merge_attn alloc p_lse");
4288 check(cudaMalloc(&d_suffix_lse, lse_size), "merge_attn alloc s_lse");
4289 check(cudaMalloc(&d_output, out_size), "merge_attn alloc output");
4290
4291 check(cudaMemcpy(d_prefix_out, h_prefix_out, out_size,
4292                 cudaMemcpyHostToDevice),
4293         "merge_attn H2D p_out");
4294 check(cudaMemcpy(d_suffix_out, h_suffix_out, out_size,
4295                 cudaMemcpyHostToDevice),
4296         "merge_attn H2D s_out");
4297 check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
4298                 cudaMemcpyHostToDevice),
4299         "merge_attn H2D p_lse");
4300 check(cudaMemcpy(d_suffix_lse, h_suffix_lse, lse_size,
4301                 cudaMemcpyHostToDevice),
4302         "merge_attn H2D s_lse");
4303
4304 int threads_per_head = head_size / 4;
4305 int total_threads = num_tokens * num_heads * threads_per_head;
4306 dim3 block(128);
4307 dim3 grid((total_threads + 127) / 128);
4308 merge_attn_states<<<grid, block>>>(
4309     d_output, d_prefix_out, d_prefix_lse, d_suffix_out, d_suffix_lse,
4310     num_tokens, num_heads, head_size);
4311 check(cudaGetLastError(), "merge_attn launch");
4312 check(cudaDeviceSynchronize(), "merge_attn sync");
4313
4314 float *h_output = (float *)malloc(out_size);
4315 check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),
4316         "merge_attn D2H");
4317
4318 float max_err = 0.0f;
4319 for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
4320     float err = fabsf(h_output[i] - h_output_ref[i]);
4321     if (err > max_err) max_err = err;
4322 }
4323 printf("| %-35s | %.6e | %-4s |\n", "MergeAttnStates", max_err,
4324        max_err < 1e-4f ? "PASS" : "FAIL");
4325
4326 // 边界测试: +inf LSE → 权重退化为 0 (空 attention 段)
4327 h_prefix_lse[0] = INFINITY; // head 0, token 0 的 LSE = +inf
4328 h_suffix_lse[0] = 0.0f;
4329 check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
4330                 cudaMemcpyHostToDevice),
4331         "merge_attn H2D inf lse");
4332 merge_attn_states<<<grid, block>>>(
4333     d_output, d_prefix_out, d_prefix_lse, d_suffix_out, d_suffix_lse,
4334     num_tokens, num_heads, head_size);
4335 check(cudaGetLastError(), "merge_attn inf launch");
4336 check(cudaDeviceSynchronize(), "merge_attn inf sync");
4337 check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),
4338         "merge_attn inf D2H");
4339
4340 // token 0, head 0 的所有元素应等于 suffix_output (prefix 权重  $\alpha=0$ )
4341 float inf_err = 0.0f;
4342 for (int d = 0; d < head_size; d++) {
4343     float err = fabsf(h_output[d] - h_suffix_out[d]);
4344     if (err > inf_err) inf_err = err;
4345 }
4346 printf("| %-35s | %.6e | %-4s |\n", "MergeAttnStates-inf", inf_err,

```

```

4347         inf_err < 1e-4f ? "PASS" : "FAIL");
4348
4349     free(h_prefix_out);
4350     free(h_suffix_out);
4351     free(h_prefix_lse);
4352     free(h_suffix_lse);
4353     free(h_output_ref);
4354     free(h_output);
4355     cudaFree(d_prefix_out);
4356     cudaFree(d_suffix_out);
4357     cudaFree(d_prefix_lse);
4358     cudaFree(d_suffix_lse);
4359     cudaFree(d_output);
4360 }
4361
4362
4363 static void test_softmax(int N) {
4364     // Use N = blockDim.x so one block processes all elements as one token.
4365     constexpr int kNumThreads = 256;
4366     if (N != kNumThreads) {
4367         printf("  OnlineSafeSoftmax: N must be %d (got %d), skipping.\n", kNumThreads, N);
4368         return;
4369     }
4370
4371     srand(42);
4372     float *h_x = (float *)malloc((size_t)N * sizeof(float));
4373     float *h_y_ref = (float *)malloc((size_t)N * sizeof(float));
4374     for (int i = 0; i < N; i++)
4375         h_x[i] = ((float)rand() / RAND_MAX) * 10.0f - 5.0f; // [-5, 5]
4376
4377     // CPU reference: softmax(x_i) = exp(x_i - max) / sum(exp(x_j - max))
4378     float max_val = -FLT_MAX;
4379     for (int i = 0; i < N; i++) if (h_x[i] > max_val) max_val = h_x[i];
4380     double sum_exp = 0.0;
4381     for (int i = 0; i < N; i++) sum_exp += (double)expf(h_x[i] - max_val);
4382     for (int i = 0; i < N; i++)
4383         h_y_ref[i] = expf(h_x[i] - max_val) / (float)sum_exp;
4384
4385     float *d_x, *d_y;
4386     check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "softmax alloc X");
4387     check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "softmax alloc Y");
4388
4389     check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "softmax H2D X");
4390
4391     dim3 block(256);
4392     dim3 grid(1); // one token covering all N elements
4393     online_safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
4394     check(cudaGetLastError(), "softmax launch");
4395     check(cudaDeviceSynchronize(), "softmax sync");
4396
4397     float *h_y = (float *)malloc((size_t)N * sizeof(float));
4398     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "softmax D2H");
4399
4400     float max_err = 0.0f;
4401     for (int i = 0; i < N; i++) {
4402         float err = fabsf(h_y[i] - h_y_ref[i]);
4403         if (err > max_err) max_err = err;
4404     }
4405     printf("| %-35s | %.6e | %-4s |\n", "OnlineSafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4406
4407     // ---- Safe Softmax ----
4408     safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
4409     check(cudaGetLastError(), "safe_softmax launch");

```

```

4410 check(cudaDeviceSynchronize(), "safe_softmax sync");
4411 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "safe_softmax D2H");
4412 max_err = 0.0f;
4413 for (int i = 0; i < N; i++) {
4414     float err = fabsf(h_y[i] - h_y_ref[i]);
4415     if (err > max_err) max_err = err;
4416 }
4417 printf("| %-35s | %.6e | %-4s |\n", "SafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4418
4419 // ---- Naive Softmax ----
4420 softmax_per_token<<<grid, block>>>(d_x, d_y, N);
4421 check(cudaGetLastError(), "naive_softmax launch");
4422 check(cudaDeviceSynchronize(), "naive_softmax sync");
4423 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "naive_softmax D2H");
4424 max_err = 0.0f;
4425 for (int i = 0; i < N; i++) {
4426     float err = fabsf(h_y[i] - h_y_ref[i]);
4427     if (err > max_err) max_err = err;
4428 }
4429 printf("| %-35s | %.6e | %-4s |\n", "NaiveSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4430
4431 free(h_x); free(h_y); free(h_y_ref);
4432 cudaFree(d_x); cudaFree(d_y);
4433 }
4434
4435
4436 static void test_rms_norm(int N, int K) {
4437
4438     srand(42);
4439     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
4440     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
4441     for (int i = 0; i < N * K; i++)
4442         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4443     float g = 1.5f; // gain
4444
4445     // CPU reference: y = (x / rms(x)) * g
4446     float epsilon = 1e-5f;
4447     for (int n = 0; n < N; n++) {
4448         double sum_sq = 0.0;
4449         for (int k = 0; k < K; k++) sum_sq += (double)h_x[n * K + k] * (double)h_x[n * K + k];
4450         float rms = sqrtf((float)sum_sq / (float)K + epsilon);
4451         for (int k = 0; k < K; k++)
4452             h_y_ref[n * K + k] = (h_x[n * K + k] / rms) * g;
4453     }
4454
4455     float *d_x, *d_y;
4456     check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "rmsnorm alloc X");
4457     check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "rmsnorm alloc Y");
4458     check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "rmsnorm H2D X");
4459
4460     dim3 block(128);
4461     dim3 grid(N);
4462     rms_norm<<<grid, block>>>(d_x, d_y, g, N, K);
4463     check(cudaGetLastError(), "rmsnorm launch");
4464     check(cudaDeviceSynchronize(), "rmsnorm sync");
4465
4466     float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
4467     check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm D2H");
4468
4469     float max_err = 0.0f;
4470     for (int i = 0; i < N * K; i++) {
4471         float err = fabsf(h_y[i] - h_y_ref[i]);
4472         if (err > max_err) max_err = err;

```

```

4473 }
4474 printf("| %-35s | %.6e | %-4s |\n", "RMSNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4475
4476 // ---- RMS Norm Vec4 ----
4477 dim3 block_rv4(32);
4478 rms_norm_vec4<<<grid, block_rv4>>>(d_x, d_y, g, N, K);
4479 check(cudaGetLastError(), "rmsnorm_vec4 launch");
4480 check(cudaDeviceSynchronize(), "rmsnorm_vec4 sync");
4481 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm_vec4 D2H");
4482 max_err = 0.0f;
4483 for (int i = 0; i < N * K; i++) {
4484     float err = fabsf(h_y[i] - h_y_ref[i]);
4485     if (err > max_err) max_err = err;
4486 }
4487 printf("| %-35s | %.6e | %-4s |\n", "RMSNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4488
4489 free(h_x); free(h_y); free(h_y_ref);
4490 cudaFree(d_x); cudaFree(d_y);
4491 }
4492
4493
4494 static void test_layer_norm(int N, int K) {
4495
4496     srand(42);
4497     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
4498     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
4499     for (int i = 0; i < N * K; i++)
4500         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4501     float g = 1.5f, b = 0.3f; // gain and bias
4502
4503     // CPU reference: y = ((x - mean) / std) * g + b
4504     float epsilon = 1e-5f;
4505     for (int n = 0; n < N; n++) {
4506         double sum = 0.0;
4507         for (int k = 0; k < K; k++) sum += (double)h_x[n * K + k];
4508         float mean = (float)sum / (float)K;
4509         double sum_sq = 0.0;
4510         for (int k = 0; k < K; k++) {
4511             float diff = h_x[n * K + k] - mean;
4512             sum_sq += (double)diff * (double)diff;
4513         }
4514         float std = sqrtf((float)sum_sq / (float)K + epsilon);
4515         for (int k = 0; k < K; k++)
4516             h_y_ref[n * K + k] = ((h_x[n * K + k] - mean) / std) * g + b;
4517     }
4518
4519     float *d_x, *d_y;
4520     check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "layernorm alloc X");
4521     check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "layernorm alloc Y");
4522     check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "layernorm H2D X");
4523
4524     dim3 block(128);
4525     dim3 grid(N);
4526     layer_norm<<<grid, block>>>(d_x, d_y, g, b, N, K);
4527     check(cudaGetLastError(), "layernorm launch");
4528     check(cudaDeviceSynchronize(), "layernorm sync");
4529
4530     float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
4531     check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm D2H");
4532
4533     float max_err = 0.0f;
4534     for (int i = 0; i < N * K; i++) {
4535         float err = fabsf(h_y[i] - h_y_ref[i]);

```

```

4536     if (err > max_err) max_err = err;
4537 }
4538 printf("| %-35s | %.6e | %-4s |\n", "LayerNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4539
4540 // ---- Layer Norm Vec4 ----
4541 dim3 block_lv4(32);
4542 layer_norm_vec4<<<grid, block_lv4>>>(d_x, d_y, g, b, N, K);
4543 check(cudaGetLastError(), "layernorm_vec4 launch");
4544 check(cudaDeviceSynchronize(), "layernorm_vec4 sync");
4545 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm_vec4 D2H");
4546 max_err = 0.0f;
4547 for (int i = 0; i < N * K; i++) {
4548     float err = fabsf(h_y[i] - h_y_ref[i]);
4549     if (err > max_err) max_err = err;
4550 }
4551 printf("| %-35s | %.6e | %-4s |\n", "LayerNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4552
4553 free(h_x); free(h_y); free(h_y_ref);
4554 cudaFree(d_x); cudaFree(d_y);
4555 }
4556
4557
4558 static void test_rope(int seq_len, int N) {
4559
4560     int total_pairs = seq_len * N;
4561     int total_elems = total_pairs * 2;
4562     size_t size = (size_t)total_elems * sizeof(float);
4563
4564     float *h_x = (float *)malloc(size);
4565     float *h_y_ref = (float *)malloc(size);
4566
4567     srand(42);
4568     for (int i = 0; i < total_elems; i++)
4569         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4570
4571     // CPU reference: 2D rotation for each pair
4572     for (int idx = 0; idx < total_pairs; idx++) {
4573         int token_pos = idx / N;
4574         int token_idx = idx % N;
4575         float x1 = h_x[idx * 2];
4576         float x2 = h_x[idx * 2 + 1];
4577         float theta = 1.0f / powf(10000.0f, 2.0f * token_idx / (N * 2.0f));
4578         float angle = (float)token_pos * theta;
4579         float cos_v = cosf(angle);
4580         float sin_v = sinf(angle);
4581         h_y_ref[idx * 2] = x1 * cos_v - x2 * sin_v;
4582         h_y_ref[idx * 2 + 1] = x1 * sin_v + x2 * cos_v;
4583     }
4584
4585     float *d_x, *d_y;
4586     check(cudaMalloc(&d_x, size), "rope alloc X");
4587     check(cudaMalloc(&d_y, size), "rope alloc Y");
4588     check(cudaMemcpy(d_x, h_x, size, cudaMemcpyHostToDevice), "rope H2D");
4589
4590     dim3 block(256);
4591     dim3 grid((total_pairs + 255) / 256);
4592     rope<<<grid, block>>>(d_x, d_y, seq_len, N);
4593     check(cudaGetLastError(), "rope launch");
4594     check(cudaDeviceSynchronize(), "rope sync");
4595
4596     float *h_y = (float *)malloc(size);
4597     check(cudaMemcpy(h_y, d_y, size, cudaMemcpyDeviceToHost), "rope D2H");
4598

```

```

4599 float max_err = 0.0f;
4600 for (int i = 0; i < total_elems; i++) {
4601     float err = fabsf(h_y[i] - h_y_ref[i]);
4602     if (err > max_err) max_err = err;
4603 }
4604 printf("| %-35s | %.6e | %-4s |\n", "RoPE", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4605
4606 free(h_x);
4607 free(h_y);
4608 free(h_y_ref);
4609 cudaFree(d_x);
4610 cudaFree(d_y);
4611 }
4612
4613
4614 static void test_mat_transpose(int row, int col) {
4615
4616     size_t size_in = (size_t)row * col * sizeof(float);
4617     size_t size_out = (size_t)col * row * sizeof(float);
4618
4619     float *h_x = (float *)malloc(size_in);
4620     float *h_y_ref = (float *)malloc(size_out);
4621
4622     srand(42);
4623     for (int i = 0; i < row * col; i++)
4624         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4625
4626     // CPU reference: y[j][i] = x[i][j] (row-major)
4627     for (int i = 0; i < row; i++)
4628         for (int j = 0; j < col; j++)
4629             h_y_ref[j * row + i] = h_x[i * col + j];
4630
4631     float *d_x, *d_y;
4632     check(cudaMalloc(&d_x, size_in), "mattrans alloc X");
4633     check(cudaMalloc(&d_y, size_out), "mattrans alloc Y");
4634     check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans H2D");
4635
4636     dim3 block(16, 16);
4637     dim3 grid((col + 15) / 16, (row + 15) / 16);
4638     mat_transpose<<<grid, block>>>(d_x, d_y, row, col);
4639     check(cudaGetLastError(), "mattrans launch");
4640     check(cudaDeviceSynchronize(), "mattrans sync");
4641
4642     float *h_y = (float *)malloc(size_out);
4643     check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans D2H");
4644
4645     float max_err = 0.0f;
4646     for (int i = 0; i < col * row; i++) {
4647         float err = fabsf(h_y[i] - h_y_ref[i]);
4648         if (err > max_err) max_err = err;
4649     }
4650     printf("| %-35s | %.6e | %-4s |\n", "MatTranspose", max_err, max_err < 1e-6f ? "PASS" : "FAIL");
4651
4652     free(h_x);
4653     free(h_y);
4654     free(h_y_ref);
4655     cudaFree(d_x);
4656     cudaFree(d_y);
4657 }
4658
4659
4660 static void test_mat_transpose_padded(int row, int col) {
4661

```

```

4662 size_t size_in = (size_t)row * col * sizeof(float);
4663 size_t size_out = (size_t)col * row * sizeof(float);
4664
4665 float *h_x = (float *)malloc(size_in);
4666 float *h_y_ref = (float *)malloc(size_out);
4667
4668 srand(42);
4669 for (int i = 0; i < row * col; i++)
4670     h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4671
4672 // CPU reference: y[j][i] = x[i][j] (row-major)
4673 for (int i = 0; i < row; i++)
4674     for (int j = 0; j < col; j++)
4675         h_y_ref[j * row + i] = h_x[i * col + j];
4676
4677 float *d_x, *d_y;
4678 check(cudaMalloc(&d_x, size_in), "mattrans_padded alloc X");
4679 check(cudaMalloc(&d_y, size_out), "mattrans_padded alloc Y");
4680 check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans_padded H2D");
4681
4682 dim3 block(16, 16);
4683 dim3 grid((col + 15) / 16, (row + 63) / 64);
4684 mat_transpose_padded<<<grid, block>>>(d_x, d_y, row, col);
4685 check(cudaGetLastError(), "mattrans_padded launch");
4686 check(cudaDeviceSynchronize(), "mattrans_padded sync");
4687
4688 float *h_y = (float *)malloc(size_out);
4689 check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans_padded D2H");
4690
4691 float max_err = 0.0f;
4692 for (int i = 0; i < col * row; i++) {
4693     float err = fabsf(h_y[i] - h_y_ref[i]);
4694     if (err > max_err) max_err = err;
4695 }
4696 printf("| %-35s | %.6e | %-4s |\n", "MatTransposePadded", max_err, max_err < 1e-6f ? "PASS" : "FAIL");
4697
4698 free(h_x);
4699 free(h_y);
4700 free(h_y_ref);
4701 cudaFree(d_x);
4702 cudaFree(d_y);
4703 }
4704
4705
4706 static void test_sgemv(int M, int K) {
4707
4708     srand(42);
4709     float *h_a = (float *)malloc((size_t)M * K * sizeof(float));
4710     float *h_x = (float *)malloc((size_t)K * sizeof(float));
4711     float *h_y_ref = (float *)malloc((size_t)M * sizeof(float));
4712     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4713     for (int i = 0; i < K; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4714
4715     // CPU reference: y[m] = sum_k A[m,k] * x[k]
4716     for (int m = 0; m < M; m++) {
4717         double sum = 0.0;
4718         for (int k = 0; k < K; k++) sum += (double)h_a[m * K + k] * (double)h_x[k];
4719         h_y_ref[m] = (float)sum;
4720     }
4721
4722     float *d_a, *d_x, *d_y;
4723     check(cudaMalloc(&d_a, (size_t)M * K * sizeof(float)), "sgemv alloc A");
4724     check(cudaMalloc(&d_x, (size_t)K * sizeof(float)), "sgemv alloc X");

```

```

4725 check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv alloc Y");
4726
4727 check(cudaMemcpy(d_a, h_a, (size_t)M * K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D A");
4728 check(cudaMemcpy(d_x, h_x, (size_t)K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D X");
4729
4730 dim3 block(32, 4);
4731 dim3 grid((M + 3) / 4);
4732 sgemv_k128<<<grid, block>>>(d_a, d_x, d_y, M, K);
4733 check(cudaGetLastError(), "sgemv launch");
4734 check(cudaDeviceSynchronize(), "sgemv sync");
4735
4736 float *h_y = (float *)malloc((size_t)M * sizeof(float));
4737 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv D2H");
4738
4739 float max_err = 0.0f;
4740 for (int m = 0; m < M; m++) {
4741     float err = fabsf(h_y[m] - h_y_ref[m]);
4742     if (err > max_err) max_err = err;
4743 }
4744 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K128", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4745
4746 // ---- SGEMV K32 ----
4747 check(cudaMemset(d_y, 0, M * sizeof(float)), "sgemv_k32 zero Y");
4748 sgemv_k32<<<grid, block>>>(d_a, d_x, d_y, M, K);
4749 check(cudaGetLastError(), "sgemv_k32 launch");
4750 check(cudaDeviceSynchronize(), "sgemv_k32 sync");
4751
4752 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k32 D2H");
4753
4754 max_err = 0.0f;
4755 for (int m = 0; m < M; m++) {
4756     float err = fabsf(h_y[m] - h_y_ref[m]);
4757     if (err > max_err) max_err = err;
4758 }
4759 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K32", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4760
4761 // ---- SGEMV K16 ----
4762 free(h_a); free(h_x); free(h_y); free(h_y_ref);
4763 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
4764
4765 int K16 = 16;
4766 h_a = (float *)malloc((size_t)M * K16 * sizeof(float));
4767 h_x = (float *)malloc((size_t)K16 * sizeof(float));
4768 h_y_ref = (float *)malloc((size_t)M * sizeof(float));
4769 for (int i = 0; i < M * K16; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4770 for (int i = 0; i < K16; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4771
4772 for (int m = 0; m < M; m++) {
4773     double sum = 0.0;
4774     for (int k = 0; k < K16; k++) sum += (double)h_a[m * K16 + k] * (double)h_x[k];
4775     h_y_ref[m] = (float)sum;
4776 }
4777
4778 check(cudaMalloc(&d_a, (size_t)M * K16 * sizeof(float)), "sgemv_k16 alloc A");
4779 check(cudaMalloc(&d_x, (size_t)K16 * sizeof(float)), "sgemv_k16 alloc X");
4780 check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv_k16 alloc Y");
4781
4782 check(cudaMemcpy(d_a, h_a, (size_t)M * K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D A");
4783 check(cudaMemcpy(d_x, h_x, (size_t)K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D X");
4784
4785 dim3 grid_k16((M + 7) / 8);
4786 sgemv_k16<2><<<grid_k16, block>>>(d_a, d_x, d_y, M, K16);
4787 check(cudaGetLastError(), "sgemv_k16 launch");

```

```

4788 check(cudaDeviceSynchronize(), "sgemv_k16 sync");
4789
4790 h_y = (float *)malloc((size_t)M * sizeof(float));
4791 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k16 D2H");
4792
4793 max_err = 0.0f;
4794 for (int m = 0; m < M; m++) {
4795     float err = fabsf(h_y[m] - h_y_ref[m]);
4796     if (err > max_err) max_err = err;
4797 }
4798 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K16", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4799
4800 free(h_a); free(h_x); free(h_y); free(h_y_ref);
4801 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
4802 }
4803
4804
4805 static void test_sgemm(int M, int N, int K) {
4806
4807     size_t size_a = (size_t)M * K * sizeof(float);
4808     size_t size_b = (size_t)K * N * sizeof(float);
4809     size_t size_c = (size_t)M * N * sizeof(float);
4810
4811     float *h_a = (float *)malloc(size_a);
4812     float *h_b = (float *)malloc(size_b);
4813     float *h_c_ref = (float *)malloc(size_c);
4814
4815     srand(42);
4816     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4817     for (int i = 0; i < K * N; i++) h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4818
4819     float *d_a, *d_b, *d_c;
4820     check(cudaMalloc(&d_a, size_a), "sgemm alloc A");
4821     check(cudaMalloc(&d_b, size_b), "sgemm alloc B");
4822     check(cudaMalloc(&d_c, size_c), "sgemm alloc C");
4823
4824     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "sgemm H2D A");
4825     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "sgemm H2D B");
4826
4827     // cuBLAS reference (row-major idiom: swap M/N, swap A/B)
4828     cublasHandle_t handle;
4829     cublasCreate(&handle);
4830     float alpha = 1.0f, beta = 0.0f;
4831     cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
4832                 &alpha, d_b, N, d_a, K, &beta, d_c, N);
4833     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H ref");
4834
4835     // Kernel
4836     cudaEvent_t start, stop;
4837     cudaEventCreate(&start);
4838     cudaEventCreate(&stop);
4839
4840     dim3 block(32, 32);
4841     dim3 grid((N + 31) / 32, (M + 31) / 32);
4842     sgemm<<<grid, block>>>(d_a, d_b, d_c, M, N, K);
4843     check(cudaGetLastError(), "sgemm launch");
4844     check(cudaDeviceSynchronize(), "sgemm sync");
4845
4846     float *h_c = (float *)malloc(size_c);
4847     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H");
4848
4849     // Verify
4850     float max_err = 0.0f;

```

```

4851 for (int i = 0; i < M * N; i++) {
4852     float err = fabsf(h_c[i] - h_c_ref[i]);
4853     if (err > max_err) max_err = err;
4854 }
4855 printf("| %-35s | %.6e | %-4s |\n", "SGEMM", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4856
4857 // ---- SGEMM Vec4 (128x128 tile, 4x4 thread tile) ----
4858
4859 dim3 grid_vec4((N + 127) / 128, (M + 127) / 128);
4860 check(cudaMemset(d_c, 0, size_c), "sgemm_vec4 zero C");
4861 sgemm_vec4<<<grid_vec4, block>>>(d_a, d_b, d_c, M, N, K);
4862 check(cudaGetLastError(), "sgemm_vec4 launch");
4863 check(cudaDeviceSynchronize(), "sgemm_vec4 sync");
4864
4865 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm_vec4 D2H");
4866
4867 max_err = 0.0f;
4868 for (int i = 0; i < M * N; i++) {
4869     float err = fabsf(h_c[i] - h_c_ref[i]);
4870     if (err > max_err) max_err = err;
4871 }
4872 printf("| %-35s | %.6e | %-4s |\n", "SGEMM-Vec4", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4873
4874 free(h_a); free(h_b); free(h_c); free(h_c_ref);
4875 cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
4876 cublasDestroy(handle);
4877 cudaEventDestroy(start);
4878 cudaEventDestroy(stop);
4879 }
4880
4881
4882 static void test_hgemmm_mma(int M, int N, int K) {
4883
4884     size_t size_a = (size_t)M * K * sizeof(half);
4885     size_t size_b = (size_t)K * N * sizeof(half);
4886     size_t size_c = (size_t)M * N * sizeof(half);
4887
4888     half *h_a = (half *)malloc(size_a);
4889     half *h_b = (half *)malloc(size_b);
4890     half *h_c_ref = (half *)malloc(size_c);
4891
4892     srand(42);
4893     for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4894     for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4895
4896     // Kernel expects B^T [NxK] row-major layout (TN layout convention).
4897     // We store h_b as B [KxN] row-major for cuBLAS, then create B^T for kernel.
4898     size_t size_b_t = (size_t)N * K * sizeof(half);
4899     half *h_b_t = (half *)malloc(size_b_t);
4900     for (int n = 0; n < N; n++)
4901         for (int k = 0; k < K; k++)
4902             h_b_t[n * K + k] = h_b[k * N + n];
4903
4904     half *d_a, *d_b, *d_b_t, *d_c;
4905     check(cudaMalloc(&d_a, size_a), "hgemmm alloc A");
4906     check(cudaMalloc(&d_b, size_b), "hgemmm alloc B (cuBLAS)");
4907     check(cudaMalloc(&d_b_t, size_b_t), "hgemmm alloc B_t (kernel)");
4908     check(cudaMalloc(&d_c, size_c), "hgemmm alloc C");
4909
4910     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemmm H2D A");
4911     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemmm H2D B (cuBLAS)");
4912     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemmm H2D B_t (kernel)");
4913

```

```

4914 // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
4915 // Note: use CUBLAS_COMPUTE_16F to match the kernel's f16.f16.f16.f16 accumulation.
4916 cublasHandle_t handle;
4917 cublasCreate(&handle);
4918 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
4919 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
4920             &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
4921             &beta_h, d_c, CUDA_R_16F, N,
4922             CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
4923 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H ref");
4924
4925 // MMA kernel (TN layout: A row-major, B^T row-major = B col-major)
4926 cudaEvent_t start, stop;
4927 cudaEventCreate(&start);
4928 cudaEventCreate(&stop);
4929
4930 constexpr int BM = 128, BN = 128, BK = 16, kStages = 3;
4931 size_t smem_bytes = kStages * (BM * BK + BN * BK) * sizeof(half); // 24576
4932 dim3 block(256);
4933 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
4934 hgemm_mma_stages_tn<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
4935 check(cudaGetLastError(), "hgemm launch");
4936 check(cudaDeviceSynchronize(), "hgemm sync");
4937
4938 half *h_c = (half *)malloc(size_c);
4939 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H");
4940
4941 // Verify
4942 float max_err = 0.0f;
4943 for (int i = 0; i < M * N; i++) {
4944     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
4945     if (err > max_err) max_err = err;
4946 }
4947 printf("| %-35s | %.6e | %-4s |\n", "HGEMM MMA", max_err, max_err < 1.0f ? "PASS" : "FAIL");
4948
4949 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
4950 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
4951 cublasDestroy(handle);
4952 cudaEventDestroy(start);
4953 cudaEventDestroy(stop);
4954 }
4955
4956
4957 static void test_hgemm_swizzle(int M, int N, int K) {
4958     // HGEMM MMA Swizzle — m16n8k16 + multistage pipeline + TN 布局 + XOR swizzle
4959     // + Register Double Buffering (kValTileK=2, BK=32)
4960     // TN layout: C[M×N] = A[M×K] × B^T[N×K]
4961     // Kernel: hgemm_mma_stages_tn_swizzle with default template params
4962     // (kValTileK=2, kStages=3, BK=32)
4963     // smem: kStages × (BM×BK + BN×BK) halves
4964
4965     size_t size_a = (size_t)M * K * sizeof(half);
4966     size_t size_b = (size_t)K * N * sizeof(half);
4967     size_t size_c = (size_t)M * N * sizeof(half);
4968
4969     half *h_a = (half *)malloc(size_a);
4970     half *h_b = (half *)malloc(size_b);
4971     half *h_c_ref = (half *)malloc(size_c);
4972
4973     srand(42);
4974     for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4975     for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4976

```

```

4977 // Kernel expects B^T [N×K] row-major layout (TN layout convention).
4978 size_t size_b_t = (size_t)N * K * sizeof(half);
4979 half *h_b_t = (half *)malloc(size_b_t);
4980 for (int n = 0; n < N; n++)
4981     for (int k = 0; k < K; k++)
4982         h_b_t[n * K + k] = h_b[k * N + n];
4983
4984 half *d_a, *d_b, *d_b_t, *d_c;
4985 check(cudaMalloc(&d_a, size_a), "hgemm_swizzle alloc A");
4986 check(cudaMalloc(&d_b, size_b), "hgemm_swizzle alloc B (cuBLAS)");
4987 check(cudaMalloc(&d_b_t, size_b_t), "hgemm_swizzle alloc B_t (kernel)");
4988 check(cudaMalloc(&d_c, size_c), "hgemm_swizzle alloc C");
4989
4990 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm_swizzle H2D A");
4991 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm_swizzle H2D B (cuBLAS)");
4992 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm_swizzle H2D B_t (kernel)");
4993
4994 // cuBLAS FP16 reference
4995 cublasHandle_t handle;
4996 cublasCreate(&handle);
4997 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
4998 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
4999             &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
5000             &beta_h, d_c, CUDA_R_16F, N,
5001             CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
5002 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_swizzle D2H ref");
5003
5004 // MMA swizzle kernel (default params: kStages=3, kValTileK=2, BK=32)
5005 constexpr int BM = 128, BN = 128, BK = 32, K_STAGE_S = 3;
5006 size_t smem_bytes = K_STAGE_S * (BM * BK + BN * BK) * sizeof(half);
5007 dim3 block(256);
5008 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
5009 hgemm_mma_stages_tn_swizzle<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
5010 check(cudaGetLastError(), "hgemm_swizzle launch");
5011 check(cudaDeviceSynchronize(), "hgemm_swizzle sync");
5012
5013 half *h_c = (half *)malloc(size_c);
5014 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_swizzle D2H");
5015
5016 // Verify
5017 float max_err = 0.0f;
5018 for (int i = 0; i < M * N; i++) {
5019     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
5020     if (err > max_err) max_err = err;
5021 }
5022 printf("| %-35s | %.6e | %-4s |\n", "HGEMM Swizzle + Reg2x", max_err, max_err < 1.0f ? "PASS" : "FAIL");
5023
5024 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
5025 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
5026 cublasDestroy(handle);
5027 }
5028
5029
5030 #if defined(NOTES_V2_ENABLE_CUTE)
5031 static void test_hgemm_cute(int M, int N, int K) {
5032     // HGEMM CuTe — SM80_16x8x16_F16F16F16F16_TN + Swizzle<3,3,3> + kStage=2
5033     // TN layout: C[M×N] = A[M×K] × B^T[N×K]
5034     // Kernel: hgemm_mma_stages_tn_cute via launch_hgemm_mma_stages_tn_cute
5035     // Tile: BM=128, BN=256, BK=32, 128 threads/block
5036
5037     size_t size_a = (size_t)M * K * sizeof(half);
5038     size_t size_b = (size_t)K * N * sizeof(half);
5039     size_t size_c = (size_t)M * N * sizeof(half);

```

```

5040
5041 half *h_a = (half *)malloc(size_a);
5042 half *h_b = (half *)malloc(size_b);
5043 half *h_c_ref = (half *)malloc(size_c);
5044
5045 srand(42);
5046 for (int i = 0; i < M * K; i++)
5047     h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5048 for (int i = 0; i < K * N; i++)
5049     h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5050
5051 // CuTe kernel expects B^T [N×K] row-major (TN layout).
5052 size_t size_b_t = (size_t)N * K * sizeof(half);
5053 half *h_b_t = (half *)malloc(size_b_t);
5054 for (int n = 0; n < N; n++)
5055     for (int k = 0; k < K; k++)
5056         h_b_t[n * K + k] = h_b[k * N + n];
5057
5058 half *d_a, *d_b, *d_b_t, *d_c;
5059 check(cudaMalloc(&d_a, size_a), "hgemm_cute alloc A");
5060 check(cudaMalloc(&d_b, size_b), "hgemm_cute alloc B (cuBLAS)");
5061 check(cudaMalloc(&d_b_t, size_b_t), "hgemm_cute alloc B_t (kernel)");
5062 check(cudaMalloc(&d_c, size_c), "hgemm_cute alloc C");
5063
5064 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm_cute H2D A");
5065 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm_cute H2D B (cuBLAS)");
5066 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm_cute H2D B_t (kernel)");
5067
5068 // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
5069 cublasHandle_t handle;
5070 cublasCreate(&handle);
5071 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
5072 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
5073             CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
5074             CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
5075 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_cute D2H ref");
5076
5077 // CuTe kernel (Stages=2, TN layout)
5078 launch_hgemm_mma_stages_tn_cute<half, 2>(d_a, d_b_t, d_c, M, N, K);
5079 check(cudaGetLastError(), "hgemm_cute launch");
5080 check(cudaDeviceSynchronize(), "hgemm_cute sync");
5081
5082 half *h_c = (half *)malloc(size_c);
5083 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_cute D2H");
5084
5085 // Verify
5086 float max_err = 0.0f;
5087 for (int i = 0; i < M * N; i++) {
5088     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
5089     if (err > max_err) max_err = err;
5090 }
5091 printf("| %-35s | %.6e | %-4s |\n", "HGEMM CuTe", max_err, max_err < 1.0f ? "PASS" : "FAIL");
5092
5093 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
5094 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
5095 cublasDestroy(handle);
5096 }
5097 #endif /* NOTES_V2_ENABLE_CUTE */
5098
5099
5100 #if defined(NOTES_V2_ENABLE_WGMMA)
5101 static void test_hgemm_wgmma(int M, int N, int K) {
5102     // HGEMM WGMMA — m64n128k16 + TMA + Warp Specialization (Hopper SM90+)

```

```

5103 // TN layout:  $C[M \times N] = A[M \times K] \times B^T[N \times K]$ 
5104 // Kernel: hgemm_wgmma_stages_tn with default template params
5105
5106 constexpr int BM = 128, BN = 128, BK = 64, kStages = 3, kNumThreads = 256;
5107
5108 // M, K must be divisible by tile dims
5109 if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
5110     printf("| %-35s | %-12s | %-4s |\n",
5111           "HGEMM WGMMMA", "SKIP", "SKIP");
5112     return;
5113 }
5114
5115 size_t size_a = (size_t)M * K * sizeof(half);
5116 size_t size_b = (size_t)K * N * sizeof(half);
5117 size_t size_c = (size_t)M * N * sizeof(half);
5118
5119 half *h_a = (half *)malloc(size_a);
5120 half *h_b = (half *)malloc(size_b);
5121 half *h_c_ref = (half *)malloc(size_c);
5122
5123 srand(42);
5124 for (int i = 0; i < M * K; i++)
5125     h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5126 for (int i = 0; i < K * N; i++)
5127     h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5128
5129 //  $B^T [N \times K]$  row-major for TN layout (same as hgemm_mma kernel)
5130 size_t size_b_t = (size_t)N * K * sizeof(half);
5131 half *h_b_t = (half *)malloc(size_b_t);
5132 for (int n = 0; n < N; n++)
5133     for (int k = 0; k < K; k++)
5134         h_b_t[n * K + k] = h_b[k * N + n];
5135
5136 half *d_a, *d_b, *d_b_t, *d_c;
5137 check(cudaMalloc(&d_a, size_a), "wgmma alloc A");
5138 check(cudaMalloc(&d_b, size_b), "wgmma alloc B (cuBLAS)");
5139 check(cudaMalloc(&d_b_t, size_b_t), "wgmma alloc B_t (kernel)");
5140 check(cudaMalloc(&d_c, size_c), "wgmma alloc C");
5141
5142 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "wgmma H2D A");
5143 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice),
5144       "wgmma H2D B (cuBLAS)");
5145 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
5146       "wgmma H2D B_t (kernel)");
5147
5148 // cuBLAS FP16 reference (row-major idiom, same as hgemm_mma)
5149 cublasHandle_t handle;
5150 cublasCreate(&handle);
5151 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
5152 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
5153             CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
5154             CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
5155 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
5156       "wgmma D2H ref");
5157
5158 // Create TMA tensor maps for A and  $B^T$ 
5159 //  $A[M \times K]$  row-major: TMA box=(BK=64, BM=128), global shape=(K, M)
5160 //  $B^T[N \times K]$  row-major: TMA box=(BK=64, BN=128), global shape=(K, N)
5161 CUTensorMap *tma_a =
5162     allocate_and_create_tensor_map(d_a, M / BM, K / BK);
5163 CUTensorMap *tma_b =
5164     allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
5165

```

```

5166 // Launch WGMMMA kernel
5167 // kBlockSwizzle=false → 2D grid, no swizzle
5168 size_t smem_bytes =
5169     kStages * (BM * BK + BN * BK) * sizeof(half); // 3*16384*2 = 96KB
5170 cudaFuncSetAttribute(
5171     hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, kNumThreads, kStages,
5172     false>,
5173     cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
5174
5175 dim3 block(kNumThreads);
5176 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
5177 hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, kNumThreads, kStages, false>
5178     <<<grid, block, smem_bytes>>>(M, N, K, d_c, tma_a, tma_b);
5179 check(cudaGetLastError(), "wgmma launch");
5180 check(cudaDeviceSynchronize(), "wgmma sync");
5181
5182 half *h_c = (half *)malloc(size_c);
5183 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "wgmma D2H");
5184
5185 // Verify
5186 float max_err = 0.0f;
5187 for (int i = 0; i < M * N; i++) {
5188     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
5189     if (err > max_err)
5190         max_err = err;
5191 }
5192 printf("| %-35s | %.6e | %-4s |\n", "HGEMM WGMMMA", max_err,
5193     max_err < 1.0f ? "PASS" : "FAIL");
5194
5195 free(h_a);
5196 free(h_b);
5197 free(h_b_t);
5198 free(h_c);
5199 free(h_c_ref);
5200 cudaFree(d_a);
5201 cudaFree(d_b);
5202 cudaFree(d_b_t);
5203 cudaFree(d_c);
5204 cudaFree(tma_a);
5205 cudaFree(tma_b);
5206 cublasDestroy(handle);
5207 }
5208 #endif /* NOTES_V2_ENABLE_WGMMMA */
5209
5210
5211 static void test_flash_attn(int seqlen, int head_dim) {
5212     // FlashAttention-2 with split-Q, MMA m16n8k16
5213     int B = 1, H = 8;
5214
5215     size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
5216
5217     srand(42);
5218     half *h_q = (half *)malloc(sz);
5219     half *h_k = (half *)malloc(sz);
5220     half *h_v = (half *)malloc(sz);
5221     for (int i = 0; i < B * H * seqlen * head_dim; i++) {
5222         h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5223         h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5224         h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5225     }
5226
5227     // CPU reference in FP32: 0 = softmax(Q @ K^T / sqrt(d)) @ V
5228     float *ref_q = (float *)malloc(sz * 4 / sizeof(half)); // 4x for float

```

```

5229 float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
5230 float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
5231 float *ref_o = (float *)malloc(sz * 4 / sizeof(half));
5232 int count = B * H * seqlen * head_dim;
5233 for (int i = 0; i < count; i++) {
5234     ref_q[i] = __half2float(h_q[i]);
5235     ref_k[i] = __half2float(h_k[i]);
5236     ref_v[i] = __half2float(h_v[i]);
5237 }
5238
5239 float scale = 1.0f / sqrtf((float)head_dim);
5240 for (int bi = 0; bi < B * H; bi++) {
5241     for (int qi = 0; qi < seqlen; qi++) {
5242         // S[qi, kj] = Q[qi, :] @ K[kj, :]^T * scale
5243         float smax = -INFINITY;
5244         float *S = (float *)malloc((size_t)seqlen * sizeof(float));
5245         for (int kj = 0; kj < seqlen; kj++) {
5246             float s = 0.0f;
5247             for (int d = 0; d < head_dim; d++)
5248                 s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
5249                     ref_k[bi * seqlen * head_dim + kj * head_dim + d];
5250             S[kj] = s * scale;
5251             if (S[kj] > smax) smax = S[kj];
5252         }
5253         // softmax
5254         double sum_exp = 0.0;
5255         for (int kj = 0; kj < seqlen; kj++) sum_exp += (double)expf(S[kj] - smax);
5256         float inv_sum = 1.0f / (float)sum_exp;
5257         // O[qi, :] = sum_kj P[qi, kj] * V[kj, :]
5258         for (int d = 0; d < head_dim; d++) {
5259             double o_acc = 0.0;
5260             for (int kj = 0; kj < seqlen; kj++)
5261                 o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
5262                         ref_v[bi * seqlen * head_dim + kj * head_dim + d];
5263             ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
5264         }
5265         free(S);
5266     }
5267 }
5268
5269 half *d_q, *d_k, *d_v, *d_o;
5270 check(cudaMalloc(&d_q, sz), "fa alloc Q");
5271 check(cudaMalloc(&d_k, sz), "fa alloc K");
5272 check(cudaMalloc(&d_v, sz), "fa alloc V");
5273 check(cudaMalloc(&d_o, sz), "fa alloc O");
5274 check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "fa H2D Q");
5275 check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "fa H2D K");
5276 check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "fa H2D V");
5277
5278 // Template params for kHeadDim=64, kStage=2
5279 constexpr int kHeadDim = 64, kStageV = 2, kPadV = 8;
5280 constexpr int kMmaAtomM = 16, kMmaAtomN = 8, kMmaAtomK = 16;
5281 constexpr int kMmaTileSeqLenQ = 4;
5282 constexpr int kMmaTileSeqLenK = 1;
5283 constexpr int kMmaTileSeqLenP = 4;
5284 constexpr int kMmaTileHeadDimV = 1;
5285 constexpr int kValTileSeqLenQ = 1;
5286 constexpr int kValTileSeqLenK = 8;
5287 constexpr int kValTileSeqLenP = 1;
5288 constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
5289
5290 constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;
5291 constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;

```

```

5292 size_t smem_bytes = (Br * (kHeadDim + kPadV) +
5293                     kStageV * Bc * (kHeadDim + kPadV) +
5294                     Bc * (kHeadDim + kPadV)) * sizeof(half);
5295
5296 dim3 block(128);
5297 dim3 grid((seqlen + Br - 1) / Br, B * H);
5298
5299 flash_attn_mma_stages_split_q<kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK,
5300    kMmaTileSeqLenQ, kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV,
5301    kValTileSeqLenQ, kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV,
5302    kStageV, kPadV>
5303    <<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, head_dim);
5304 check(cudaGetLastError(), "fa launch");
5305 check(cudaDeviceSynchronize(), "fa sync");
5306
5307 half *h_o = (half *)malloc(sz);
5308 check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa D2H");
5309
5310 float max_err = 0.0f;
5311 for (int i = 0; i < count; i++) {
5312     float err = fabsf(__half2float(h_o[i]) - ref_o[i]);
5313     if (err > max_err) max_err = err;
5314 }
5315 printf("| %-35s | %.6e | %-4s |\n", "FlashAttn-SplitQ", max_err, max_err < 1e-1f ? "PASS" : "FAIL");
5316
5317 free(h_q); free(h_k); free(h_v); free(h_o);
5318 free(ref_q); free(ref_k); free(ref_v); free(ref_o);
5319 cudaFree(d_q); cudaFree(d_k); cudaFree(d_v); cudaFree(d_o);
5320 }
5321
5322
5323 int main(int argc, char *argv[]) {
5324 #if defined(NOTES_V2_ENABLE_WGMMMA)
5325     cuInit(0); // Driver API init required for cuTensorMapEncodeTiled (TMA, sm_90a+)
5326 #endif
5327     int M = 1024, N = 1024, K = 1024;
5328     if (argc > 3) { M = atoi(argv[1]); N = atoi(argv[2]); K = atoi(argv[3]); }
5329
5330     printf("=== notes-v2.cu verification harness ===\n");
5331     printf("| %-35s | %-12s | %-4s |\n", "Kernel", "Max Err", "Pass");
5332     printf("|-----|-----|-----|\n");
5333
5334     test_block_reduce(N);
5335     test_dot(N);
5336     test_relu(1024);
5337     test_elementwise(1024);
5338     test_histogram(1024);
5339     test_merge_attn_states(512, 16, 128);
5340     test_softmax(256);
5341     test_rms_norm(8, 128);
5342     test_layer_norm(8, 128);
5343     test_rope(8, 128);
5344     test_mat_transpose(256, 256);
5345     test_mat_transpose_padded(256, 256);
5346     test_sgemv(256, 128);
5347     test_sgemm(M, N, K);
5348     test_hgemm_mma(M, N, K);
5349     test_hgemm_swizzle(M, N, K);
5350 #if defined(NOTES_V2_ENABLE_CUTE)
5351     test_hgemm_cute(M, N, K);
5352 #endif
5353 #if defined(NOTES_V2_ENABLE_WGMMMA)
5354     test_hgemm_wgmma(M, N, K);

```

```

5355 #endif
5356     test_flash_attn(1024, 64);
5357
5358     printf("=== All tests done ===\n");
5359     return 0;
5360 }
5361
5362 // =====
5363 // Quick build & run reference
5364 // =====
5365 // # sm_86 + CuTe (Ampere, RTX 30/40 系列, 无 WGMMMA) :
5366 // nvcc -std=c++20 -O2 -arch=sm_86 -DNOTES_V2_ENABLE_CUTE \
5367 //     -I ../../third-party/cutlass/include \
5368 //     -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm86.bin
5369 //
5370 // # sm_89 单独编译 + 运行:
5371 // nvcc -std=c++20 -O2 -arch=sm_89 -lcublas -lcuda notes-v2.cu -o notes_v2_sm89.bin
5372 //
5373 // # sm_89 + CuTe (需要 CUTLASS include 路径, 编译 CUDA_HOME 指向的 CUTLASS 或
5374 //     项目内置 third-party/cutlass) :
5375 // nvcc -std=c++20 -O2 -arch=sm_89 -DNOTES_V2_ENABLE_CUTE \
5376 //     -I ../../third-party/cutlass/include \
5377 //     -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm89.bin
5378 //
5379 // # sm_90a 单独编译 + 运行 (需要 Hopper GPU, H100/H200 均可) :
5380 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a \
5381 //     -DNOTES_V2_ENABLE_WGMMMA -lcublas -lcuda notes-v2.cu -o notes_v2_sm90.bin
5382 //
5383 // # sm_90a + CuTe + WGMMMA:
5384 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a \
5385 //     -DNOTES_V2_ENABLE_CUTE -DNOTES_V2_ENABLE_WGMMMA \
5386 //     -I ../../third-party/cutlass/include \
5387 //     -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm90.bin

```